

A level Computer Science Programming Project

Candidate name: Archie Russell Pennycook

Candidate number: 1542

Centre name: South Wilts Grammar School

Centre number: 66735

Project Title: Solar System Simulation

Candidate statement of evidence.....	4
Coding styles	4
Techniques used.....	5
Analysis.....	5
Introduction.....	5
How a simulation of the solar system works	6
What teachers want in a simulation of the solar system	6
Current alternatives.....	9
NASA's Eyes on the Solar System.....	9
PhET: My Solar System.....	10
What I've learned from the alternatives	10
Modelling the problem.....	11
Class diagram	11
NASA's API.....	12
The maths of simulating a solar system.....	14
Verlet Integration	14
Gravity	14
Vectors.....	17
Displaying the system	17
Interface	20
Objectives.....	21
Documented design.....	23
Modules	23
pygame.....	23
horizonsParser, datetime & requests.....	23
getEphemeris.....	23
Getting the request.....	24
Parsing the request.....	25
extractValue.....	30
Finding the scale.....	32
Finding the value.....	32
math(s)	33
numpy.....	34
vectors.....	34
Magnitude of a vector	34
Finding a unit vector	35
Finding a normal vector	35

File Structure.....	35
Data dictionary.....	36
Classes.....	38
Satellite / planet.....	40
Overall system design.....	45
Initialisation.....	45
Adding the planets.....	45
Setting up pygame.....	48
System parts.....	49
Simulating a tick.....	49
Methods used.....	52
addGPE / addKE / addMomentum.....	53
gravity.....	53
secondLaw.....	54
verlet.....	54
Display.....	54
The start screen.....	56
Displaying the solar system.....	56
Methods used.....	60
offscreen.....	60
focusAdjustment.....	60
scaledPos.....	60
drawPlanet.....	61
displayPlanets.....	61
displayLines.....	62
drawArrow.....	63
displayArrows.....	65
Comparison mode.....	66
Interface.....	67
Menu.....	67
Inputs.....	70
left / right arrows.....	70
up / down arrows.....	70
space.....	70
F.....	70
A.....	
71.....	

W, S, E and D	71
Mousewheel up and down	71
Miscellaneous functions.....	72
com.....	72
sumPhysicalProperties	73
zoom.....	73
Technical Solution.....	75
horizonsParser.....	75
extractValue	75
getEphemeris.....	76
main.....	77
vectors	77
planets	78
Setup.....	78
Classes.....	79
Functions.....	83
generateSolarSystem.....	88
Setup 2.....	88
main.....	88
Testing.....	92
Evaluation.....	101
Overall evaluation.....	101
Optimisation.....	104
Evaluation by objective	104
References.....	106

Candidate statement of evidence

Coding styles

Style	Evidence page
Annotation used effectively where required	Throughout code, 74-90
Meaningful identifier names	Throughout code, 74-90
Consistent style throughout	
Self documenting code	Throughout code, 74-90
Appropriate indendation	Throughout code, 74-90
Use of constants	79, 88

Managed casting of types	76, 78, 86, 88
Minimal use of global variables	
Good use of local variables	Throughout code but see specifically the section on functions, 83-87
Modularisation	Throughout code, especially in section on functions 83-87
Well designed UI	See testing video and evaluation where client says she likes the interface
Good exception handling	See 91 where I implement a logarithmic scale to stop G and mass becoming negative
Modules – subroutines with common purpose grouped	
Cohesive modules	
Loosely coupled modules	
Modules with appropriate interfaces	

Techniques used

Technique	Evidence pages
Complex scientific model	Throughout code, it's a simulation of the solar system. See 82, 83, 87.
Hash tables, lists Of equivalent standard	Lists are used throughout the code, see the records system used to display orbital lines - 82, 84.
Complex OOP model	79-83
List operations	82, 84
Complex user defined algorithms	See simulateTick - 87, and the algorithm used to parse NASA data, 75-77
Dynamic generation of objects based on a complex OOP model	generateSolarSystem(), 88
Calling parameterised APIs and parsing JSON / XML	75-77
Multi-dimensional arrays	Throughout all the code
Dictionaries	75-77 used for parsing data with vectors
Single dimensional arrays	Throughout all the code
Appropriate choice of simple data types	Throughout all the code

Analysis

Introduction

The solar system is an important part of the physics curriculum, and it's something that children often find more interesting than other core physics topics such as circuits or mechanics. I think it's important to teach the solar system because it's fun, and because it can also help students understand the more boring topics, such as forces or energy, as forces and conservation of energy can be demonstrated with the solar system.

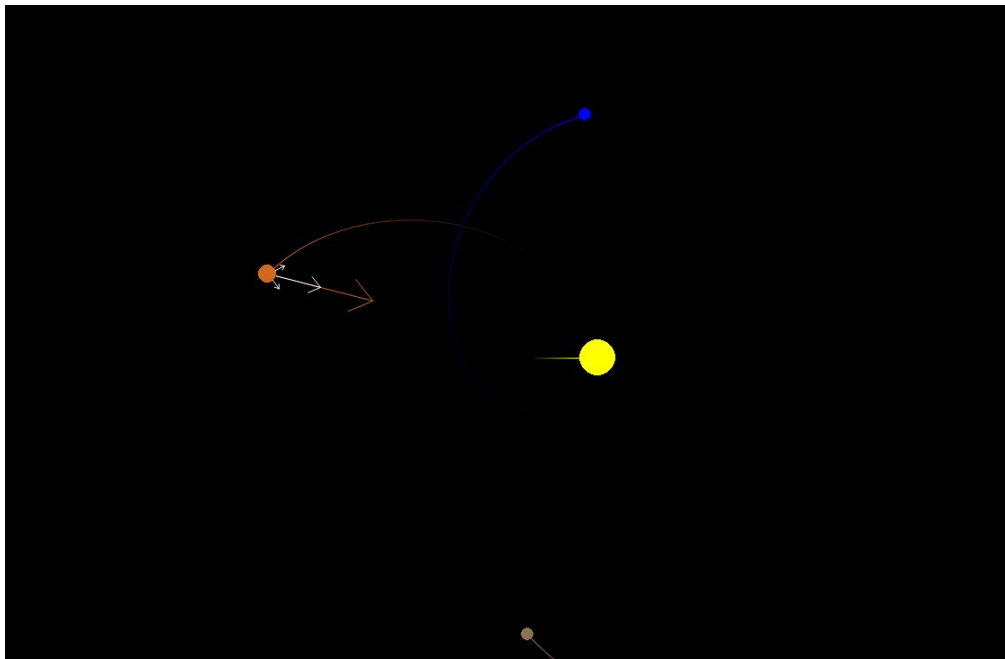
However, there is a lack of interactive tools available for teaching the solar system that also incorporate other physics topics such as energy. It would be useful to teachers, and students, to have a program that can use an interesting topic like space that also teaches important concepts from other topics such as conservation of momentum or Newton's laws.

How a simulation of the solar system works

The orbits of the planets are governed by two fundamental laws of physics:

- Newton’s second law of motion: $F = ma$, where F is the force acting on a body, m is the mass of a body, and a is the acceleration of the body
- Newton’s law of universal gravitation: $F = \frac{GMm}{r^2}\hat{r}$, once again F is the force, G is something called the “gravitation constant”, M and m represent the masses between two bodies, and r^2 is the distance between them.

To demonstrate this, a force diagram is attached.



The orange planet has 4 arrows around it. These are the forces acting on it. The arrow shows the direction of each force and the length of the arrow represents the strength of the force. The white arrows are individual forces, you can see one pointing towards the brown planet, one pointing towards the blue planet, and one pointing towards the yellow sun. The arrow pointing towards the sun is much longer than the other arrows, representing that it has a stronger effect. This is because the sun has a larger mass, and so the force of gravity due to it is a lot stronger.

The brown arrow is the resultant force. This is what happens when you pull the planet in the direction of all three arrows at the same time. Because the force from the sun is so much stronger than the force due to the other planets, the resultant force is pretty similar to the force from the sun.

What teachers want in a simulation of the solar system

These are extracts from an interview with the South Wilts physics department

Question:

“What content is covered when teaching lower school classes?”

Answer:

"We do a little bit of work with Y7 looking at the order of the planets in the Solar System and the properties of some of the planets.

"In Y11 we look at the objects in the solar system and orbital motion. They need to know that gravity provides a centripetal force and how closer planets orbit faster.

How this impacts the project:

I need to accurately display the solar system with the planets in the correct order, show the orbits of the planets, and have a way to show that gravity provides a centripetal force.

Question:

Do you currently use any interactive tools when teaching the solar system? If so, which, and how do you use them? Do you think there's anything they're lacking that would help kids learn better?

Answer:

Not with Y11.

Y7 have the opportunity to complete independent research on a planet in the solar system and we share this link <https://eyes.nasa.gov/apps/orrery/#/home>. It has easy links to extra information and a size comparison.

It would be good to see this combined with something that shows the orbital motion of satellites.

https://phet.colorado.edu/sims/html/my-solar-system/latest/my-solar-system_all.html

This simulator is great to show the effect of changing mass on the theoretical solar system. I wonder if there could be a simulator which enabled us to 'play' with the masses in our actual solar system to see what happens – probably this would be more useful Y11 onwards.

How this impacts the project:

I need to create something that shows the motion of planets, as well as having size comparisons, like in the link above. Should be possible to edit the masses of objects to show the effect it has too like in the PHeT simulator. I think allowing the user to change the value of G would be informative as well, to show what it actually does.

Question:

What other features would you want in an interactive program that would make it useful for teaching?

Answer:

With younger students it is good to be able to show them information when setting them independent tasks to avoid them just copying and pasting things they don't understand from incorrect sources.

Impact on project:

Planets, satellites and other celestial objects should display their properties (e.g. mass and size) so that the program can act as a source of information, rather than students finding erroneous sources online.

Question:

What format would be most convenient, e.g. would a website be preferable to an app you had to download?

Answer:

An executable file would work well

Question:

Should the program focus entirely on the solar system, or would it be useful to also use the orbits to demonstrate concepts like Newton's laws, conservation of energy and conservation of momentum?

Answer:

We look at the solar system right at the end of Y11 so if there was a program which linked to key Newtonian Physics, it would be a great way to re-visit these laws in a different context.

Impact on project:

Should include things such as force diagrams and total energy to link the solar system to these concepts.

A review of the features requested:

1. Display extra information in the program
2. Size comparison
3. Interactively changing the mass of the planets
4. Demonstrating concepts of Newtonian physics
5. Make the program downloadable e.g. a python file

Feature 1 can be implemented with an object oriented approach. There will be a celestial body class. Celestial bodies will be stored as objects. Their masses, positions and sizes will be stored

as attributes. I can just display those attributes (these attributes will be fetched from a NASA database as explained later, which is a reliable source.)

Feature 2 can be done by splitting the screen in half, displaying the chosen object in the left half, and scaling the left half so that it fits the chosen object. Then display the other chosen object for comparison in the right half at the same scale.

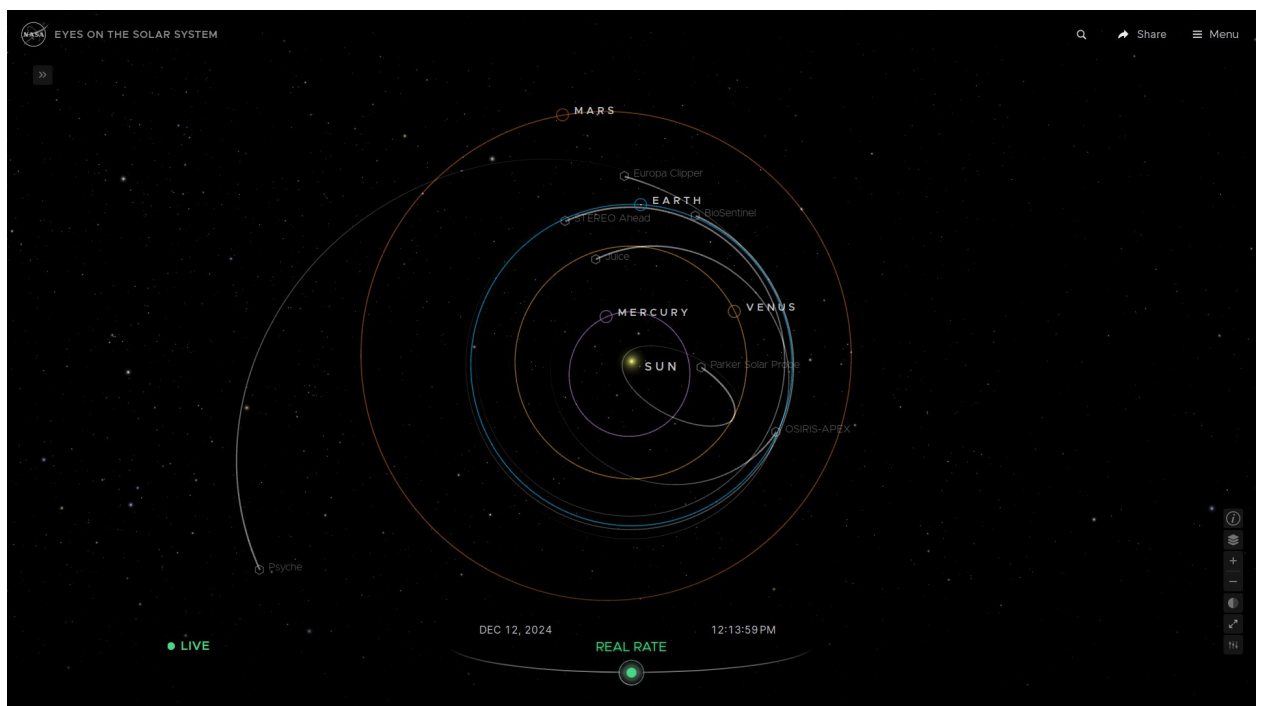
Feature 3 is doable. The mass of planets is already stored as an attribute for them. All that needs to be done is to let users edit the mass of said planets.

For feature 6 the challenge is figuring out how to do it in a way that a 12 year old can understand. The program already conserves momentum, energy, and uses Newton's laws of motion because it's supposed to be an accurate simulation. The hard bit is trying to make it clear to kids how those things are happening in the program, and help them get a better understanding of it. I will try displaying both the potential and kinetic energy of the system, as well as the total. This will show that while the types of energy change, the total amount of energy stays constant.

Current alternatives

NASA's Eyes on the Solar System

<https://eyes.nasa.gov/apps/solar-system/#/home>



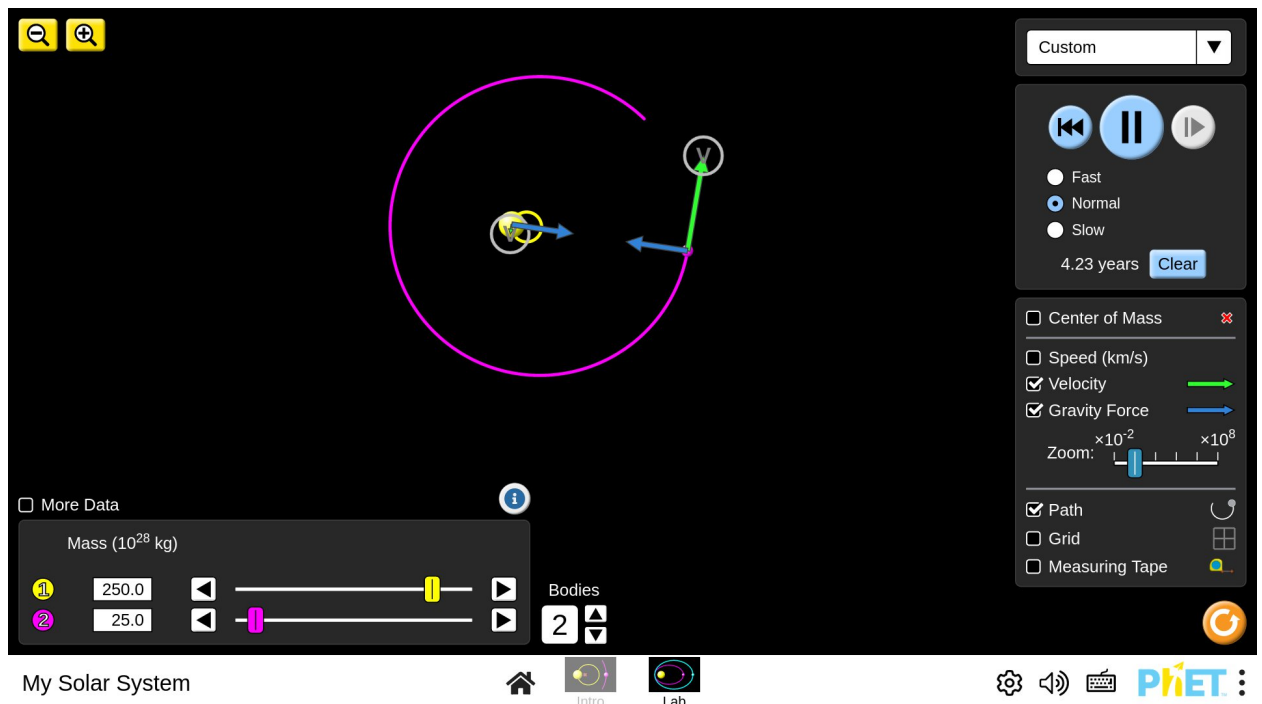
Eyes is a suite of products developed by NASA used for visualising their data. Their orrery, shown in the figure above, demonstrates the orbits of planets, asteroids, comets, and certain satellites. My school currently uses this to teach year 7's, primarily because it has links to further reading for their research project. It also accurately shows the trajectories of planets and other celestial bodies.

This site allows you to move the camera and to focus on specific planets, which is something I think is very beneficial for teaching as it gives you different perspectives of the solar system, and it is something I want in my own project.

However, the site lacks a few things. There is no connection to Newtonian mechanics, nothing demonstrating it, and no way to change the mass of the planets.

PhET: My Solar System

https://phet.colorado.edu/sims/html/my-solar-system/latest/my-solar-system_all.html



PhET (Physics Education Technology) is a website hosting a large amount of interactive physics simulations where students can tweak parameters and see the effects in real time. Its simulations are used by schools all around the world, and mine is no exception. As mentioned in the interview, my school finds the PhET simulation for the solar system useful because it lets you change the masses of the bodies in the solar system. It also includes force and velocity vectors, which can be used to help link back to Newtonian mechanics.

The drawback is that it's all on a fictional solar system. You can't put it into context with our actual planets. This would both be entertaining for students (who doesn't want to know what would happen if the Moon weighed as much as Jupiter?) and help them visualise the effects of changing mass.

What I've learned from the alternatives

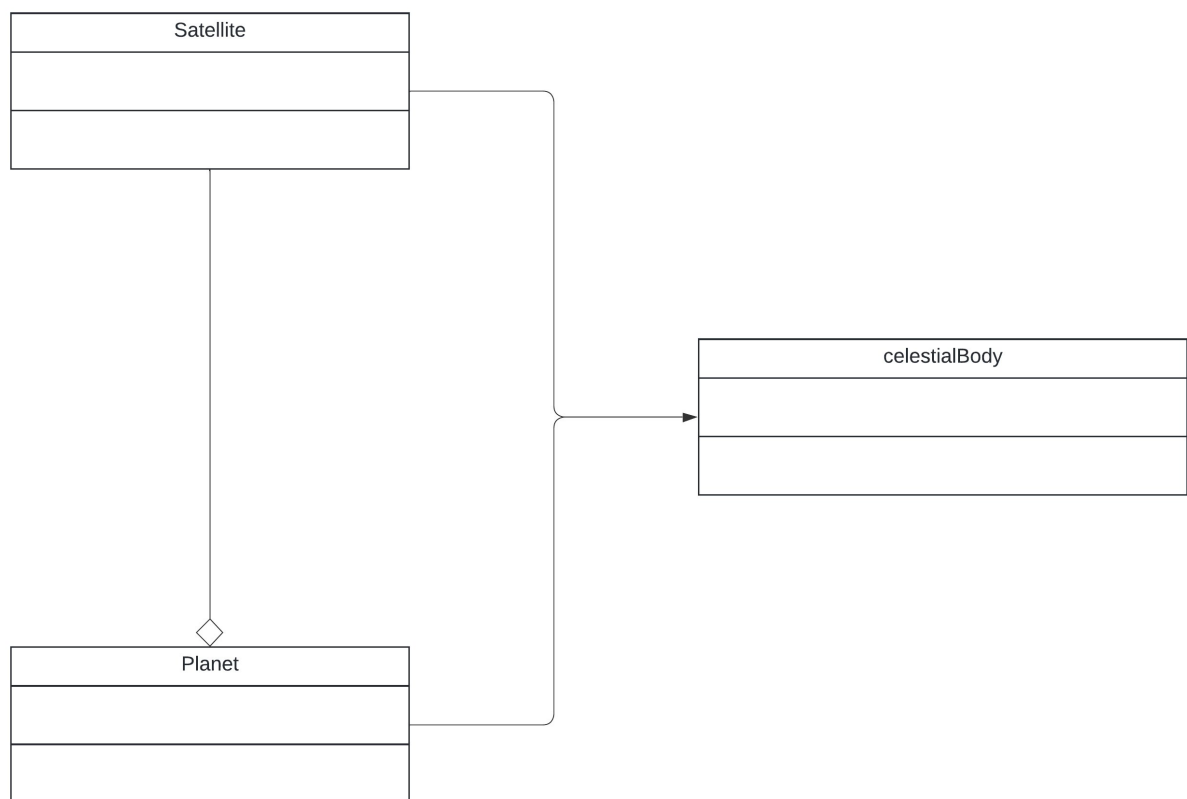
I aim to merge all the useful features in the alternatives listed above, and package them in an intuitive, easy to use software that benefits both teachers and students. This means being able to edit attributes of planets like in the PhET simulation, while also being accurate to our own solar system like NASA's Eyes.

Some general design principles / tips from looking at the alternatives:

- A downloadable program is the most accessible format for teachers
- Interactivity and being able to edit parts of the system is beneficial to learning and should be in the program
- The program should show the actual solar system, not a fictional one
- Arrows and other visual tools should be used to intuitively show physical quantities such as force and velocity

Modelling the problem

Class diagram



The main object in the solar system is a celestialBody, this is a catch all for pretty much every object in the system. Satellite and planet both inherit from celestialBody to tweak the behaviour slightly. The distinction between planet and satellite in the program is based on the astronomical definition, not the colloquial definition. A satellite is a body that orbits a host that isn't a star, while a planet is anything else. That's the only distinction, this means that the Sun and the Voyager satellites are classed as planets, because they do not have a host they orbit. Meanwhile the moon is a satellite because it orbits the Earth.

Satellites need to always orbit a host planet, and this host is one of their attributes, so they are aggregated with Planet. The Satellite's existence is not dependent on the planet, e.g. if the Earth disappeared today the moon would continue existing, so aggregation is used instead of composition.

The methods have been omitted for brevity, there are a lot that are required. They will be in the class diagram in the design.

NASA's API

The client requested the ability to change masses (simple), but for real planets in our solar system (less simple). This means I need to somehow have data on all of the planets, and to keep it up to date. Luckily, NASA has a solution for us:

<https://ssd.jpl.nasa.gov/horizons/>

Horizons is an online database that hosts real time information on planets. The idea will be to access this data using their API, which uses HTTP POST queries, and then translate that data to parameters for planets in my program, and load the solar system. Parsing the data may be tricky, but once I have the mass, positions and velocities of the planets it should be rather simple to load the solar system. This is because planets are already stored as objects with all of those attributes in my code, so I just need to create objects with their attributes set to be the ones retrieved from the Horizons query.

This is the request I will be using:

[https://ssd.jpl.nasa.gov/api/horizons.api?format=text&COMMAND='"+target+"'&OBJ_DATA='YES'&MAKE_EPHEM='YES'&EPHEM_TYPE='VECTORS'&START_TIME='"+startTime+"'&STOP_TIME='"+stopTime+"'&CENTER='500@0'&STEP_SIZE='1%20d'&QUANTITIES=''](https://ssd.jpl.nasa.gov/api/horizons.api?format=text&COMMAND=')

Target, startTime and stopTime represent variables that will be set by the program. The rest of the request will not change. Here is an explanation of the important settings I've used in the request:

EPHEM_TYPE: This has multiple options. However the other options are all intended for uses not relevant to my program, e.g. making telescope observations or planning space missions to other planets. VECTORS is the type that NASA recommends for programming on their own site:

https://ssd-api.jpl.nasa.gov/doc/horizons.html#ephem_type

So it's the one that I use.

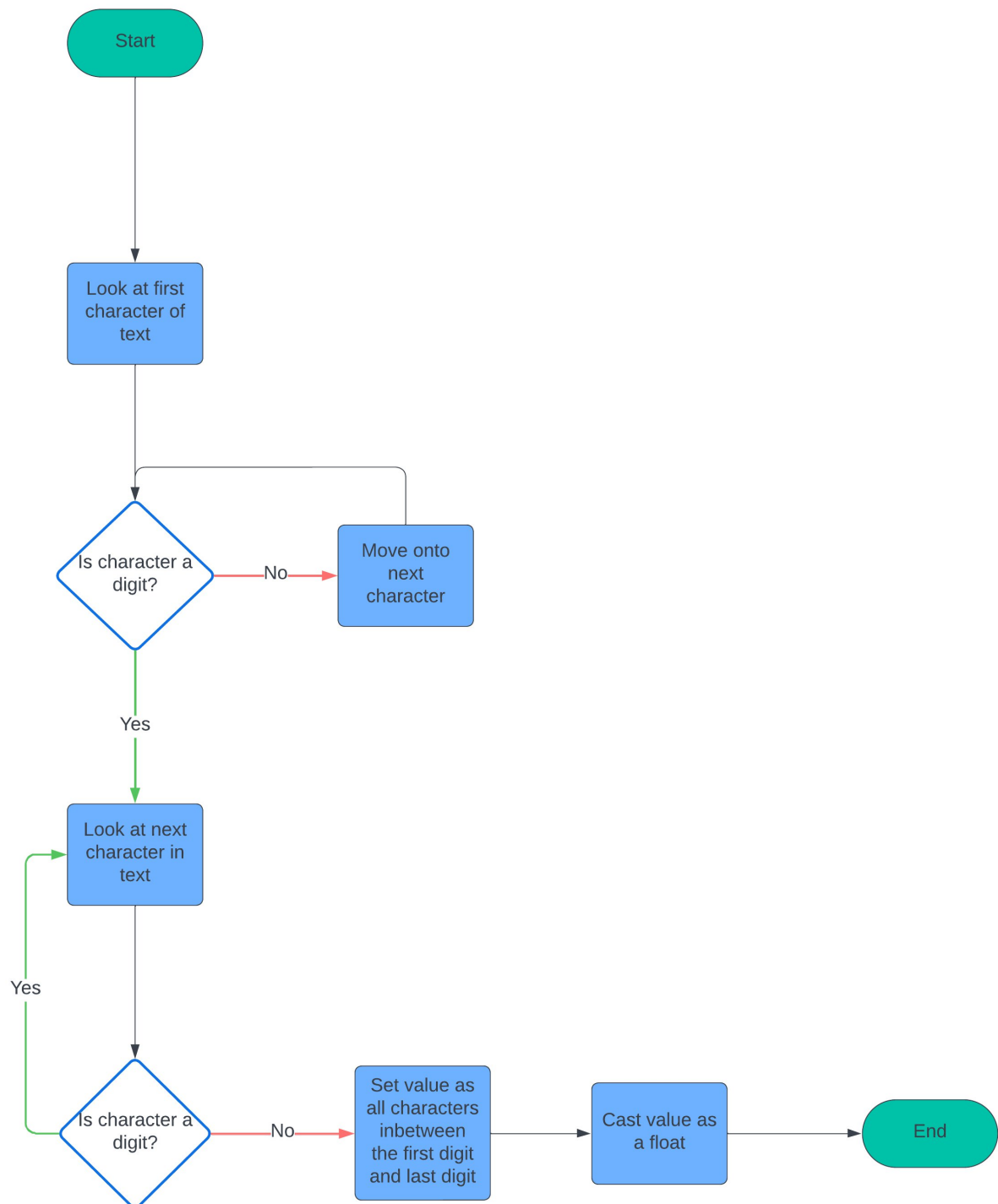
CENTER: This is the reference location for the coordinates returned by Ephemeris. 500@0 is the ID for the Solar Barycenter, the center of mass of the solar system. This is the most objective reference point available for our solar system, so is the one that I use.

STEP_SIZE: This just means the ephemeris generates coordinates every day, this isn't really relevant to the program as I'm only using the initial coordinates given in the response and simulating the rest from there.

QUANTITIES: This lets you request a bunch of very cool information, but none of it is needed by my program, so to make getting the request faster and to reduce the size of the requests it is set to be empty.

Once the request is received, it must be parsed so that the relevant parameters can be extracted. The basic strategy I'll use is to first search for key phrases in the text like "Mass" or

“Radius” which indicate that a parameter I want is about to come up. Once a key phrase is found, this algorithm will be used to get the numerical value out:



Because NASA formats their data horribly, there are a lot of caveats involved that will need to be overcome in order to successfully parse the data. This will be covered in the design.

The functions to request and extract data from NASA will be in a separate file, `horizonsParser`, so that it can be imported as a module.

The maths of simulating a solar system

Verlet Integration

Verlet integration is a computationally efficient method for simulating the motion of objects.

This is the equation for basic Verlet:

$$x_{n+1} = 2x_n - x_{n-1} + a_n \Delta t^2$$

This looks very scary, but isn't actually that bad. x_n is the current position of the object. x_{n-1} is its previous position. x_{n+1} is its updated position. a_n is the acceleration of the object. Δt^2 means a very very small step forward in time.

We can do some factoring to get a more useful form of the equation:

$$x_{n+1} = x_n + x_n - x_{n-1} + a_n \Delta t^2$$

$$x_{n+1} = x_n + (x_n - x_{n-1}) + a_n \Delta t^2$$

$$x_{n+1} = x_n + v_{n-1} \Delta t + a_n \Delta t^2$$

$$x_{n+1} = x_n + (v_{n-1} + a_n \Delta t) \Delta t$$

$$x_{n+1} = x_n + v_n \Delta t$$

This is the equation that will actually be used in code. What it means is that, given an object at position x_n , and with a certain speed v_n , its position x_{n+1} after moving forward a small amount of time (Δt) is its old position + its speed multiplied by that small amount of time. Example:

A car is 9 miles along a road (so $x_n = 9$). It is moving at 60 miles an hour. After 1 minute ($\Delta t = 1 \text{ min} = 1/60 \text{ hr}$), the new position of the car will be $x_{n+1} = 9 + 60 * \left(\frac{1}{60}\right) = 10$ miles along the road.

So, all we need to do to find the new positions of our planets after each tick, is add their current velocities multiplied by a time factor (Δt) to their current position.

There is also a similar equation for velocity and acceleration that will be used throughout the code:

$$v_{n+1} = v_n + a_n \Delta t$$

Where you add the acceleration to get the new velocity.

The Δt term lets us change the speed of the simulation. The larger it is the faster the simulation is, it might cover a year of orbits in a single second for instance, but also the larger Δt is the less accurate the simulation is. The user will be able to change Δt with keybinds to suit their own needs.

Gravity

As mentioned earlier, the two equations used to simulate gravity are:

$$F = ma$$

$$F = \frac{GMm}{r^2} \hat{r}$$

The goal here is simply to work out the acceleration on a planet due to gravity. Once that is done, the acceleration of the planet is handed over to the Verlet integration algorithm, which handles everything else.

Let's start with the simpler equation, $F = ma$. Here, F is the force, m is the mass of the object, and a is the acceleration. We will calculate the force using the other equation, $F = \frac{GMm}{r^2} \hat{r}$, and we already know the mass of all of our planets. We want the acceleration. So, solving for acceleration we get:

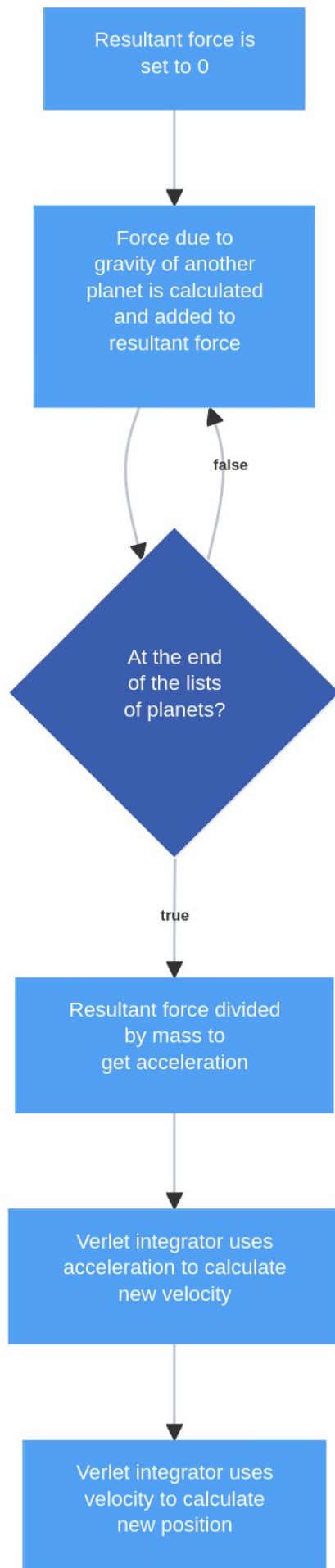
$$a = \frac{F}{m}$$

Which is quite nice. To find the acceleration, we just take the force acting on the planet and divide it by its mass.

Now let's cover $F = \frac{GMm}{r^2} \hat{r}$. Here, M and m are the masses of the two objects in question (M is conventionally the mass of the larger planet), r is the distance between them, $\hat{r}$ is a unit vector pointing from one planet to another, and G is a physical constant. It's important to note that the F here is NOT the same F as the one in $F = ma$. The F in $F = ma$ is the *resultant* force, the sum of all the forces acting on an object, while the F in this equation is just the force due to the gravitational pull of one planet.

This means to get the resultant force acting on a planet, we must compute $F = \frac{GMm}{r^2} \hat{r}$ for that planet and all the other bodies in the system. We then add those forces up (some may be negative and cancel each other out) to get the resultant force. This is then given to $F = ma$, and the acceleration from that is given to Verlet integration. Note that we must

To summarise everything covered in this section, a flow chart on the next page describes what happens to a single planet on each tick of the program:



Vectors

To use verlet integration and Newton's law of gravity we will need to have access to a few basic vector operations. These are:

1. Finding the magnitude of a vector
2. Finding the unit of a vector
3. Finding the perpendicular to a 2d vector

To find the magnitude of a vector you simply square each element, add them up, and then take the square root. E.g. if we have the vector (1, 2, 2) then the magnitude is $\sqrt{1^2+2^2+2^2}=\sqrt{1+4+4}=\sqrt{9}=3$. This is needed to find "r" in the gravity equation, as the distance between two planets is the magnitude of the vector between them.

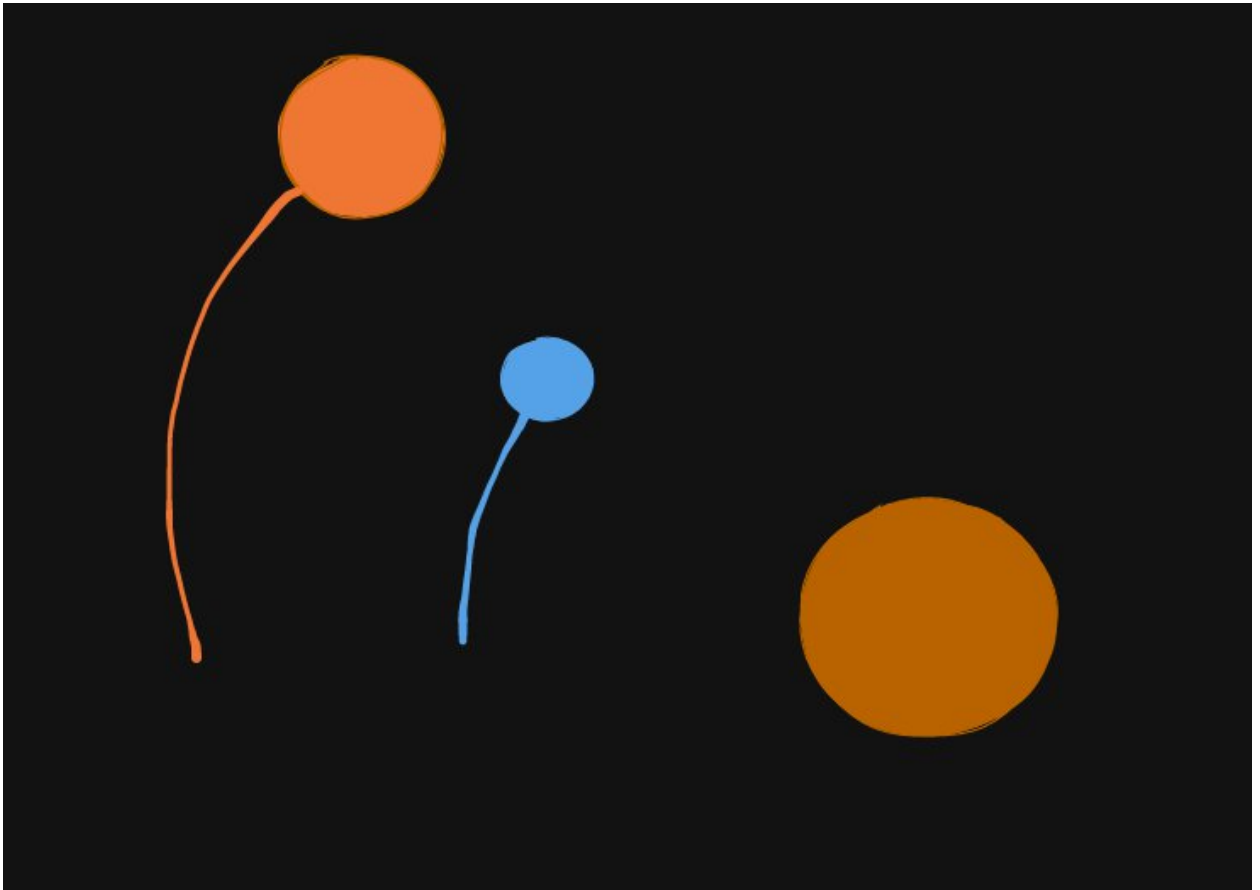
To find the unit of a vector is very simple, you find its magnitude and then divide every element by it. So the unit of (1, 2, 2) would be (1/3, 2/3, 2/3). This is needed to find $\hat{r}$ in the gravity equation.

To find a perpendicular to a 2d vector you swap the elements around and negate one of them. So e.g. the two perpendiculars to (1,2) would be (-2,1) and (2,-1). It is arbitrary which you negate so I will just negate the first element. This is needed to draw arrows, the two lines at the end of the arrow need to lie on a perpendicular to the main line of the arrow. This will be explained more in the design, drawing arrows is surprisingly technical.

These operations will be put in a separate file so they can be imported as a module.

Displaying the system

I will use a simplistic design philosophy for this project. I want to get the relevant information across: Relative sizes of planets, orbital paths, positions of planets. But I want to do this without impacting performance or making the display hard to understand. To do this, I will use solid coloured circles for the planets, as this makes it clear where they are, and thin lines to show their trajectories. This is a mockup of what the system would look like with a few planets:

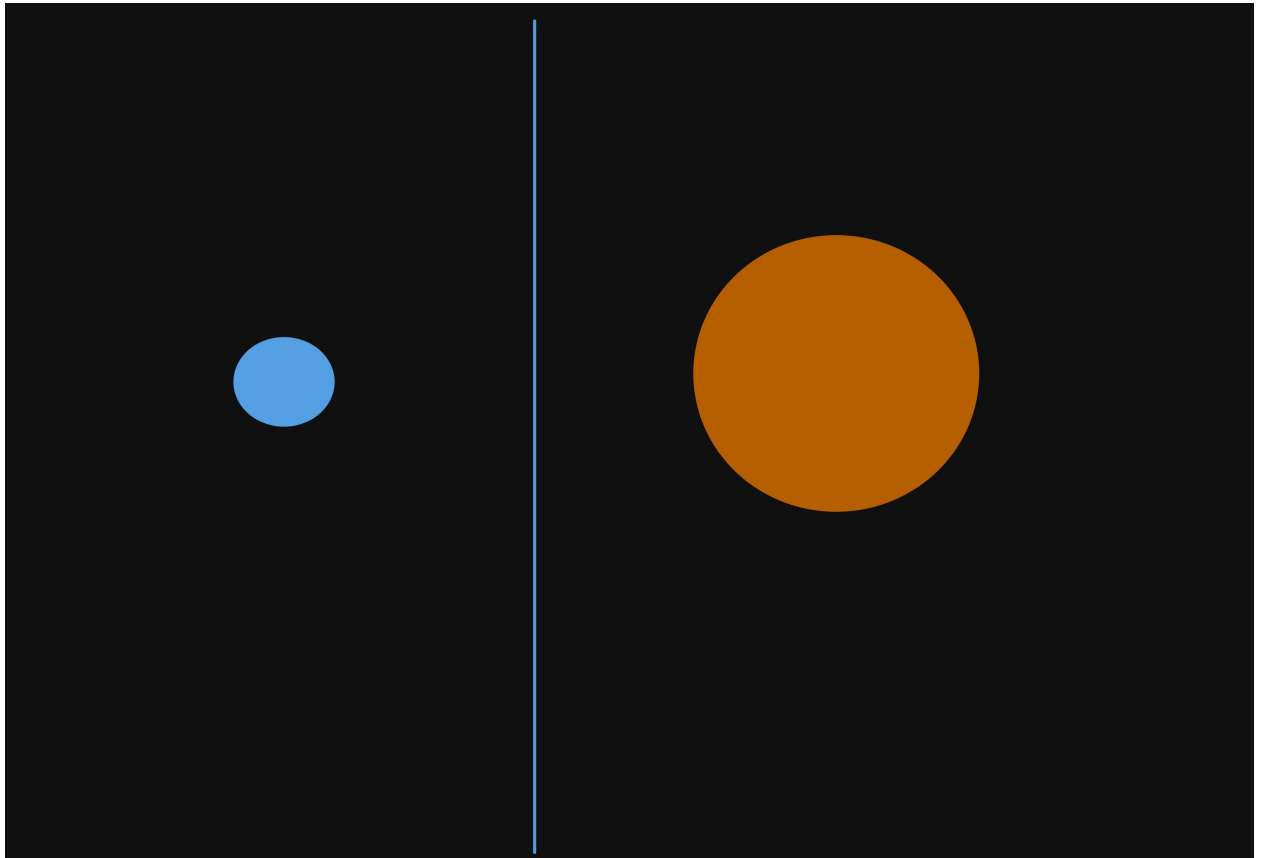


The two smaller circles on the left with lines coming off them are planets, orbiting the central mass in the middle, which represents a star. The star does not have any lines coming off it as stars do not tend to move much relative to the centre of mass of their solar system, because they are so big. Sometimes our sun will move slightly though, and this would be represented with a line showing its path.

This approach makes all the information I want to convey clear, and is quite simple to understand, but is also very easy to render so will not impact performance.

I want all the distances to be to scale in my program, but this is quite difficult as the distances involved in the solar system are absolutely huge. To achieve this I will use a scaling factor, so everything in the system will be scaled by the same amount. This keeps all the relative sizes accurate, e.g. the moon will still be roughly 1/4 the size of Earth, but it won't take a 2000 kilometre wide screen to display it. To do this in code I will take the radius of a planet, multiply it by the scaling factor, and then draw a circle with that radius. If the scaling factor is chosen correctly then all the planets will be visible and it should look like an accurate model of our solar system.

This also makes size comparisons, which were requested by the client, quite easy to do. You simply draw the circles for the two desired planets side by side, rather than at their normal position. Because of the scaling factor the two planets will have the correct relative sizes, so the size comparison will be accurate. This is a mockup of what a size comparison might look like:



Like mentioned when looking at NASA's Eyes, I want the user to have the ability to move the camera. I will not give complete freedom to move the camera, as the solar system is very big and this could easily lead to them getting lost in a lot of empty space between two planets, and just seeing a blank screen. Instead, I think the camera should centre on a planet or other object and the user should be able to choose which object it focuses on. As well as being able to centre on the planets like in NASA's Eyes, I want to also let the user centre the camera on the centre of mass of the system. While this isn't particularly useful for our solar system usually, as the centre of mass is always inside or very close to the surface of the sun, this will be helpful when users start changing the mass of planets, as the solar system will quickly spin out of control and focusing on the centre of mass will make everything easier to see. It will also place you inbetween the sun and whatever planet you've increased the mass of by an absurd amount, so you get a good position to view the system.

Focusing on planets will be done by pressing the left and right arrow keys, this will cycle through objects, which will be ordered based on their proximity to the sun, so pressing the right arrow key makes you move further away from the sun and the left arrow key makes you move closer (until you reach the furthest out object then it will bring you back to the sun and the loop starts again.)

The way the focusing will be done in code is by calculating the vector between the selected object and the centre of the coordinate system. This is our "adjustment", which can then be added to the positions of all the planets during the display process, so that they're all in the correct position relative to the selected planet. The full process is quite mathematical so will be explained in detail during the design.

The user should be able to zoom in and out as well. This lets them focus on the motion of one or two planets, or get a full scale view of the system.

Interface

I will use a simple text and keyboard based menu to allow the user to interact with the program. This will be used to display the energies and momentum as needed to teach conservation of energy / momentum, as well as to let the user edit the mass of planets. I want the user to be able to switch the planet that the program is focused on, and to be able to speed up or slow down the simulation to their liking. I will display text at the top of the screen showing the relevant statistics like the total energy in the system, the mass of the planet, etc. This can be done using pygame's `font.render()` method. A start screen will be displayed at the beginning of the program that explains all the keybinds, as well as obviously letting the user start the simulation (which will be done through pressing enter). Here is a mockup of what the start screen will look like:

```
Press the left and right arrow keys to switch between planets
Press the up and down arrow keys to speed up and slow down time
Press the space key to switch the focus to the centre of mass of the solar system
Press 'F' to show the forces acting on the selected planet
Press 'C' to enter comparison mode to compare sizes of planets
Press 'A' to edit the strength of gravity ( $G$ ) and the mass of planets
Press Enter to start the program!
```

The last line telling the user how to start the program is coloured red, differently from the other lines, to get their attention because if they don't see it they won't know how to start the program.

An "edit mode" will be included that lets the user edit the masses of planets and the value of G by pressing keybinds. The mode should explain all the necessary keybinds so that users do not need to memorise them. Here is a mockup of what the interface will look like outside of edit mode:

Press 'A' to edit G & planet mass	Object: Earth
GPE : $-3.81e+35$ J	Mass: $5.97e+24$ kg
KE : $1.89e+35$ J	Radius: $6.37e+3$ km
Total energy: $-1.92e+35$ J	Distance from sun: $1.5e+8$ km
Total momentum: $5.59e+27$ Ns	
G : $6.67e-11$ Nm ² /kg ²	

All of the information relevant to the physics the kids are learning is shown; the energies, the momentum, and stats about the planet. Kids are expected to know the acronyms GPE and KE for GCSE, so it is okay to use them.

Here is a mockup of what the interface will look like in edit mode:

Press 'A' to stop editing attributes	Object: Earth
Press 'W' to increase value of G	Mass: 5.97×10^{24} kg
Press 'S' to decrease value of G	Radius: 6.37×10^3 km
Press 'E' to increase planet's mass	Distance from sun: 1.5×10^8 km
Press 'D' to decrease planet's mass	
G: $6.67 \times 10^{-11} \text{ Nm}^2/\text{kg}^2$	

It shows the keybinds the user needs to press to interact with the program, and the value of G is still displayed so that the user can see the effects of their changes. The mass of the planet is also still shown so that the user can see their changes to that too.

Reading the inputs will be done with pygame's event.get() system and else if statements. The elifs will be used to check what key is pressed. When 'A' is pressed this will toggle a boolean value, changingAttributes, and when this boolean is true the program will accept inputs of W, S, E and D and change G / the masses accordingly. When it is false the program will not accept those inputs. The other inputs that are mentioned in the start screen will also all be done with elifs, when the space key is pressed it toggles a boolean that makes the program focus on the centre of mass, when the up arrow is pressed it increments the timescale, etc.

We don't want G, the mass or the timescale to be negative, as this would be weird and could lead to many bugs. timeScale is simply changed in increments of a tenth of a year, which works fine, so an if statement will just be used to check if it goes below zero and change it back to 0. For G and the mass, these need to be able to take on values across a huge range of orders of magnitudes. The Sun weighs one hundred million times more than our moon. If we were to choose any fixed increment to change mass by, either the moon's mass would increase by a factor of a few million on a single button press, or the sun's mass would barely change at all. To deal with this a logarithmic scale is used, upon pressing a button to increase or decrease a planet's mass or G, the value will change by $0.5 \times \text{floor}(\log(\text{value}))$ where the log is base 10 and floor returns the largest integer less than log(value). This means the amount that a planet's mass changes by is relative to the current mass of the planet. This also has the side effect of stopping the values from becoming negative, as the increment gets smaller and smaller as you decrease the mass.

Objectives

1. Be physically accurate
 - a. Model the effects of gravity using Newton's laws of motion
 - i. These laws should be implemented as methods for a class used to represent planets and other celestial bodies
 - ii. These laws will require the use of vector operations, the vector operations should be implemented in a separate file, vectors, so they can be called from a module.
 - b. Calculate the motion of objects using verlet integration
 - i. This should also be implemented as methods for the celestialBody class.
 - c. Energy and momentum should be conserved (this should naturally arise from using Newton's laws if they're implemented correctly)
2. Provide an interface that facilitates teaching of the concepts requested by client
 - a. Have a start screen that explains the keybinds to improve user experience

- b. Explain how Newtonian physics connects to the solar system through force diagrams and showing key concepts like energy conservation
 - i. Include force diagrams to aid with this by showing how the forces cause the motion of the planets
 - c. Have a menu that displays stats and lets the user edit certain values as requested by client
 - i. Should display kinetic, potential and total energy, and momentum (This aids objective 2b, explaining how Newtonian physics connects to the solar system.)
 - ii. Should show information about the planet from a reliable source (NASA, see objective 5)
 - 1. Mass
 - 2. Radius
 - 3. Distance from Sun
 - iii. The information should be stored as attributes of the celestialBody class (or in the case of distance from sun can be calculated from celestialBody class).
 - iv. Users should be able to press "A" to enter a mode where they can edit attributes
 - 1. Should let users edit the value of G
 - a. G should never become 0 or negative, this would cause a lot of severe bugs due to the way the maths of gravity works. Hence a logarithmic scale will be used so that G can get closer and closer to 0 but never reach it
 - 2. Should let users edit the mass of planets
 - a. Once again this shouldn't become negative
 - d. Let users speed up and slow down the simulation so they can look at the solar system far into the future
- 3. Accurately display the solar system with a scale model
 - a. This should preserve the relative sizes and distances of all the planets
 - b. Orbital lines should be shown to demonstrate the path of the planets
 - c. The user should be able to focus the camera on any planet of their choosing, or on the center of mass
 - d. Let users zoom in and out
- 4. Include the ability to compare sizes of two planets as requested by client.
 - a. The menu should work slightly differently here and display info on both planets being compared, instead of displaying info on one planet and then displaying info about energies and momentum.
- 5. Access NASA JPL's Horizons system to automatically import up to date data on planets via API requests
 - a. Should be able to parse the data to extract relevant values (mass, size, position, velocity).
 - b. Use said data to load our real solar system into a list of planets (celestialBodies) via dynamic object creation
- 6. Be sufficiently optimised and not suffer frame drops as this would impact the user experience

Documented design

Modules

Here I will explain the modules used in my program, the reasons for using them / where I use them, and for the ones that I created I will explain how they work.

pygame

This module is needed for objectives 2, 3 and 4 to display the menu, display the solar system and compare planet sizes.

For my program to be of any use, you need to be able to see the planets. It doesn't matter how accurate the physics is if you can't see it in action. I have chosen pygame for handling the graphics as it allows easy drawing of lines and circles, which are the two shapes I need to draw planets and their orbital paths as well as things like force arrows. Pygame also allows for input handling which means I can make events happen on key presses, e.g. changing the selected planet.

Extensive use will be made of the `pygame.draw()` function to draw the planets and other parts of the graphics. How I plan to use this function will be explained later in the display section.

horizonsParser, datetime & requests

These modules are needed for objective 5 to access the NASA database.

I want my program to be accurate. That means the initial conditions of the planets should match their actual positions at the time of launch. The easiest way to do this is by fetching the initial conditions from a server, and I have chosen NASA's Horizons database as the place to fetch the values from because NASA is a reliable source of information, their servers have good uptime, and their API is easy to use (even if the data provided for it is horribly formatted.) Hence I will create a function, `getEphemeris` (ephemeris is the technical name of the kind of data that NASA provides) that fetches the required data and put it in a separate file, `horizonsParser`, so it can be imported as a module by the main program.

To achieve this, the `requests` and `datetime` python modules will need to be imported. `Requests` is needed so that I can actually make a request using `requests.get()`, and `datetime` is needed to get the current date to put in the request.

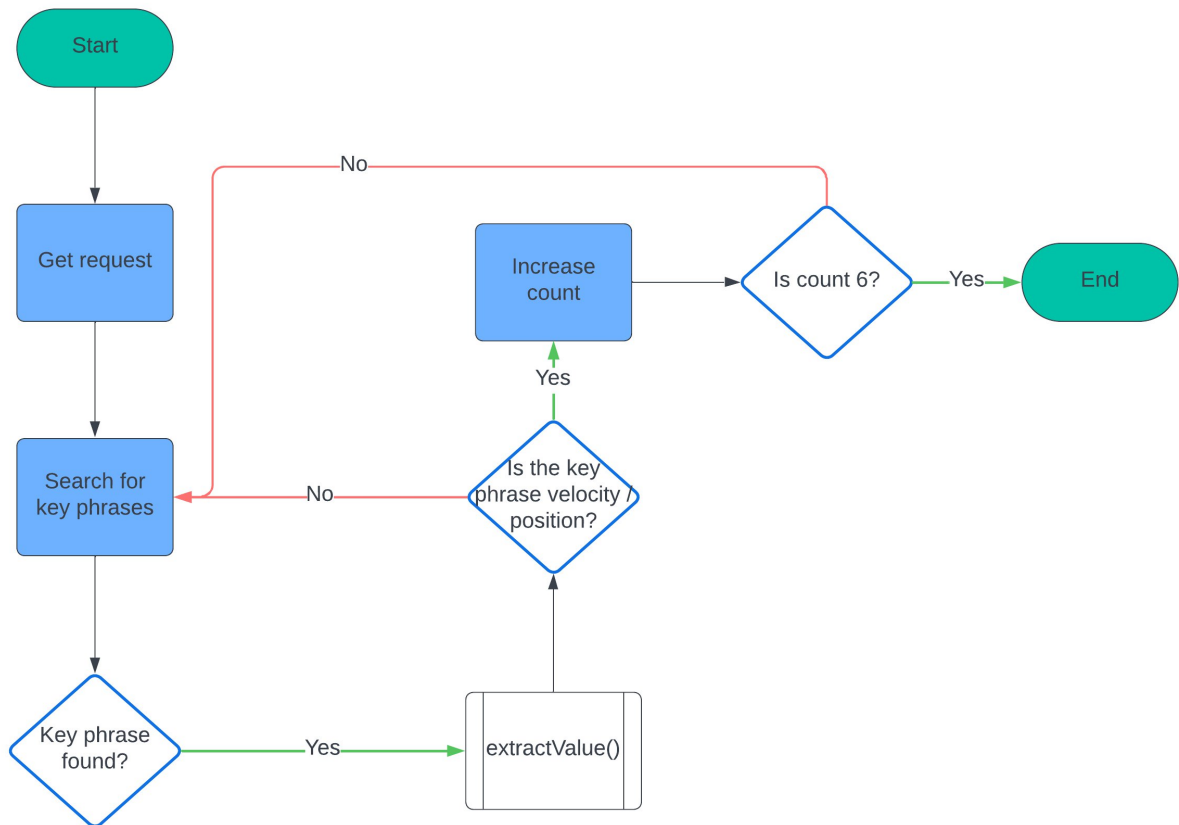
Also, NASA does not provide a useful format for their data. They do provide a json, but it only has one section which is just a bit of text containing the entire response. I have emailed them about fixing this but they did not reply. This means that to actually get values out of their ephemeris a lot of parsing must be done, which is the main reason this module exists.

getEphemeris

The way `getEphemeris` will work is by first getting the request from NASA's database, and then iterating through the characters in the response searching for key phrases that indicate an attribute that I'm looking for (mass, position, velocity, radius). Once a key phrase is found, the position of the key phrase, along with the text of the response, is given to a function called "`extractValue`" which extracts the numeric value of said attribute and returns it. This repeats until

all desired attributes are found and are returned in a list. This will be done by starting a counter once the key phrase for position is found (other attributes appear multiple times before position in some ephemeris's returned by NASA, but not other, so a counter used before then is not helpful) and breaking the for loop in the program once the counter is at 6 (i.e. all 3 coordinates for position and all 3 for velocity have been found.)

Here is a flowchart summarising this information:



Getting the request

getEphemeris will be given the target planet to get the request for. The first thing to do is to cast this target to a string, in case it's been given in the form of a numerical ID, as the request format can only take strings.

NASA then also requires a start and stop time for the ephemeris, as it has the capability to produce a list of coordinates for a planet over a time period. We just want the current position of the planet, but NASA does not let you make a request for just a single day, so I go with the shortest option available, the start time being the current day and the stop time being the day after. This will be found using `datetime.datetime.now()` from the `datetime` module. NASA requires the date to be in `yyyy-mm-dd` format, so I will format the date like this using `.strftime()` from the `datetime` module.

This is all we need to make the request, as the rest of the parameters do not need to change and will be the same for every request. This is the request that I have settled on using after some trial and error:

[https://ssd.jpl.nasa.gov/api/horizons.api?format=text&COMMAND='"+target+"'&OBJ_DATA='YES'&MAKE_EPHEM='YES'&EPHEM_TYPE='VECTORS'&START_TIME='"+startTime+"'&STOP_TIME='"+stopTime+"'&CENTER='500@0'&STEP_SIZE='1%20d'&QUANTITIES=''](https://ssd.jpl.nasa.gov/api/horizons.api?format=text&COMMAND=')

As you can see target, startTime and stopTime are inserted using string concatenation. Some other notable settings are:

EPHEME_TYPE: This has multiple options. However the other options are all intended for uses not relevant to my program, e.g. making telescope observations or planning space missions to other planets. VECTORS is the type that NASA recommends for programming on their own site:

https://ssd-api.jpl.nasa.gov/doc/horizons.html#ephem_type

So it's the one that I use.

CENTER: This is the reference location for the coordinates returned by Ephemeris. 500@0 is the ID for the Solar Barycenter, the center of mass of the solar system. This is the most objective reference point available for our solar system, so is the one that I use.

STEP_SIZE: This just means the ephemeris generates coordinates every day, this isn't really relevant to the program as I'm only using the initial coordinates given in the response.

QUANTITIES: This lets you request a bunch of very cool information, but none of it is needed by my program, so to make getting the requests faster and to reduce the size of the requests, it is set to be empty.

Once we've got the request, I will assign it to the variable "response." The text of the response can be accessed by doing response.text.

Parsing the request

We need 4 key attributes to create a planet, the size, mass, position and velocity. The way we will get these is by searching for key phrases in the ephemeris that signify that a numeric value for one of these attributes is about to appear. To make things easier to understand, here is a sample ephemeris for Earth:

Revised: April 12, 2021

Earth

399

GEOPHYSICAL PROPERTIES (revised May 9, 2022):

Vol. Mean Radius (km)	= 6371.01+-0.02	Mass x10 ²⁴ (kg)=	5.97219+-0.0006
Equ. radius, km	= 6378.137	Mass layers:	
Polar axis, km	= 6356.752	Atmos	= 5.1 x 10 ¹⁸ kg
Flattening	= 1/298.257223563	oceans	= 1.4 x 10 ²¹ kg
Density, g/cm ³	= 5.51	crust	= 2.6 x 10 ²² kg
J2 (IERS 2010)	= 0.00108262545	mantle	= 4.043 x 10 ²⁴ kg
g _p , m/s ² (polar)	= 9.8321863685	outer core	= 1.835 x 10 ²⁴ kg
g _e , m/s ² (equatorial)	= 9.7803267715	inner core	= 9.675 x 10 ²² kg
g _o , m/s ²	= 9.82022	Fluid core rad	= 3480 km
GM, km ³ /s ²	= 398600.435436	Inner core rad	= 1215 km
GM 1-sigma, km ³ /s ²	= 0.0014	Escape velocity	= 11.186 km/s
Rot. Rate (rad/s)	= 0.00007292115	Surface area:	
Mean sidereal day, hr	= 23.9344695944	land	= 1.48 x 10 ⁸ km
Mean solar day 2000.0, s	= 86400.002	sea	= 3.62 x 10 ⁸ km
Mean solar day 1820.0, s	= 86400.0	Love no., k2	= 0.299
Moment of inertia	= 0.3308	Atm. pressure	= 1.0 bar
Mean surface temp (Ts), K	= 287.6	Volume, km ³	= 1.08321 x 10 ¹²
Mean effect. temp (Te), K	= 255	Magnetic moment	= 0.61 gauss Rp ³
Geometric albedo	= 0.367	Vis. mag. V(1,0)=	-3.86
Solar Constant (W/m ²)	= 1367.6 (mean), 1414 (perihelion), 1322 (aphelion)		

HELIOCENTRIC ORBIT CHARACTERISTICS:

Obliquity to orbit, deg	= 23.4392911	Sidereal orb period	= 1.0000174 y
Orbital speed, km/s	= 29.79	Sidereal orb period	= 365.25636 d
Mean daily motion, deg/d	= 0.9856474	Hill's sphere radius	= 234.9

```

*****
Ephemeris / WWW_USER Sat Nov 9 06:59:32 2024 Pasadena, USA / Horizons
*****
Target body name: Earth (399) {source: DE441}
Center body name: Solar System Barycenter (0) {source: DE441}
Center-site name: BODY CENTER
*****
Start time : A.D. 2024-Nov-07 00:00:00.0000 TDB
Stop time : A.D. 2024-Dec-07 00:00:00.0000 TDB
Step-size : 1440 minutes
*****
Center geodetic : 0.0, 0.0, 0.0 {E- lon(deg), Lat(deg), Alt(km)}
Center cylindric: 0.0, 0.0, 0.0 {E- lon(deg), Dxy(km), Dz(km)}
Center radii : (undefined)
Output units : KM-S
Calendar mode : Mixed Julian/Gregorian
Output type : GEOMETRIC cartesian states
Output format : 3 (position, velocity, LT, range, range-rate)
Reference frame : Ecliptic of J2000.0
*****
JDTDB
X Y Z
VX VY VZ
LT RG RR
*****
$$$$$
2460621.500000000 = A.D. 2024-Nov-07 00:00:00.0000 TDB
X = 1.044144958748656E+08 Y = 1.036332233686863E+08 Z = 2.171908221627772E+04
VX=-2.144784999470549E+01 VY= 2.104051139446795E+01 VZ=-1.526211354835993E-03
LT= 4.907161205829759E+02 RG= 1.471129919697947E+08 RR=-4.008514817144361E-01

```

So for example, if we want to get the mass of the planet, we need to look for the phrase “Mass “. Except this appears multiple times, so to be truly unique, we need to look for the phrase “Mass x” which only appears once. This will be done by iterating through each character with a for loop and checking that character along with the next 5 characters after it. So I will check `response.text[i:i+6]` where `i` is the dummy variable I’m using in a for loop. The phrases needed differ by planet, *and* have different lengths sometimes, as NASA does not have a standard formatting for their ephemeris, so a lot of OR statements will be used inside the program to handle all the different possible cases. The key phrases have been manually found by me through trial and error with different ephemerides (yes that’s the plural of ephemeris) and then put into the program.

Once a key phrase is found, the numerical value needs to be extracted. I will skip over how this works for now and explain it in the next section on `extractValue`. Once we have the numerical value, the relevant attribute (e.g. mass) is set to be that value. So here mass would be set to “ 5.97219×10^{24} ” as that is the value given by the ephemeris.

A slightly different approach needs to be used for position and velocity, as they are vectors. I will create two numpy arrays for position and velocity at the start and set all the elements to 0, these elements can then be updated as the values for them are found. This is because arrays are fixed length so you cannot append values to them.

To decide which element to update, a dictionary will be used containing all the key phrases, like this:

```

1 posDict = {
2     'X':0,
3     'Y':1,
4     'Z':2,
5 }
```

And I will then update `position[posDict[response.text[i:i+3]]]` so that I update the correct element. I will do the same thing for velocity, but a different dictionary will be used as the key phrases for velocity are different.

Once the final velocity element, `velocity[2]`, is found (i.e. it's no longer 0, the chance of a velocity component of a plane naturally being 0 is so small that it will never happen during the lifetime of the Earth), then the for loop should break and size, mass, position and velocity should be returned in a list.

There are two optional flags that can be set in `getEphemeris`. The first will be “useGM.” Some ephemerides, for example the ones for Jupiter’s moons, give a physical attribute “GM” instead of the planet’s mass. The mass however can be derived from this by simply multiplying the value by a constant. If “useGM” is true then the program should ignore the key phrases relevant to mass and instead search for the key phrases relevant to GM, and should extract the value and multiply by the constant to then get the mass of the planet.

The other flag is “ignoreMassSize”. For particularly small objects, especially human-made satellites, the mass and size of the object are not given. In this case the mass and size of the object will be manually passed into `getEphemeris` as arguments and the key phrases relating to them will be ignored.

Now that all of that is explained, here is some pseudocode for `getEphemeris`:

```

1  FUNCTION getEphemeris(target, useGM=False, ignoreMassSize=False, mass=0, size=0)
2      target ← str(target) // Casting target to string as that's what requests.get() will want
3      startTime ← datetime.now().strftime("%Y-%m-%d") // formatting start time as yyyy-mm-dd
4      stopTime ← (datetime.now() + datetime.timedelta(days=1)).strftime("%Y-%m-%d") // Adding 1 day
    to get the stopTime, aka tomorrow
5      response ← requests.get("https://ssd.jpl.nasa.gov/api/horizons.api?
format=text&COMMAND='"+target+"'&OBJ_DATA='YES'&MAKE_EPHEM='YES'&EPHEM_TYPE='VECTORS'&START_TIME='"+
+startTime+"'&STOP_TIME='"+stopTime+"'&CENTER='500@0'&STEP_SIZE='1%20d'&QUANTITIES=''") // Using
string concatenation to get a request with the desired target and times
6
7      position ← [0.0, 0.0, 0.0] // numpy arrays will be used in the real code
8      velocity ← [0.0, 0.0, 0.0]
9      // These dictionaries are so we know what element to update while extracting values for
    position and velocity, as X, Y and Z coordinates are given separately
10     posDict ← {
11         'X':0,
12         'Y':1,
13         'Z':2,
14     }
15     velDict ← {
16         'VX':0,
17         'VY':1,
18         'VZ':2
19     }
20
21     FOR i ← 0 TO LEN(response.text)
22         IF ignoreMassSize = FALSE THEN // Checking the boolean flags to see if we need to treat
    mass / size differently or not
23             IF useGM = FALSE THEN
24                 IF response.text[i:i+6] = "mass x" OR response.text[i:i+6] = "mass, " THEN //
    searching for mass keyphrases, will use .lower() in actual code because case doesn't matter (and
    differs based on planet because NASA sucks)
25                     mass ← extractValue(response.text[i:i+50],TRUE) // The "TRUE" here means that
    we are expecting to have to find a scale for the mass (i.e. 10^24 for earth)
26                     ENDIF
27                 ELSE
28                     IF response.text[i:i+6] = "gm, km" OR response.text[i:i+6] = "gm  (" OR
    response.text[i:i+6] = "gm (km" THEN
29                         mass ← extractValue(response.text[i+20:i+50], FALSE, 1/(6.6743015*10**-20))
    // The number at the end means to multiply the result by that amount, this is because the value
    we're given is GM, so we need to divide by G to get M
30                     ENDIF
31                 ENDIF
32                 IF response.text[i:i+16] = "vol. mean radius" OR response.text[i:i+16] = "mean
    radius (km)" THEN
33                     size ← extractValue(response.text[i:i+50], FALSE, 1000) // radius is given in km
    so need to multiply by 1000 to get it in metres
34                 ENDIF
35             ENDIF
36

```

```

37 IF response.text[i:i+3] IN posDict THEN
38     position[posDict[response.text[i:i+3]]] ← extractValue(response.text[i:i+40], FALSE,
1000) // position is given in km, need to multiply by 1000
39 ENDIF
40 IF response.text[i:i+3] IN velDict THEN
41     velocity[velDict[response.text[i:i+3]]] ← extractValue(response.text[i:i+40], FALSE,
1000)
42 IF velocity[2] ≠ 0 THEN
43     BREAK // Once velocity is found we exit the loop, as velocity is always the last
value given in the ephemeris
44 ENDIF
45 ENDIF
46 ENDFOR
47 RETURN [size, mass, position, velocity]
48 ENDFUNCTION

```

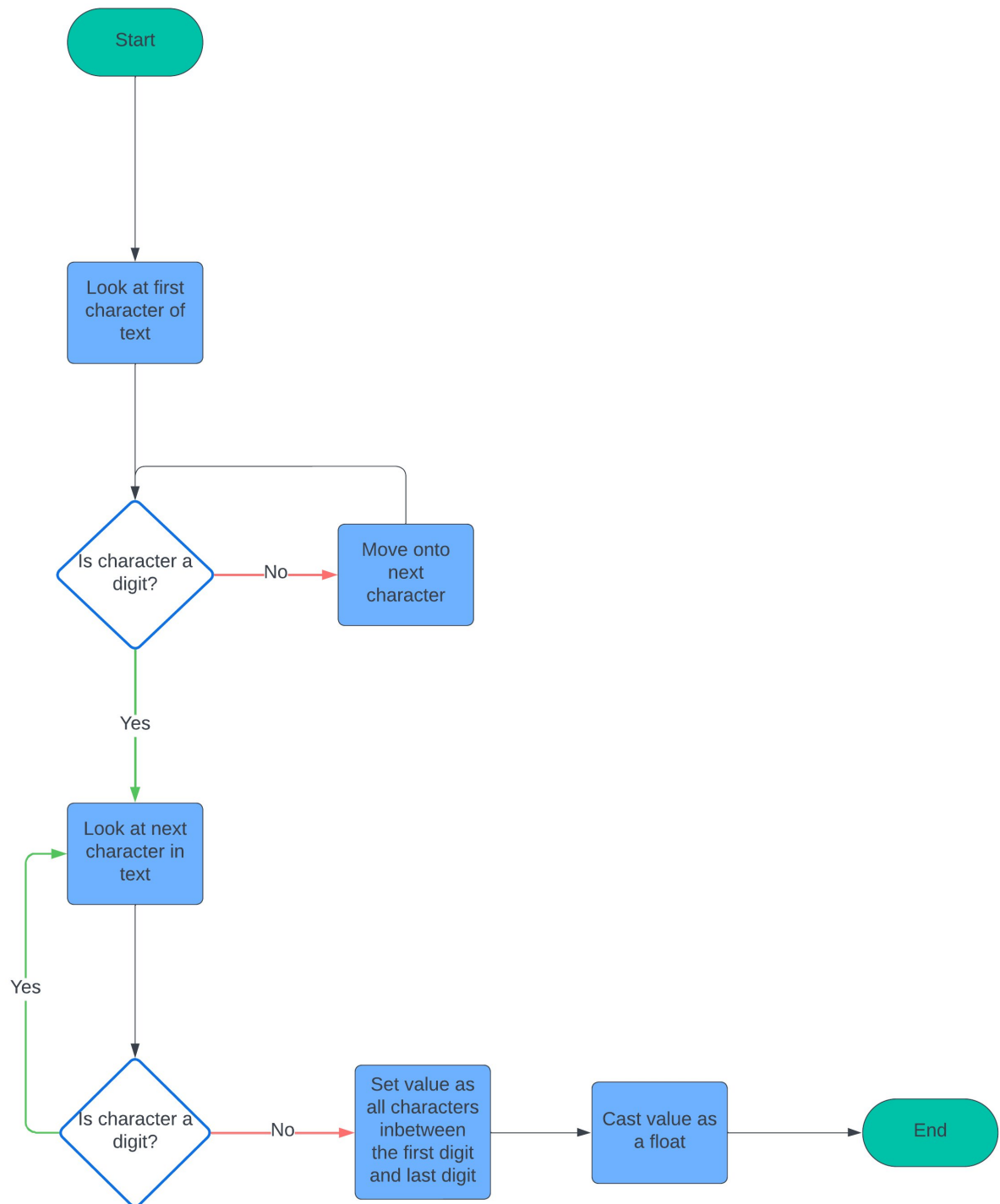
extractValue

Now that we have a high level overview of how getEphemeris works, let's analyse how it actually gets numeric values from the text. extractValue will be given a section of the response starting from the key phrase and continuing for a few dozen characters so that it only gets the number relating to the attribute we're interested in and doesn't accidentally find the value of something else. extractValue will need to do two things:

1. Find, if there is one, the scale given by NASA. E.g. they might give the scale as 10^{24} , and then the numeric value would be 57.9. This means you should interpret the number as $57.9 * 10^{24}$. If no scale is found, scale should be set to 1 by default.
2. Find the actual numeric value for the attribute.

We will pass in a boolean value telling the function whether or not it should look for a scale. We will also have the option of passing in a "conversion factor", which is just a number we want to multiply the final answer by. Handy if the value is given in non-SI units by NASA and we want to convert it to SI.

Here is a flowchart explaining the basic algorithm used to get a number from a piece of text once the position of a key phrase has been found:



This same basic algorithm will be used by both the loop that finds the scale, and the loop that finds the numeric value of the attribute we're interested in. Here is how one might implement it in pseudocode with a given piece of text:

```

1   text = "foo 123 bar"
2   FOR i ← 1 TO 50
3   IF text[i] in ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"] THEN // can use .isDigit()
    for this in python
4   FOR j ← 1 TO 50
5   IF text[i+j] NOT in ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"] THEN // Finding
    where the last digit is
6       value = float(text[i:i+j]) // Cast value to be the float of the text from the
    first digit to the last digit
7   ENDIF
8   ENDFOR
9   ENDFOR
10  ENDFOR

```

Some caveats will need to be introduced, with the scale we need to evaluate it as a power of 10, and with the value we need to take into account the fact that it could be negative or have a decimal point and could also be written using “E notation” (a way of writing standard form that uses E to represent the 10^x part, e.g. $1250 = 1.25 * 10^3 = 1.25E3$). I will describe how I’ve handled this in the individual sections on finding the scale and finding the value.

Finding the scale

The scale in the ephemeris is always in the form 10^x , where x is a natural number. This means we can find a ^ character (they only appear in scales and nowhere else) and then use the algorithm above to find x. Since x is always a natural number, no extra tricks are needed with the algorithm. Once we have found x, we evaluate 10^x using python’s eval function and set scale equal to that.

We will then also create a variable, “scaleAt”, and set it equal to the position of the last digit. This is so that the next part of the function, which finds the value, can start just after the scale. If we do not do this then we would end up accidentally finding the scale again when trying to find the value

Finding the value

We will need to make some modifications to the algorithm above when it comes to finding the value. Instead of just checking if the current character is a digit, we need to check if it’s a digit, a + sign, a – sign, a decimal point . or E. Luckily if you use float() on a string written in E notation in python it can successfully convert that string to a number, so we do not need to worry about evaluating E notation ourselves. Python can also handle positives and negatives when casting to a float.

Now the only thing to worry about is “error margins”. NASA wants to make it clear that their values are not exact, so they put error margins on them. This means that instead of writing a mass as 500, they might write it as 500 ± 1 . This is a bit problematic. We count + and – as “digits” because we need to be able to handle negative values, so our program would continue until it got to the 1, and only then would it cast the string to a float. But python cannot cast 500 ± 1 as a float, because it’s not a simple number. So, what we need to do is search for any instances of ‘+-’ in the string, and if we find one, then we slice the string to only include characters before the +-.

Once that's done, we can safely float our value, and we're finished. All we need to do is return $\text{scale} * \text{value} * \text{conversionFactor}$, and that's the attribute for our planet.

Here is some pseudocode for the whole function:

```

1 FUNCTION extractValue(text, expectingScale=FALSE, conversionFactor=1)
2     scaleAt ← 0 // Position of the scale so that we can skip it when we're finding the value
3     scale ← 1 // Set scale to 1 by default unless something else is found
4
5     IF expectingScale = TRUE THEN
6         FOR i ← 0 TO 49
7             IF text[i] = '^' THEN
8                 FOR j ← 1 TO 24
9                     IF text[i+j].isDigit() = FALSE
10                        scale ← eval(text[i-2:i+j].replace('^','**')) // This gives us scale =
10**x where x is the natural number that has just been found
11                        BREAK
12                    ENDIF
13                scaleAt ← i+j // Set scaleAt so we can skip over the scale when finding the value
14            ENDFOR
15        ENDIF
16    ENDFOR
17 ENDIF
18
19 FOR i ← scaleAt+1 TO 40
20     IF text[i].isDigit() OR text[i] = '-' THEN
21         FOR j ← 1 TO 50
22             IF text[i+j].isDigit() = FALSE AND text[i+j] not in ['-','.', 'E', '+'] THEN //
23                 IF text[i+k:i+k+2] = "+-" THEN // getting rid of error margins
24                     j ← k // This means that when we do float(text[i:i+j]) anything after
25                     the error margin is not included
26                     BREAK
27                 ENDIF
28             ENDFOR
29             value ← float(text[i:i+j])
30         ENDIF
31     ENDFOR
32     RETURN scale*value*conversionFactor // Need to multiply so that the scale is taken into account
33 ENDFUNCTION

```

The numbers chosen for i and j to range over are fairly arbitrary, I just picked some numbers that would be large enough that we'd get all the numbers we needed but wouldn't include any numbers from other attributes that we might accidentally return.

math(s)

This module (as well as numpy and vectors) are needed to do the maths required to accurately simulate the solar system, so are needed for objective 1.

Due to the way I'll handle vectors in my physics I do not need to make extensive use of python's math module. Other implementations of Newton's second law would require a lot of trigonometry which would require the math module, but I have avoided this, which is good for performance as trigonometric functions are computationally expensive.

However, I do still need to be able to square root values so that I can find the magnitude of vectors, so I need the math module (this can be done with `**0.5` without math, but this is worse for readability and makes it less clear what the code is doing.) I also need to calculate logarithms for non-linear scaling when drawing force arrows (this will be explained more in the display section), for which I definitely need the math module. Logs will also be used at one point for changing the value of G, so that the changes are always relative to the order of magnitude at G at that point (explained later down in input section.)

As I am British, and because this program is intended for a British audience, I will import math as maths. This improves readability for a British person, who may be disturbed by seeing what they perceive as an incorrect spelling of maths otherwise.

numpy

Numpy is needed for one thing and one thing only. Its arrays. Numpy arrays support vector addition and subtraction, which is a life saver for me as you cannot do physics without it. I could have created my own implementation of vectors but this would have hurt performance. Numpy is mostly written in C, which is a compiled programming language with a lot less overhead than python. This means that numpy arrays are a lot, lot faster than any implementation you could create using python. Numpy has also been used for hundreds of thousands of projects around the world without issue, whereas any implementation of vectors I could make would be prone to bugs. Hence, I use numpy's arrays to create vectors instead.

vectors

Physics, especially gravity, relies heavily on vectors. Because of this, some way to implement vectors is needed. As explained earlier I will use arrays from the numpy module as they support vector addition and subtraction, which is very useful. There are a few vector operations that my program needs, these will be implemented in a separate file from the main one and imported as a module. The module will be imported as "vec" which is why you will see things like "vec.mag(vector)" in later pseudocode. The three operations needed are:

1. Finding the magnitude of a vector
2. Finding the unit vector for a vector
3. Finding a normal to a vector

Luckily these are all relatively simple to implement.

Magnitude of a vector

The magnitude of a vector is the square root of the sum of the squares of all its elements. This is easier to understand in code, here is some pseudocode:

```

1  FUNCTION mag(vec)
2      magnitude ← 0
3  FOR i ← 1 TO LEN(vec)
4      magnitude ← vec[i]**2
5      magnitude ← magnitude**1/2
6  ENDFOR
7      RETURN magnitude
8  ENDFUNCTION

```

Finding a unit vector

The unit of a vector is a vector in the same direction as it of magnitude 1.

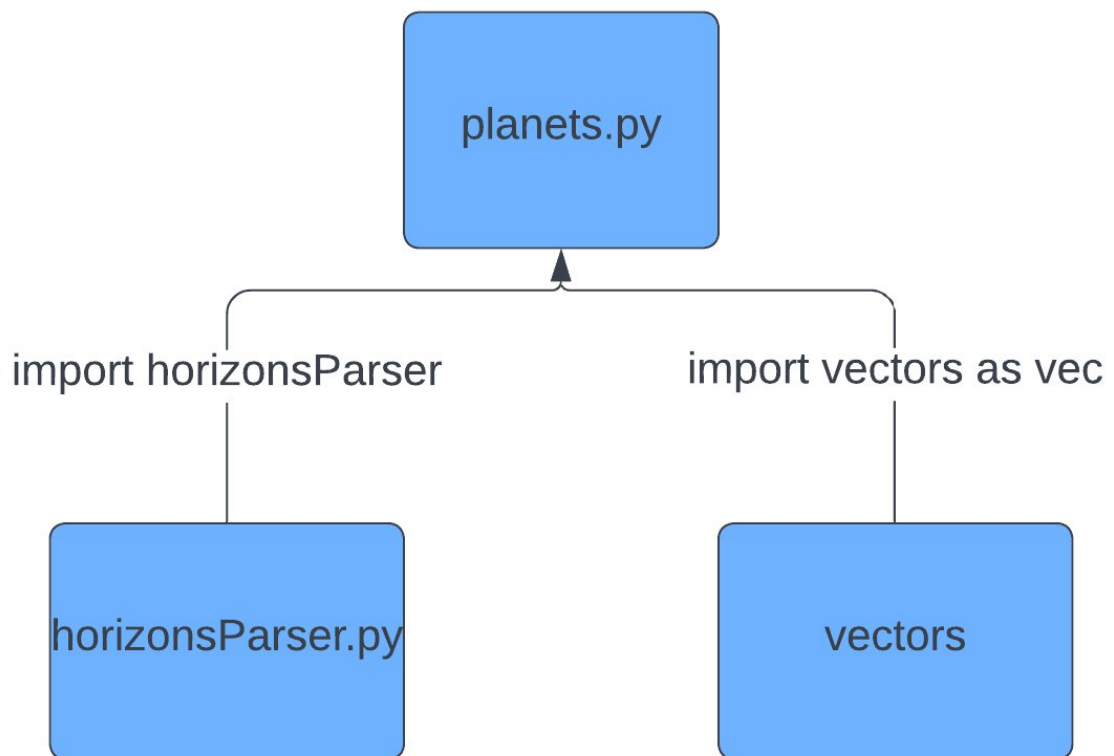
To find a unit vector, simply divide every element in it by the magnitude of the vector. Numpy supports dividing whole vectors by a number, so this is even easier, just divide the vector by the magnitude. So, the `unit()` function will simply return `vector/mag(vector)`

Finding a normal vector

For a vector (a,b) , a vector normal to that vector is $(-b,a)$. However we want this vector to be of length 1, so that it is easy to scale. Hence we will return `unit(-b,a)`.

File Structure

I only have 2 modules that I've created, `vectors` and `horizonsParser`, so this keeps the file structure fairly simple. I have a file for each of those modules, along with the main file that runs the program. Here is a diagram showing the file structure:



Data dictionary

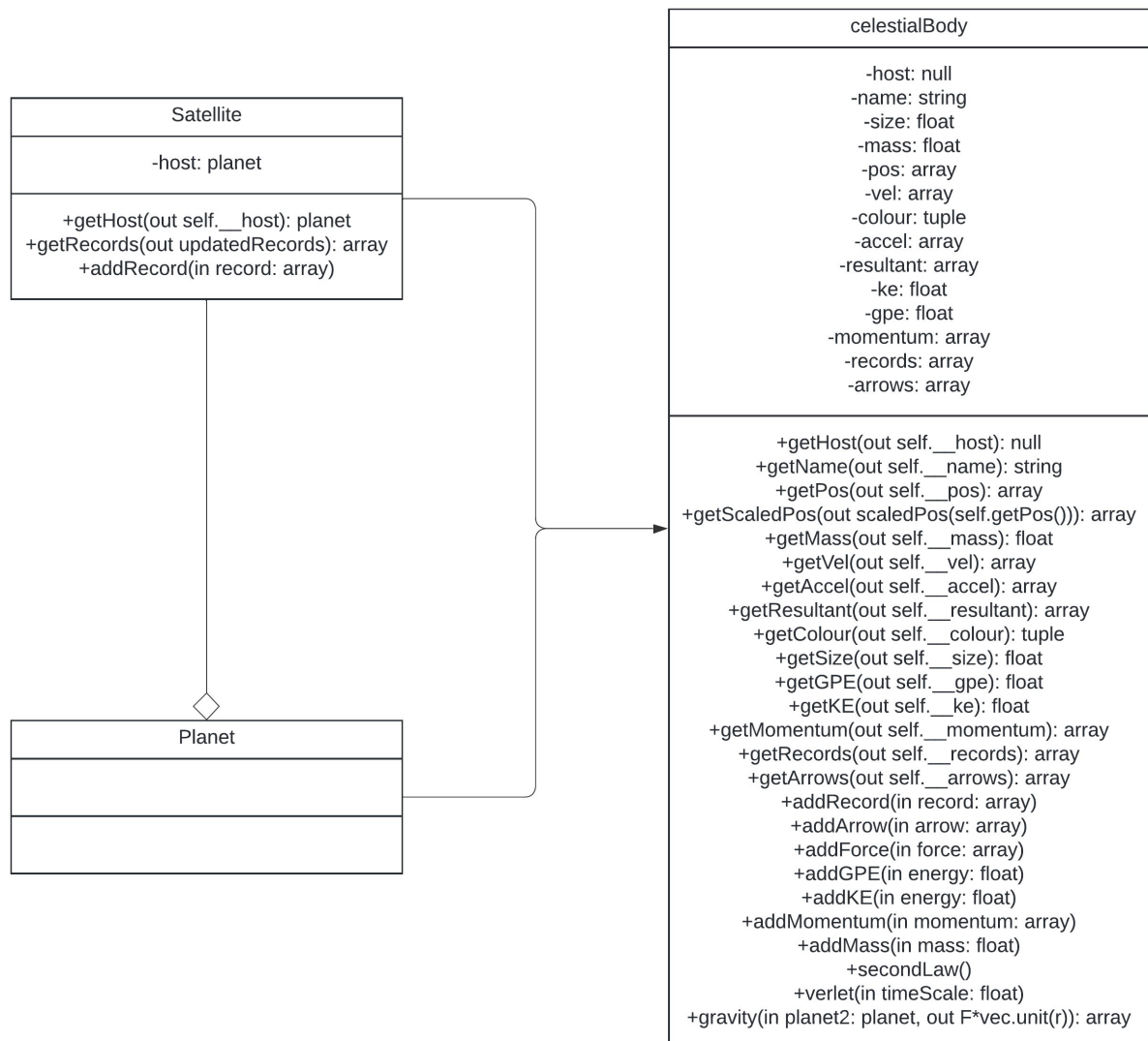
Here are the important global variables (some of the less important ones are left out) in the main program and their types, initial values and a description of their purpose:

Data item	Type	Initial value	Description
x	integer	0	x and y are mapped to 0 and 1 respectively for indexing purposes, this means if I want the x component of a vector I just need to write vector[x], which makes the program a lot more readable.
y	integer	1	See above description.
framerate	integer	120	How many frames to display per second
YEAR	float	31536000/framerate	How long an Earth year is in the program, timeScale is done relative to this.
timeScale	float	0.05*YEAR	How much real life time is covered

			during each tick of the program
distScale	float	$4 \cdot 10^{-9}$	How zoomed out the display should be
G	float	$6.6743015 \cdot 10^{-11}$	Determines how strong the gravitational force is
MAX_LINES	integer	200	This sets the max amount of orbital lines that can be drawn on screen, with no limit they would just continue increasing and this would lead to large amounts of lag.
planets	array	Empty	A list of all the celestial bodies in the program, including non-planet objects (it's called planets as celestialBodies is a bit cumbersome to type all the time)
focus	integer	0	The current planet to center the camera on, 0 means center on the sun
comFocus	bool	False	Whether or not to focus on the center of mass of the solar system
comparison	bool	False	Whether or not to compare the sizes of different planets
started	bool	False	Used in the loop displaying the start screen, once the user starts the program started is set to true and the loop is therefore exited
changingAttributes	bool	False	Turned to true when the user presses 'A', this enters them into edit mode which lets them change the value of G and the masses of planets.

Classes

There will be three classes, celestialBody, planet and satellite. Here is a diagram showing how they're connected:



As you can see, here, there aren't many classes. **celestialBody** is the main template for every object in the solar system, and has all the basic methods needed by any object. **Planet** and **Satellite** both inherit from **celestialBody** and use all of the methods in it. All celestial bodies need to be able to:

- Output their attributes
- Keep a record of their previous positions so that we can draw orbital lines
- If they're the currently selected planet, keep a record of the forces acting on them so that force arrows can be drawn if the user has them turned on
- Keep a record of their physical properties: Kinetic and potential energy, and momentum, so that these can be summed up across all bodies and displayed to demonstrate conservation of energy and momentum
- Have methods that facilitate physical simulation so that the program accurately represents the solar system.

The methods that are related to physical simulation will be covered in the section on physics. So let's cover the rest.

First, outputting their attributes. This is all done with “getter” functions. You can see a function is a getter function because it starts with a get. Every get function in the class diagram returns the attribute in its name, i.e. getVel() returns vel (velocity) and getKE returns ke (kinetic energy.) I do this rather than just using planet.pos or planet.vel so that I can implement encapsulation, which improves security. It also means I can edit the output before it's returned, which is used in the satellite subclass to change the way its orbital lines look to make them look better. Making everything a getter leaves room open for further editing in the future if needed, and makes the program more robust. Note that there is a getHost() function even though the host attribute is always null and never used by the celestialBody class. This is to ensure compatibility with the satellite subclass, we can run getHost() on an object without having to worry about if it's a satellite or planet, and if the host is null then we just ignore it.

Next, keeping a record of their previous positions. This is quite simple, as we'll see later I'll just add their position to a list of all their positions while simulating the tick. This list is “records”, which you can see in the attributes in the class diagram. To add a record to it, we'll have a function, addRecord(). This is important, because having a function do it means we can get rid of the oldest entries in the list whenever we add a record, so that the list of positions doesn't grow indefinitely. addRecord() will use the line length to shorten the list whenever it gets too long. Here is a pseudocode snippet that demonstrates how that might work:

```

1  PROCEDURE addRecord(self, record)
2      self.__records.APPEND(record) // Add latest position to the records
3  IF LEN(self.__records) > LINE_LENGTH THEN
4      // This uses string slicing to get rid of the oldest records if the list is too long
5      self.__records ← self.__records[LEN(self.__records)-LINELENGTH:]
6  ENDPROCEDURE

```

So e.g. if LINE_LENGTH was 10 and records was 11 long, then it would set records to be records[11-10:] which means all but the first element of records, so it would be back to 10 elements like we wanted.

Keeping a record of the arrows is fairly similar. We want to draw arrows from one planet displaying the forces on it from the other planets, and also an arrow displaying the resultant force. This means we need to draw one arrow for every planet in the system except the one we've selected, as well as an arrow displaying the resultant force. This means overall the amount of arrows drawn should be the same as the amount of planets in the system, so we can use the same kind of code as with records to reset the length, we just replace LINE_LENGTH with len(planets).

Keeping a record of their physical attributes like kinetic energy and momentum will be done by having an attribute for each property we want to measure, resetting all the attributes to 0 at the start of a tick and then calculating the attributes during the tick. This will be explained in the physics section as they are done at the same time as working out all the physics

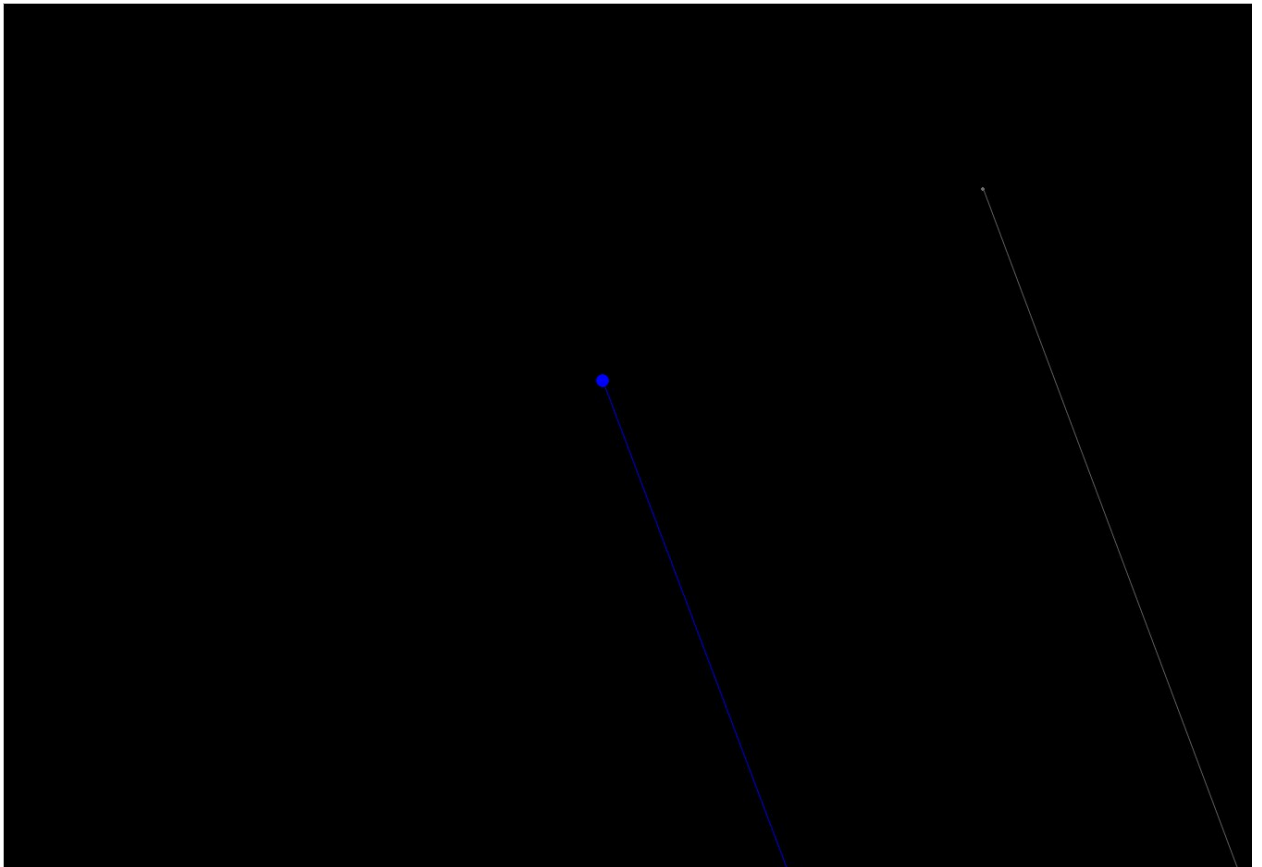
Satellite / planet

These classes are both subclasses of `celestialBody`. The distinction between planet and satellite in my program is a bit different than how the words are used in normal life. In my program, a satellite has a host object, and its orbital lines are drawn relative to the motion of the host object. This is why it's called a satellite, as the proper, physical definition of a satellite is just an object orbiting another object, e.g. the moon is a satellite of Earth. A planet will not have changes from the parent class, `celestialBody`, but the class is there to make clear the distinction between planet and satellite.

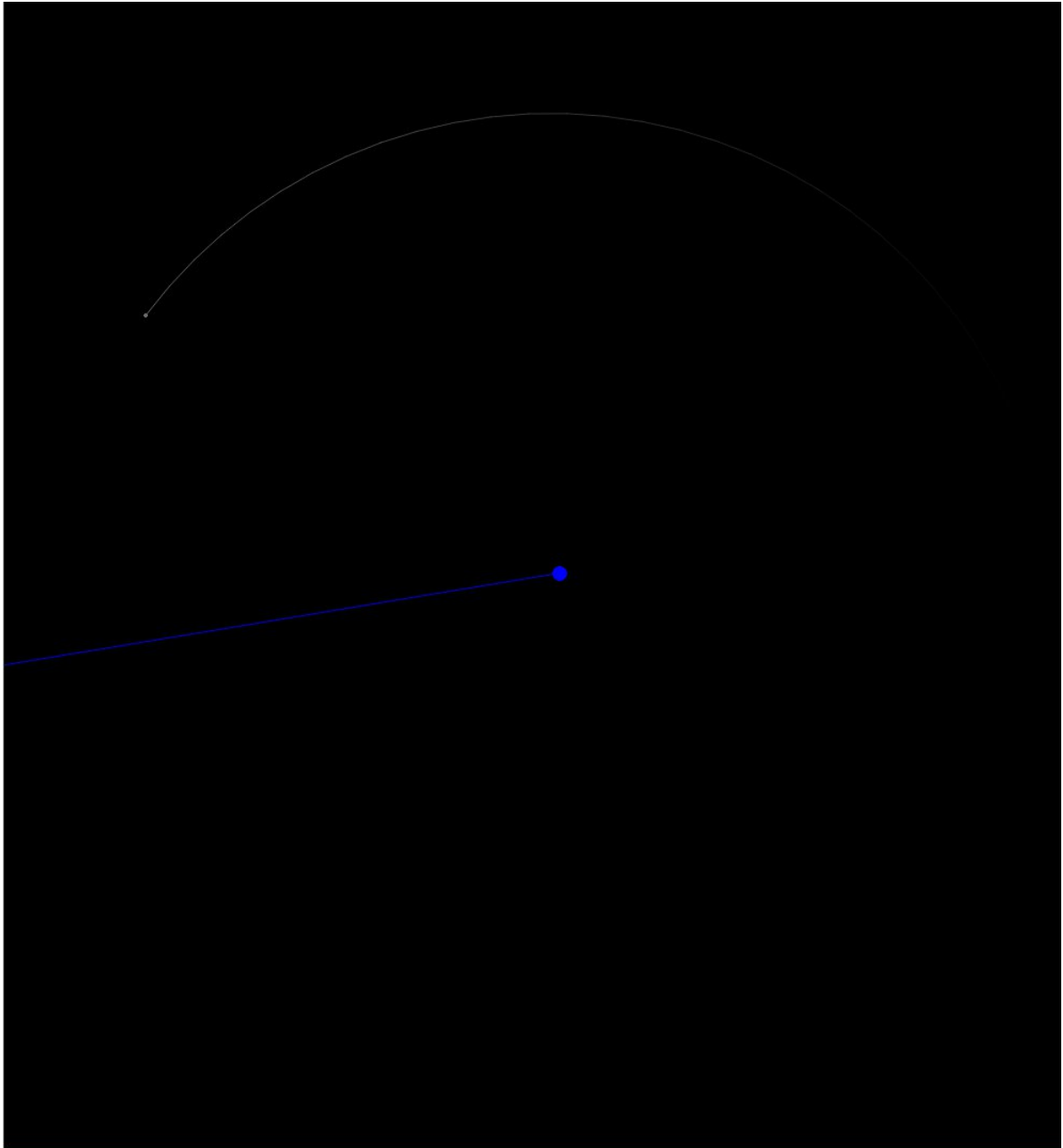
Since the only thing the satellite class needs to do differently is change how the orbital lines are drawn, there are only a few things we need to change. First, we should make the host attribute something other than null. We will use aggregation to set the host attributes, the host attribute should be the object that the satellite has as a host. This object will be another planet. Satellites orbiting satellites is possible, but there is no instance of that in our solar system, so in the class diagram I just showed satellite as aggregated with planets, but you could aggregated it with another satellite if you really wanted to. All this means that `getHost()` will now return an object instead of null.

Secondly, `addRecord()` and `getRecords()` should be edited so that the path they display is relative to the motion of the host.

But why do the orbital lines need to be drawn relative to the motion of the host? Well, this is what the moon's path looks like if its path is not drawn relative to Earth's motion:

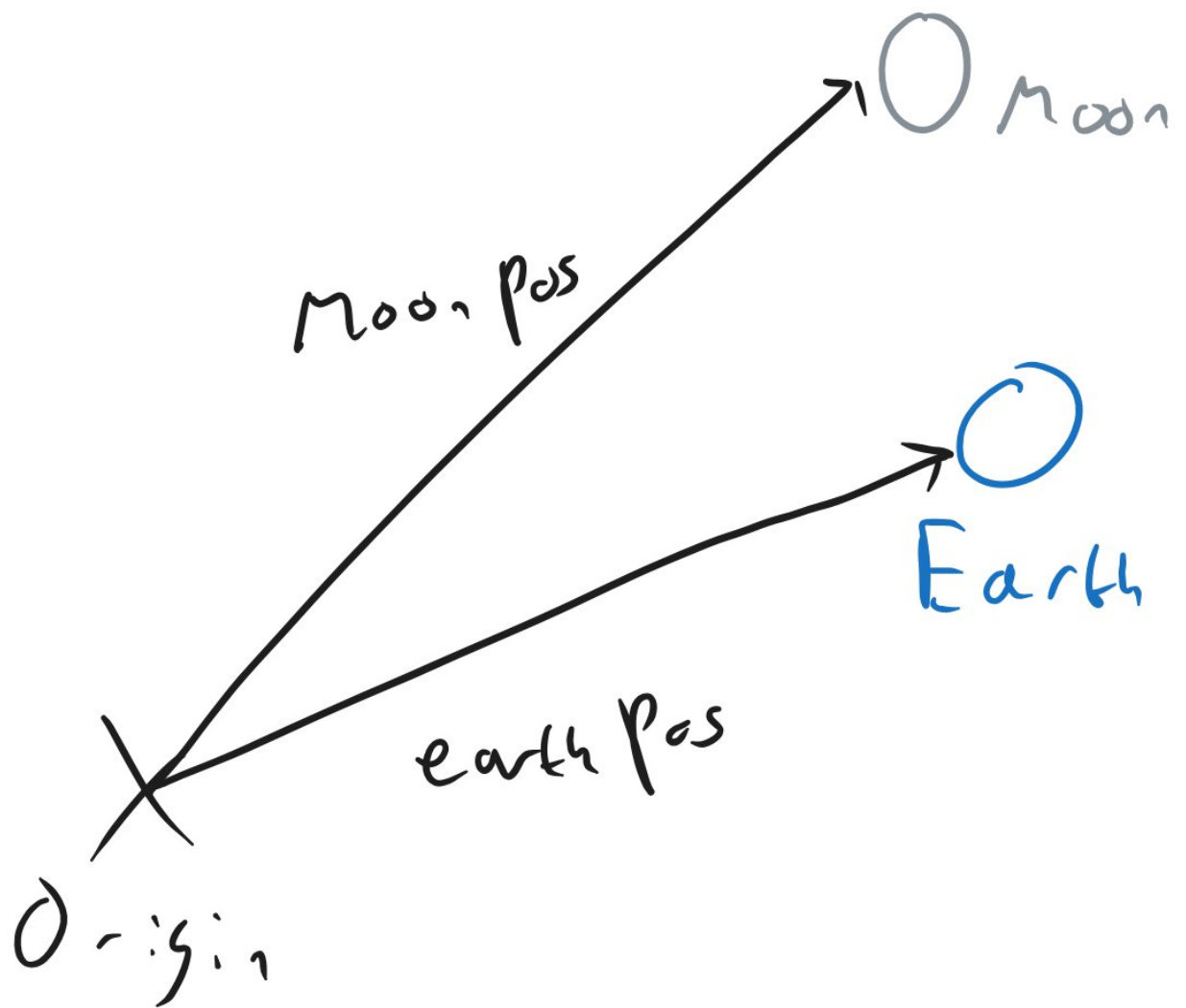


The moon is orbiting Earth, but it's also orbiting around the sun. The path the moon follows just makes it look like it orbits the sun, with very slight wiggle because of its orbit around the Earth. This is not very illustrative and will confuse students as they might not get how the moon is technically orbiting both the Earth and the Sun. When we talk about the moon's orbit, we mean its orbit around Earth. What this means physically is the motion of the moon relative to the motion of Earth. If we draw its path relative to Earth's motion, we get this:

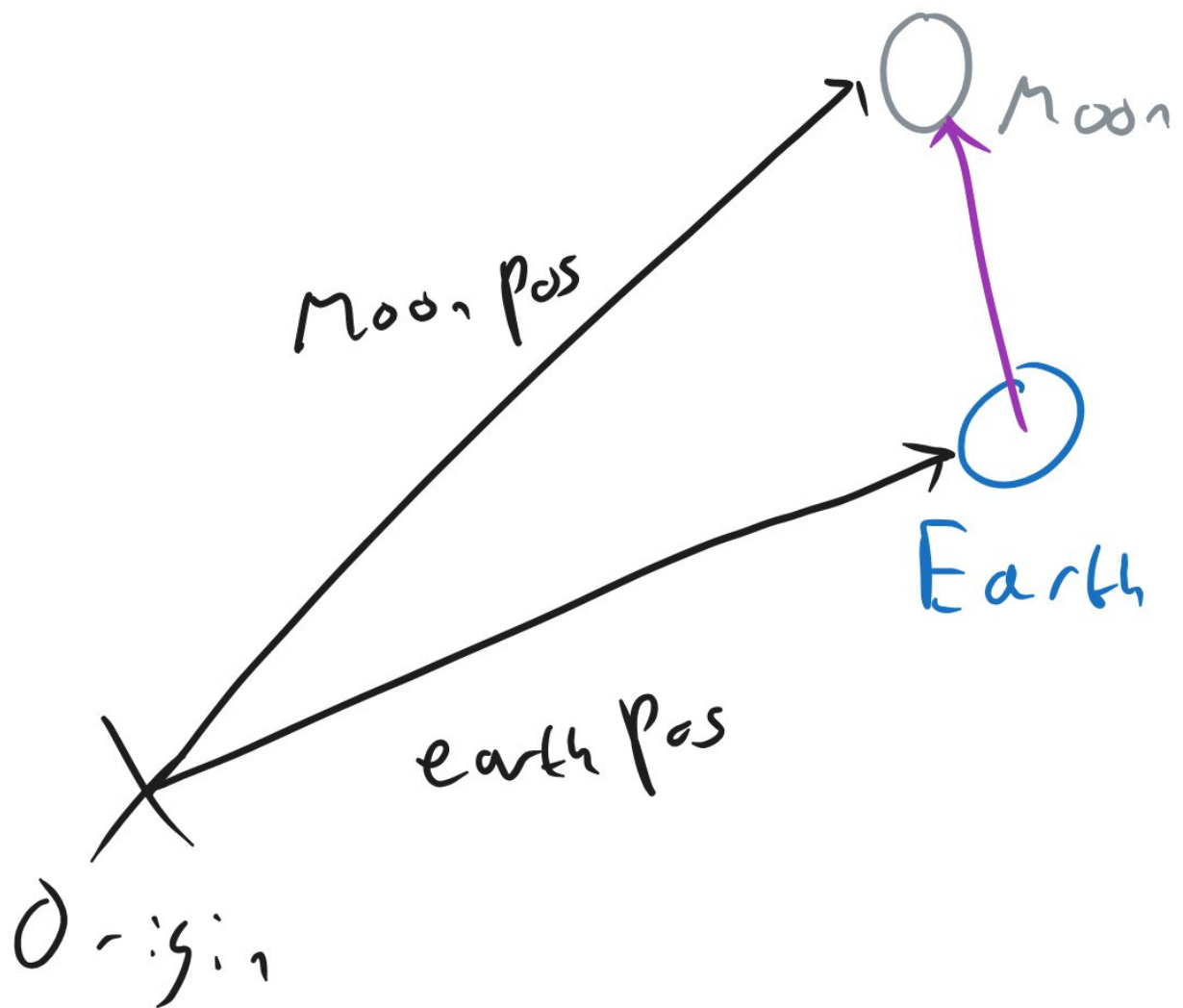


The path drawn is the arc of a circle around Earth, exactly as it should be. This is much better.

How do we achieve this in code? Well, for a normal `celestialBody` object, its `records` is a list of displacements from the origin. We can think about displacements as a vector pointing from the origin to another point in space, like this:



We want to instead have the moon's displacement relative to Earth, i.e. an arrow pointing from Earth to the moon, like the purple arrow in this diagram:



It turns out, rather conveniently, that the purple arrow there is just the vectors $\text{moonPos} - \text{earthPos}$. So we can just subtract one position from the other to get the relative position. This is how `addRecord()` will work for satellites, it will find the host's current position and then add $\text{record} - \text{hostPosition}$ instead of just adding `record` to the array of records, this way we have a history of all the *relative* positions of the satellite, not the actual positions.

`addRecord()` will use the same trick as in `celestialBody` to ensure that the list of records does not exceed the desired length.

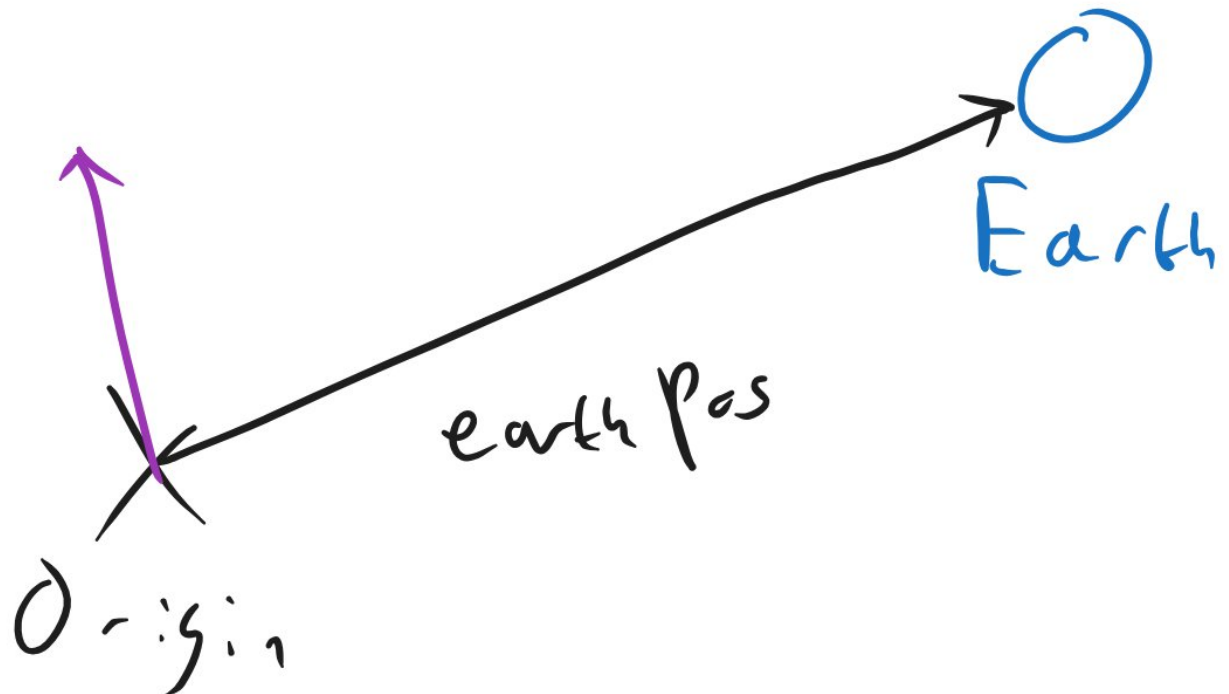
Here is pseudocode explaining how it will all work:

```

1  PROCEDURE addRecord(self, record)
2      hostPos = self.getHost().getPos()
3      record = record - hostPos // Recording the *relative* motion rather than absolute motion
4      self.__records.APPEND(record)
5  IF LEN(self.__records) > LINE_LENGTH THEN // Same code as used in the normal addRecord
6      self.__records ← self.__records[LEN(self.__records)-LINELENGTH:]
7  ENDPROCEDURE

```

Next, we need to consider how `getRecord()` will work. If we just return the records as is, the vectors from the Earth to moon, and try and draw them, we have a problem. The program doesn't know to start at the Earth and then follow the vector to get to the point where the line is supposed to be drawn. It will instead start at the origin and then follow the vector. For example, if we use the same purple arrow as in the previous diagram, we'd get this:



Which is not good, as that is nowhere near the Earth.

There is a very simple fix though, all we need to do is add the Earth's current position to all the records. This can be done with a for loop, like this:

```

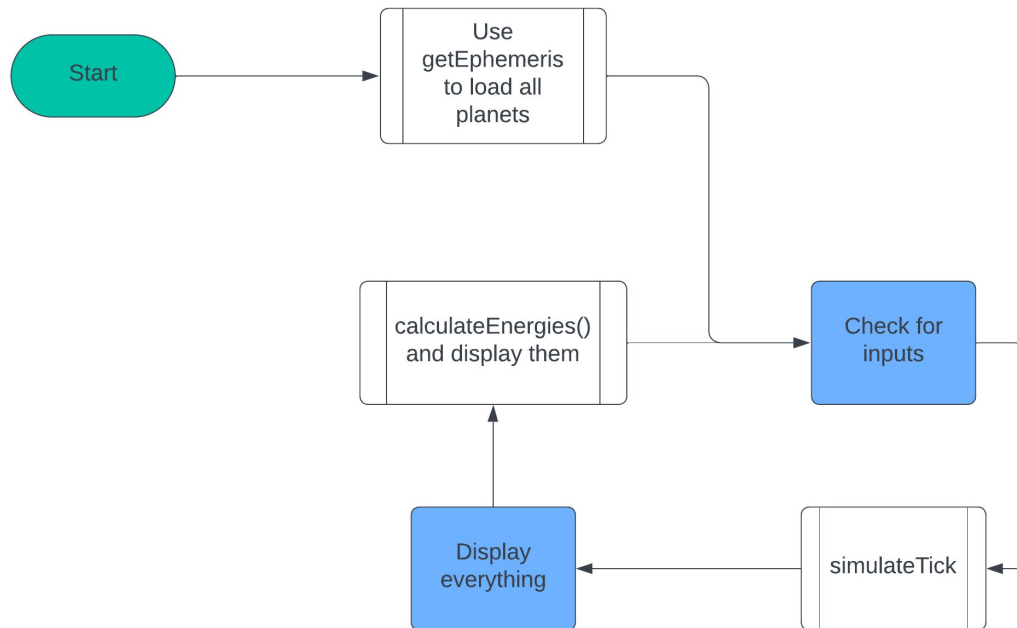
1 FUNCTION getRecord(self)
2     updatedRecords ← []
3     hostPos ← self.getHost().getPos()
4     FOR i ← 0 TO LEN(self.__records)-1
5         updatedRecords.append(self.__records[i]+hostPos)
6     ENDFOR
7     RETURN updatedRecords
8 ENDFUNCTION

```

Note that this will be done with list comprehension when I implement it in python to make the code more pythonic.

Overall system design

This is a system flowchart explaining how my program works:



`getEphemeris()` uses Nasa's Horizons database to get the positions of the planets at the date and time the program. The program then enters its main loop. It checks for any inputs in case the user has tried to do anything, it simulates the tick and does all the physics, it displays the planets, and then it calculates the energies and outputs them to the console to demonstrate conservation of energy as requested by client. In my system parts section I will explain everything in the main loop of this flowchart in detail.

Initialisation

Here I will explain everything to do with how the program is started, including all the variables created, as well as how `getEphemeris()` is used to instantiate all the celestial body objects. Essentially everything before getting to the loop in the system flowchart.

3 things happen in the initialisation. All the necessary variables are defined, every planet that's going to be simulated is instantiated as an object and pygame is set up so that we can start displaying everything. The important variables are mentioned above in the Data Dictionary, so let's cover the other two parts.

Adding the planets

A planet requires 6 attributes to be instantiated: Name, size, mass, position, velocity and colour. A satellite additionally requires a host to be given. NASA provides four of these, the size, mass, position and velocity in their ephemerides, which will be referenced as `ephemeris[0]`, `ephemeris[1]` etc as `getEphemeris()` returns a list. NASA does also provide a name but it's

incredibly hard to extract due to how they present it, and because the name won't change over time (unlike velocity and position) I've decided to ignore that. So, name and colour will have to manually be given. To get the ephemeris, we also have to pass in an ID, that will also have to manually be given.

The colours for the planets are fairly arbitrary. I've just picked colours that I think represent the planets quite well, so e.g. the sun is yellow, Earth is blue and so on. I've chosen solid colours instead of pngs of the planets to keep the program simple, and it makes it easier to modify to create custom solar systems with made up planets where you don't necessarily have images of them.

My program will include the Sun, all 8 planets, Pluto and Voyager 1 and 2. The sun and planets are in there because, well, they're what make the solar system. If my solar system simulator didn't have planets it would be a pretty bad one. Pluto has been added as it's still recognisable and historically was a planet, and the voyagers have been added for educational purposes as they're an important part of human history and something that children should know about. It's also cool to see how far away they are from the rest of the solar system. I've decided to only add our moon, as adding Jupiter and Saturn's moons would quickly get out of hand (they have hundreds of them.) 600 objects all circling around Jupiter is not something my program could cope with.

Each time a planet object is created, it will be appended to a list, "planets", which as mentioned in the Data Dictionary will contain all celestial objects. This list should contain the planets roughly in order of how far away they are from the Sun, so it should go Sun, Mercury, Venus, ... Neptune, Pluto, Voyager 1, Voyager 2. This means the sun will be instantiated first so that it is the first object appended to the list.

The ID for the planets follows a basic pattern. Mercury is 199, Venus is 299, Earth is 399, and so on up to Pluto being 999 (I guess NASA still counts Pluto as a planet, silly organisation.) This means we can iterate through them with a loop, and have a list of planet colours and names to iterate through at the same time to easily create all the planets. The sun, moon and the two voyager satellites will need to be created manually as their IDs do not follow any simple pattern.

As mentioned, the sun will be instantiated first.

The moon should be created just after the Earth is created, so that if you go to the next celestial object from Earth it takes you to the moon. The moon is a satellite with Earth as its host, so it will be given Earth for its host attribute.

Jupiter has an odd quirk. Its mass is given in grams, not the SI unit, kilograms. It is the only planet with this trait. So, a check will need to be made to see if a planet is Jupiter, once we do get to Jupiter, we need to divide its mass by 1000 to convert it to kilograms before it is added to the list of planets.

The voyager satellites' ephemerides do not give their mass and size, so those will be provided manually by me. This will require using the "ignoreMassSize" flag in the `getEphemeris()` function. Because the Voyagers need this special treatment with `getEphemeris`, they will be created manually without their names or colours being in a list like the sun and planets.

Here is pseudocode putting that all into effect:

```

1  FUNCTION generateSolarSystem()
2      planets ← []
3
4      planetNames ← ["Sun", "Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn", "Uranus",
"Neptune", "Pluto"] // Naming all the planets
5      planetColours ← [(255, 255, 0), (65, 68, 74), (139, 115, 85), (0, 0, 255), (255, 99, 47),
(250, 164, 87), (195, 146, 79), (98, 174, 230), (67, 109, 252), (204, 168, 132)] // Giving colours
to all the planets
6      moonNames ← ["Moon", "Io", "Europa", "Ganymede", "Callisto"]
7      moonColours ← [(111, 109, 114), (253, 245, 144), (68, 169, 241), (92, 88, 76), (70, 103, 97)]
8      ephemeris ← horizonsParser.getEphemeris(10) // Adding the sun
9      planets.append(planet(planetNames[0], ephemeris[0], ephemeris[1], ephemeris[2], ephemeris[3],
planetColours[0]))
10
11  FOR i ← 1 TO 9 // Adding all the planets
12      ephemeris ← horizons.parser.getEphemeris(i*100+99) // Planet IDs are all in the form x99
with x being between 1 and 9
13  IF i = 4 THEN // Add the moon after having added the Earth
14      moonEphemeris ← horizonsParser.getEphemeris(301)
15      planets.append(satellite(moonNames[0], moonEphemeris[0], moonEphemeris[1],
moonEphemeris[2], moonEphemeris[3], moonColours[0], planets[3])) // Moon is created with satellite
instead of planet so that its orbital lines are relative to Earth's motion. Earth (planets[3]) is
provided for its host attribute.
16  ENDIF
17  IF i = 5 THEN
18      ephemeris[1] ← ephemeris[1]/1000 // Converting Jupiter's mass to kilograms.
19  ENDIF
20  IF i = 6 THEN // Add jupiter's moons after jupiter has been added but before Saturn is
added
21  FOR j ← 1 TO 4 // IDs follow a pattern, 501, 502 etc so can be generated
22      moonEphemeris ← horizonsParser.getEphemeris(500+j, True)
23      planets.append(satellite(moonNames[j], moonEphemeris[0], moonEphemeris[1],
moonEphemeris[2], moonEphemeris[3], moonColours[j], planets[6]))
24
25      planets.append(planet(planetNames[i], ephemeris[0], ephemeris[1], ephemeris[2],
ephemeris[3], planetColours[i])) // Add the planet to the list of planets using the ephemeris data
26  ENDFOR
27
28  // Manually adding voyager 1 and 2 as we need to set their mass & size because NASA doesn't
provide it.
29  ephemeris ← horizonsParser.getEphemeris(-31, False, True, 722, 13) // Voyager 1
30  planets.append(planet("Voyager 1", ephemeris[0], ephemeris[1], ephemeris[2], ephemeris[3],
moonColour)) // moonColour is used as it is grey, and satellites are grey, so I thought it would
be close enough.
31
32  ephemeris ← horizonsParser.getEphemeris(-32, False, True, 722, 13) // Voyager 2
33  planets.append(planet("Voyager 2", ephemeris[0], ephemeris[1], ephemeris[2], ephemeris[3],
moonColour))
34  RETURN planets
35  ENDFUNCTION

```

The list planets will be set to the output of this function.

Setting up pygame

This will be a simple section, there's just a list of different things we need to create for pygame to work properly.

First thing's first, `LINE_LENGTH`. As mentioned in the data dictionary, we already have `MAX_LINES`, which determines how many total orbital lines can be on the screen at once. Now that we know how many planets we have, we can use this to work out the length of the orbital line of each of the planets. We simply divide `MAX_LINES` by the amount of planets there are, and int it. Aka set `LINE_LENGTH = MAX_LINES/(len(planets))`

However I'll actually do `MAX_LINES / (len(planets)-3)`, because the moon and the two voyagers are hardly ever visible and don't really impact performance, so we can ignore them.

Next thing to do is to run `pygame.init()`, without this pygame will not do anything. We also need to create our screen. I will do this with `pygame.display.set_mode((0,0), pygame.FULLSCREEN)` as this means pygame will handle the size of the window. This lets the program work on monitors of any resolution, thus making it more accessible to people with different kinds of monitors.

We can also use the window that pygame just created after we made our screen to find the centre of the screen, which is needed for `scaledPos()` to scale things accurately so that the focus is always at the center. To do this, we simply get the width and height of the window with `pygame.display.get_surface().get_size()`, and then divide that by 2 to get the centre. This array will be called `centre`, so `centre[x]` is the x position of the centre and `centre[y]` is the y position. This also means that `centre[x]*2` and `centre[y]*2` are the width and heights of the window respectively.

We need to create the clock for keeping a consistent tickrate, this is done with `pygame.time.Clock()`.

We need to create two surfaces used for comparing planets, `comparisonSurface1` and `comparisonSurface2`. They are both made `centre[x]-3` wide and `centre[y]*2` tall, this means that they take up the full screen vertically and about half the screen each horizontally, with a 6 pixel gap in the middle for a barrier between the two surfaces.

`planetsToCompare` is created, this is a length 2 list containing the index of the first planet and second planet to compare, so `planets[planetsToCompare[0]]` returns the object for the first planet and `planets[planetsToCompare[1]]` returns the object for the second planet.

And finally, we need to create the interface. This means selecting a font (I use `codenewroman` at size 18, and the same font at size 32 for the start menu) and creating two menu surfaces for the text to be displayed on. These menu surfaces will have lowered opacity to make them look nicer and so that they don't block out objects behind them.

Now we're done setting up the program, we can look at the main loop, which is what the next section is on.

System parts

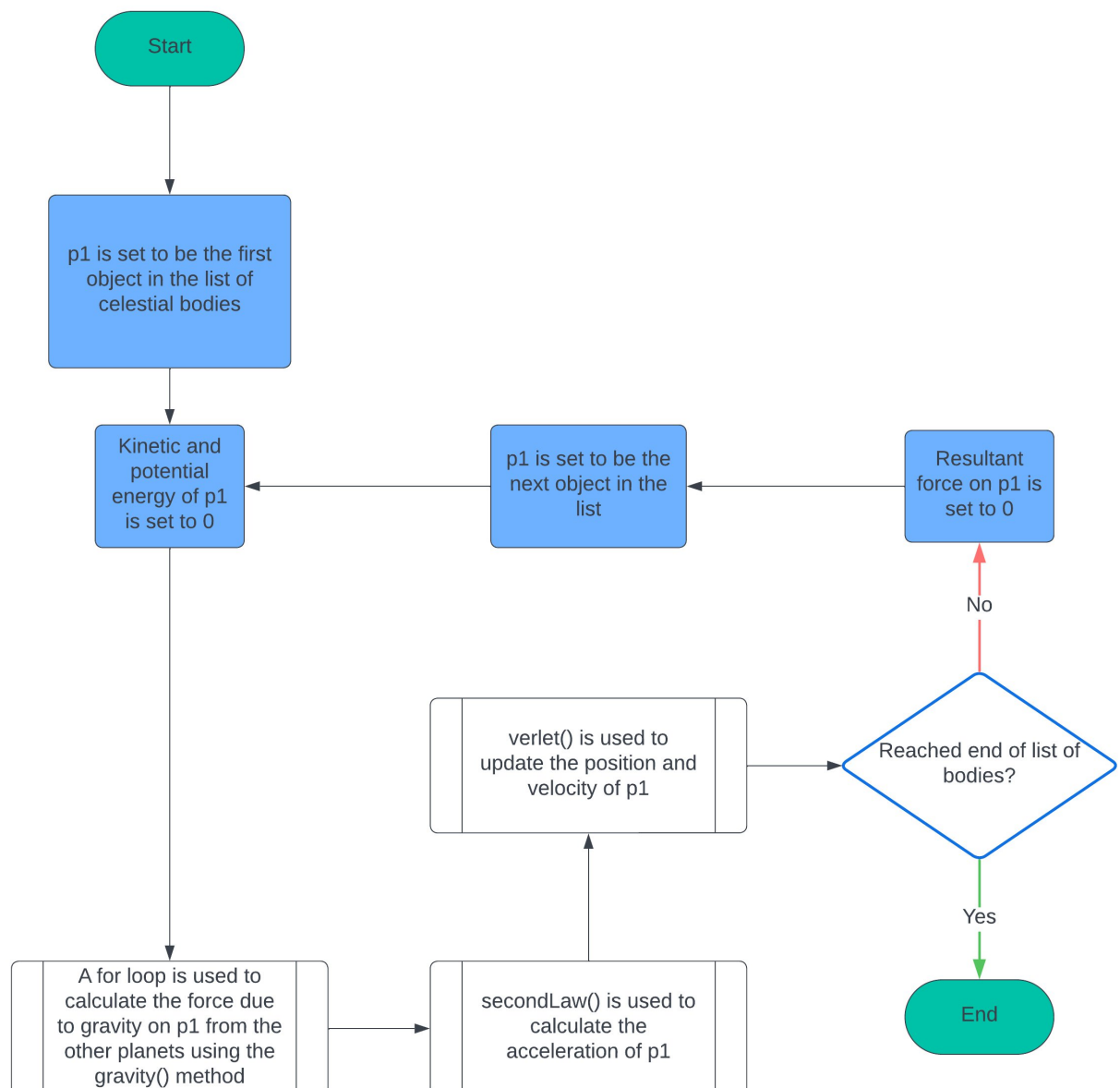
Here I will explain the main loop in the system flowchart.

There are three main parts to the loop, the physics, which governs how the planets actually move, the display, which shows the solar system and includes some tricks to make the camera move / follow other planets, and the inputs / interface which lets the user interact with and see details about the program.

Simulating a tick

Here I will explain the physics in the program, the part that makes the planets actually behave like planets, accurately. I will describe the behaviour of `simulateTick()`, which is the function that does all the physics for a single tick, as well as handling some information that will later be needed for the display part of the program.

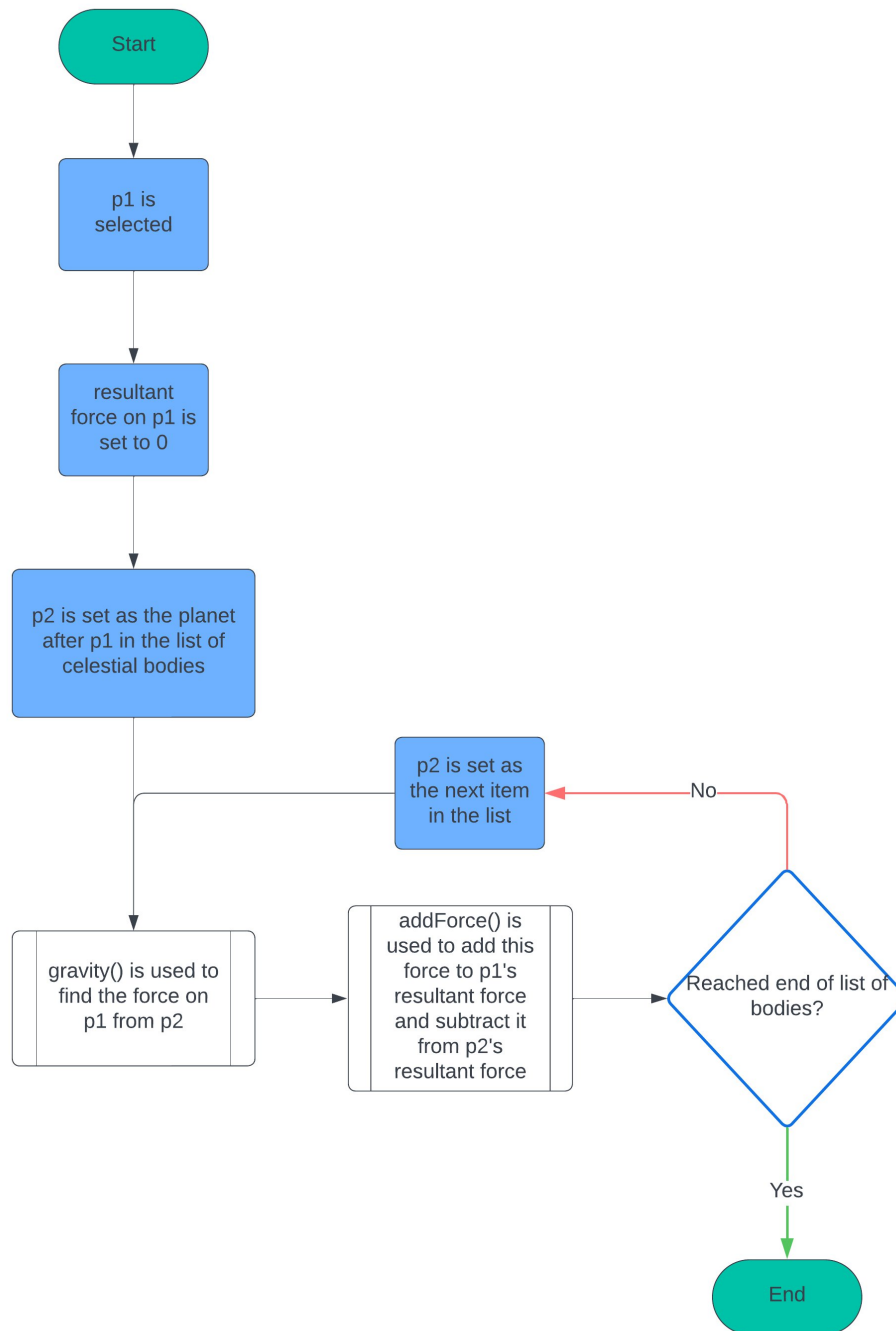
The flowchart below gives a high level overview on how physics is simulated during a single tick in my program:



Here, the function (simulateTick()) takes in planets, focus and comFocus. Planets is a list of all the celestial bodies, focus and comFocus are for the display and not relevant yet. simulateTick() iterates over this list to do calculations for all the planets. It sets the kinetic and potential energies of the planet to 0, they are worked out during the verlet integration and gravity calculations. It works out the resultant gravitational force on each planet, and then uses secondLaw() to work out the acceleration on each planet. Verlet integration is then performed (explained later below) to get the new position and velocity of the planet. The resultant force on the planet is then reset to 0 so it can be calculated next tick.

The program needs to be able to draw orbital lines showing the paths of planets. To do this we will draw lots of small lines that will create what looks like a smooth trajectory. To draw the small line, we need two points. Those two points will be the position of the planet at the start of the tick, i.e. before verlet() is run (that's when its position is updated) and its position after verlet(). These two points will be added to the lines attribute of the planet.

The flowchart below describes the for loop used to calculate the gravitational force on a planet.



Force is a vector quantity, this means to find the resultant force on a planet, you have to find all of the individual forces on it from other planets and sum them. Hence, we use a for loop to iterate through the list of planets and use the gravity() function to find the forces. We then add each force to the resultant attribute of a planet until we've reached the end of the list.

We know however, because of Newton's third law, that each action has an equal and opposite reaction. This means that the force on p1 from p2 is the negative of the force on p2 from p1. So, to save some iterations, we can also subtract the calculated force from p2's resultant force. This means that each for loop starts at the planet just after p1, rather than at the start of the list of planets every time.

Also, we want the option to display arrows illustrating the forces on the currently selected planet. To do this, we check with an if statement if the p1 or p2 are selected, and if so, we add the force acting on them from the other to their arrows attribute, and we make these arrows white.

At the end of the loop, if p1 is the selected planet, we add another arrow representing its resultant force, and make this arrow the same colour as the planet to signify that it's the resultant.

Now that we've covered how simulateTick works, here is some pseudocode with comments to demonstrate it more concretely:

```

1  ▾  PROCEDURE simulateTick(planets, focus)
2      FOR p1 IN planets
3          p1.addGPE(-p1.getGPE()) // Resetting energies and the momentum to 0
4          p1.addKE(-p1.getKE())
5          p1.addMomentum(-p1.getMomentum)
6
7          FOR p2 in planets[planets.index(p1)+1:] // don't need to consider planets that have
already been looked at so list slicing is used to avoid them, this makes the program a lot faster
8              p1gravity ← p1.gravity(p2) // Gravitational force on p1 from p2
9              p1.addForce(p1gravity)
10             p2.addForce(-p1gravity) // Newton's third law, equal and opposite forces
11
12             // Drawing force arrows if the planet is selected
13  ▾  IF planets.index(p1) = focus:
14              p1.addArrow(["white", p1gravity])
15          ENDF
16  ▾  IF planets.index(p2) = focus:
17              p2.addArrow(["white", -p1gravity])
18          ENDF
19      ENDFOR
20      // Drawing resultant arrow with the colour of the planet
21  ▾  IF planets.index(p1) = focus:
22          p1.addArrow([p1.getColour(), p1.getResultant()])
23
24          p1.secondLaw() // calculate acceleration
25          p1.verlet() // update velocity and position of planet
26          p1.addRecord([p1.getPos()]) // adding a record of the planet's position so we can draw
orbital lines
27          p1.addForce(-p1.getResultant) // resetting force to 0 so it can be calculated again next
tick
28      ENDFOR
29  ENDPROCEDURE

```

Methods used

All the methods used in simulateTick() are methods of the celestial body class, because simulate tick is operating on attributes of celestial bodies (their positions, velocities, etc.) This is nice as it means functions do not need to take many arguments, as a lot of the values they need can be found as attributes of the object they're operating on.

I won't explain the basic getter / setter methods like `addArrow` and `getPos` as they're not relevant to the physics and are explained in the section on classes.

`addGPE / addKE / addMomentum`

Self explanatory, pass in a value and it adds that value to the GPE or KE attribute of the body respectively. These exist because one of the purposes of the program is to demonstrate that certain physical properties (momentum, energy) are conserved. Because my program uses the laws of physics to do the simulation, the quantities are indeed conserved as expected.

`gravity`

This uses Newton's law of gravitation to find the force on one planet from another. This was explained in the analysis, but for a quick recap, here is how to find the magnitude of the gravitational force on a planet, `p1`, from another planet, `p2`:

$$F = \frac{Gm_1m_2}{r^2}$$

Where r is the vector from planet 1 to planet 2.

This is calculated by finding the vector from `p1` to `p2`, by doing `p2.getPos() - p1.getPos()`.

But force is a vector quantity, so the full equation is:

$$F = \frac{Gm_1m_2}{r^2} \hat{r}$$

Where $\hat{r}$ is the unit vector pointing from `p1` to `p2`, calculated using the `unit()` function from the vector module.

It also makes sense to use this function to calculate the GPE of the planets at the same time, to save on calculations later. If we already have the force it is very easy to calculate GPE, the GPE of planet1 from planet 2, E , is given by:

$$E = -F|r|$$

Where $|r|$ is the magnitude of the vector r . The GPE on planet 2 is the same but with the opposite sign. So we just add these values to the GPE attributes of the respective planets.

Here is commented pseudocode putting these ideas into action:

```
1  FUNCTION gravity(self, planet2)
2      r ← planet2.getPos() - self.getPos() // Using vector properties to find the vector from self
      to planet 2
3      F ← G*(self.getMass()*planet2.getMass())/(vec.mag(r)**2) // Newton's law of universal
      gravitation
4      self.addGPE(-F*vec.mag(r)) // Updating the GPE of self and planet 2
5      planet2.addGPE(F*vec.mag(r))
6      RETURN F*vec.unit(r) // return force as a vector so that it can be used by other functions in
      the program
```

secondLaw

This uses $F = ma$, Newton's second law of motion, to set the acceleration attribute on the planet, which can then be used by the verlet integrator to update the positions and velocity. It's given the resultant force of the planet, which will have been calculated by summing the results of gravity() in the nested for loop section of simulateTick()

verlet

Verlet integration was explained in the analysis, but for a quick recap, here are the formulas for finding velocity and position with verlet integration:

$$x_{n+1} = x_n + v_n \Delta t$$

$$v_{n+1} = v_n + a_n \Delta t$$

Here Δt is the timeScale variable in the program.

As verlet is updating velocity, it is a convenient time to set the properties of the planet relating velocity, the kinetic energy and momentum.

Here are the formulas for both (E is kinetic energy and p is momentum):

$$E = \frac{1}{2}mv^2$$

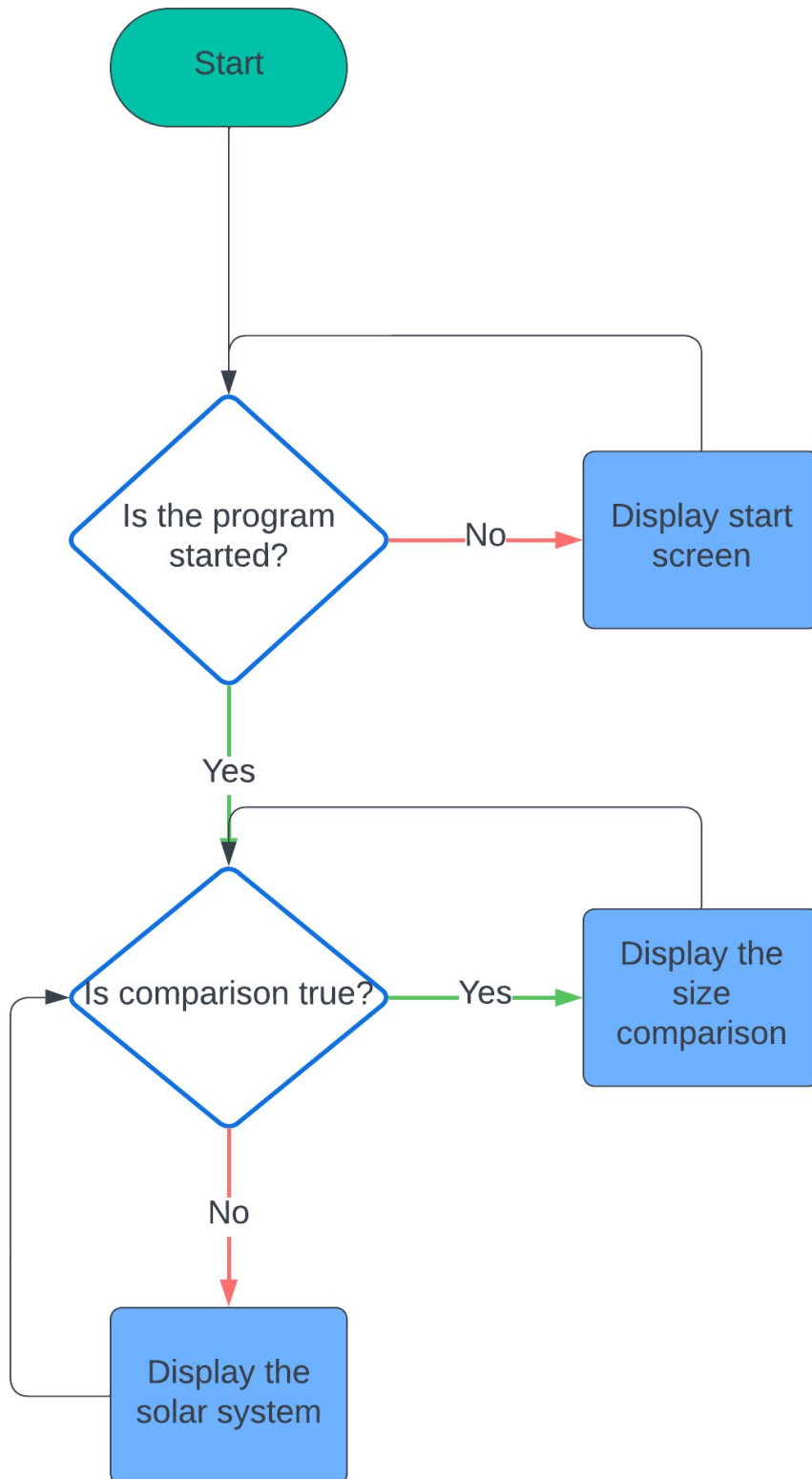
$$p = mv$$

Here is some pseudocode for performing the verlet integration:

```
1  ✓  PROCEDURE verlet(self, timeScale)
2      self.__vel ← self.getVel() + self.getAccel()*timeScale // v1 = v0 + a*dt where v0 and v1 are
      old and new velocity
3      self.__pos ← self.getPos() + self.getVel()*timeScale // x1 = x0 + v*dt where x0 and x1 are
      old and new positions
4      self.addKE(0.5*self.getMass()*vec.mag(self.getVel())**2) // E = 0.5mv2
5      self.addMomentum(self.getMass()*self.getVel()) // p = mv
```

Display

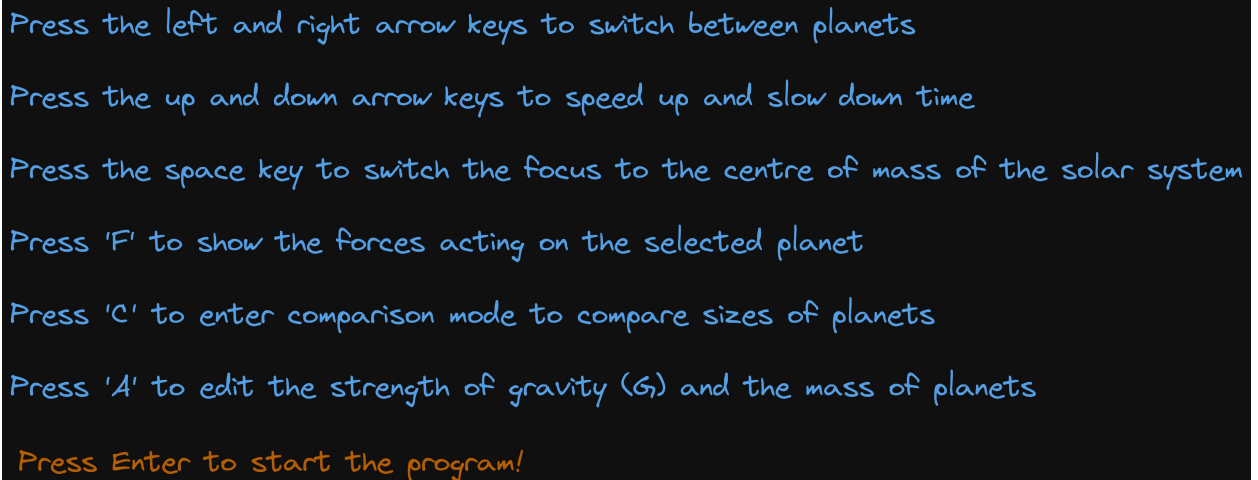
There are three states the display can be in. When the program is first launched it will enter a loop where it displays the start screen. Once the user presses enter to start the simulation a boolean variable, started, will be set to true, so that the program can exit the loop. The display will then either show the solar system and let the user switch focus between different planets, or show two planets side-by-side so their size can be compared. Which it does will be dependent on whether or not a boolean variable, comparison, is true. Here is a flowchart demonstrating how the state of the display is decided:



Let's look at these states individually.

The start screen

As shown in the analysis, the start screen will be a collection of text explaining to the user how the program works, and telling them how to start the program, like this:



Press the left and right arrow keys to switch between planets

Press the up and down arrow keys to speed up and slow down time

Press the space key to switch the focus to the centre of mass of the solar system

Press 'F' to show the forces acting on the selected planet

Press 'C' to enter comparison mode to compare sizes of planets

Press 'A' to edit the strength of gravity (G) and the mass of planets

Press Enter to start the program!

This will be done very simply in code, there will be a boolean variable, started, set to False initially. A while loop will continuously run displayStartScreen() which just uses pygame's font.render() method to display all the text, and it will check for inputs. If enter is pressed, then started will be set to True and the program will exit the loop.

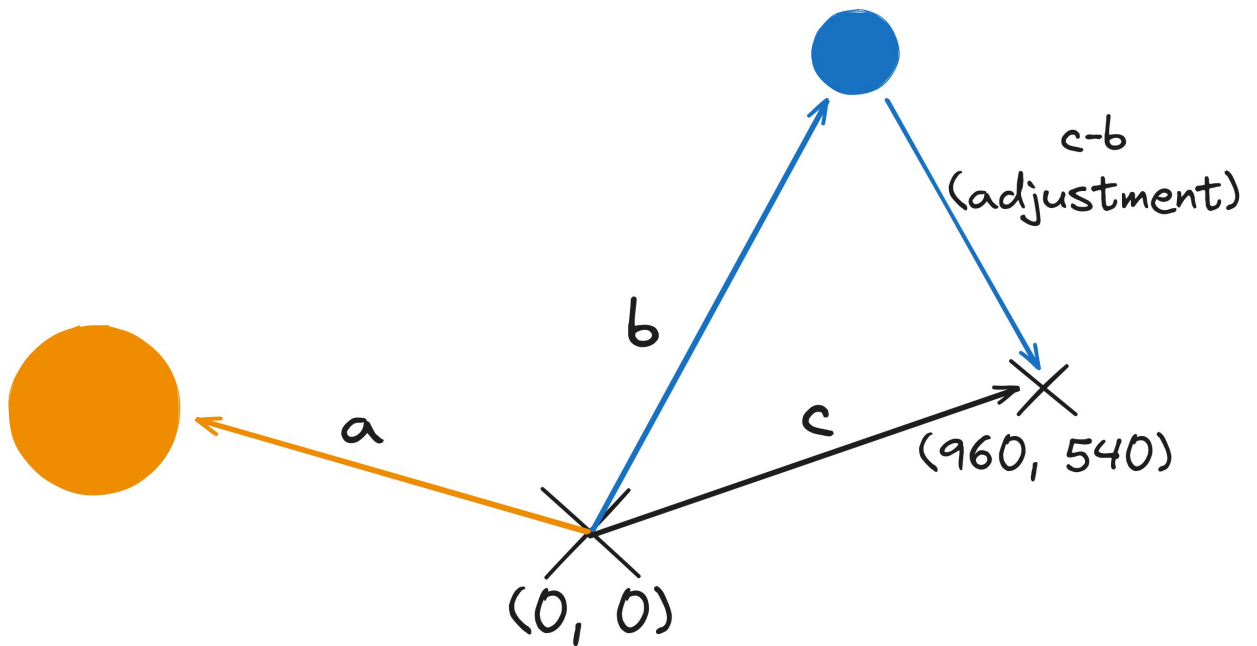
Displaying the solar system

The purpose of the display is to give a view of the solar system, enable zooming in and out, and have the ability to focus the centre of the view (the "camera") on any specific object / the c.o.m. (this is an actual acronym used by physicists for the centre of mass) of the system.

But the solar system is huge, so how can we display it accurately? This is done by scaling the positions of the planets. Inside the display, instead of using the actual positions of planets we used their scaled positions. This just means we multiply their position vector by a scalar, the distConst. distConst is effectively a conversion from metres to pixel, it's how many pixels are in a metre. scaledPos(vec) will do this for us, and return that vector*distConst. Doing everything this way will preserve the relative sizes and distances of the system. This will make the program educationally useful as it helps kids understand the scale of the solar system and so is suitable for the client.

distConst will be determined by how zoomed in the user is, a smaller distConst means a more zoomed out solar system.

Next, we need to make the display be centred on a specific planet. This means we want the planet in question to be displayed at ($w/2$, $h/2$) on the screen where w is the width in pixels of the window and h is the height. This is calculated by using pygame's get_size() method to get (w , h) and then dividing it by 2. Here is a diagram showing the situation:

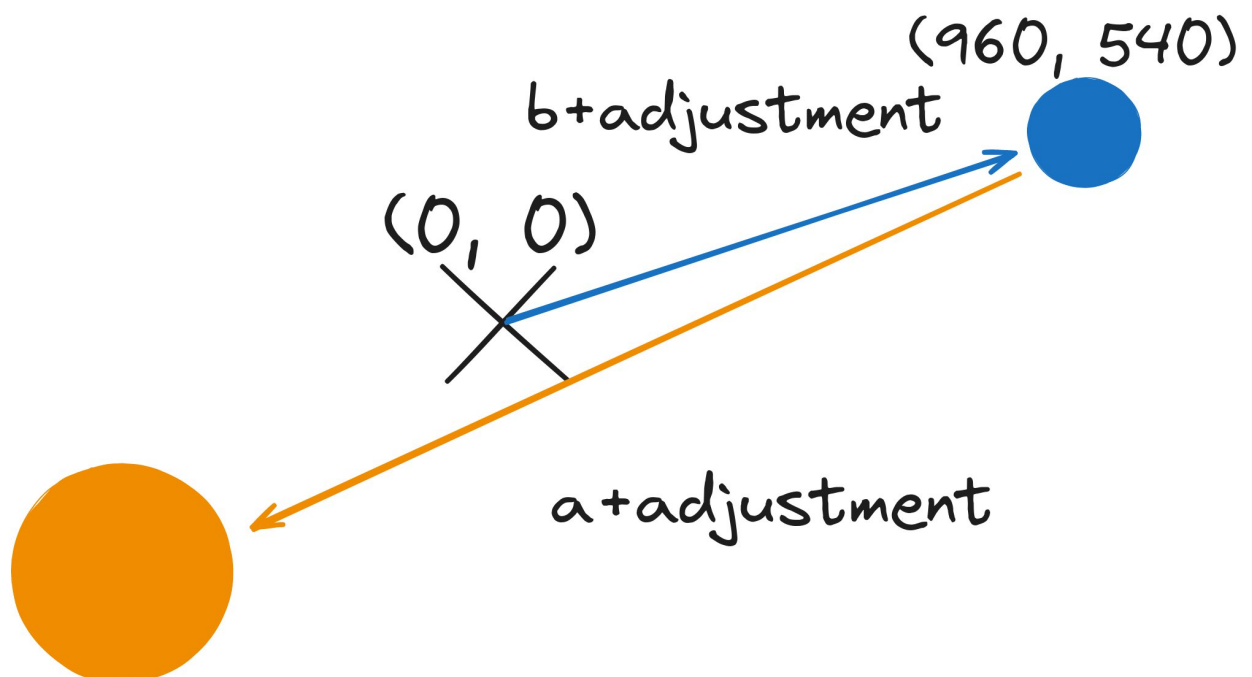


Here $\mathbf{a}$ is the scaled position vector of the sun and $\mathbf{b}$ is the scaled position vector of Earth. $\mathbf{c}$ is the centre of the screen, (960, 540) for a standard 1080p display. The “adjustment” is calculated, the adjustment is the vector pointing from Earth to the centre of the screen, $\mathbf{c} - \mathbf{b}$. If we were to add this adjustment vector to Earth’s position vector, we would just end up with $\mathbf{c}$, the location of the centre:

$$\mathbf{b} \text{ (Earth's position)} + \mathbf{c} - \mathbf{b} \text{ (adjustment)} = \mathbf{c}.$$

So if we calculate the adjustment and then add it to Earth’s position, and then display Earth at that location, Earth will be at the centre of the screen.

If we want to display the sun we now just add the adjustment to its position vector. This will put the sun at the same position relative to Earth as it was before. The result is this:



So Earth would be displayed at the centre of the screen, and the sun would be displayed in the bottom left as it is down and to the left from Earth. We can repeat this process for any planet we want, to display it we just scale its position vector by `distConst` and then add the adjustment to that vector. We then display a circle at that position on the screen. If we want to focus on the centre of mass instead of a planet, that is very similar, the maths is the exact same except you use the scaled position vector for the c.o.m. Here is pseudocode for `focusAdjustment()`, the function that calculates the adjustment:

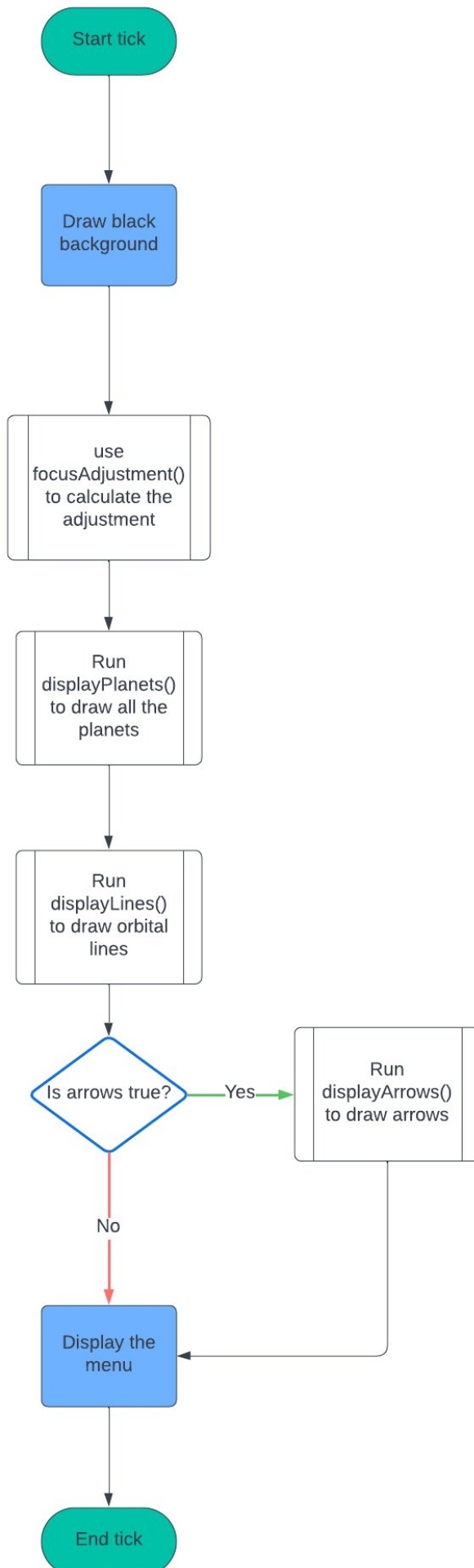
```

1  FUNCTION focusAdjustment(planets, focus, comFocus)
2  IF comFocus = TRUE THEN
3      currentFocus ← com(planets)
4  ELSE
5      currentFocus ← planets[focus].getPos() // Setting the focus to be either a planet or the
        centre of mass
6  ENDIF
7      focusDisplacement ← center - scaledPos(currentFocus)[:2] // Calculate the adjustment. The
        [:2] in the list is necessary as the adjustment should only be 2d, we're only displaying a 2d
        image.
8  RETURN focusDisplacement
9  ENDFUNCTION

```

So, for every object that I want to draw (orbital lines and planets pretty much), I will take their position, scale it and add the adjustment and draw it at that position. Now that we understand that let's look at how the solar system is actually displayed.

This is the render pipeline for displaying the solar system:



First we draw a black background, because space is black so the background should be black. Next the adjustment is calculated as this is needed to draw any object as explained above. Then we draw the objects in the order planets, lines, and if applicable, arrows. This order is chosen as sometimes with objects that don't move much because they're very massive (e.g. the sun) will have visible movement, but the orbital path will stay entirely inside the object. Drawing paths after the planets lets that path still be visible. Likewise with arrows, for very small forces the arrow is contained entirely within the planet. Drawing the arrow after drawing the planet lets it still be visible. Finally the menu is displayed, showing information on attributes and the currently selected planet. The text should be visible above everything else which is why the menu is the last part to be rendered. The menu itself will be covered in the next section on interface, so is not described here.

To make the program more optimised, objects are not drawn if they're offscreen, so I have made a function, `offscreen()`, to check if objects are out of view.

I will now go through each method individually.

Methods used

offscreen

This is very simple. A vector is offscreen if either its x or y coordinate is less than 0 or greater than the width or height of the screen respectively. So an if statement is just used to check this, here's pseudocode showing how:

```

1  FUNCTION offscreen(vector)
2  IF vector[x] < 0 OR vector[x] > centre[x]*2 OR vector[y] < 0 OR vector[y] > centre[y]*2 THEN
    // x = 0 and y = 1, they're used for readability and consistency with mathematical notation
3  // Return true if any of the coordinates are negative or are greater than the width or
    height of the screen.
4  RETURN TRUE
5  RETURN FALSE

```

focusAdjustment

This was explained earlier in the section with pseudocode.

scaledPos

Very simple, literally just multiplies the input by `distConst`, here is the pseudocode:

```

1  FUNCTION scaledPos(position)
2  GLOBAL distScale // distScale is only ever edited by one function, zoom(), other than that
    it's never changed so it's okay to use it as a global variable
3  RETURN distScale * position
4  ENDFUNCTION

```

If you're questioning why this is even a function, it's to improve the readability of the program. It makes it clear I'm scaling the position.

drawPlanet

drawPlanet() needs to be able to draw a planet at any place on any surface. This is needed for the comparison mode as in that mode we are drawing planets at specific points that aren't related to the position of the planet itself, and during the comparison the planets are drawn on surfaces other than the screen surface. It also needs the planet object itself to get the size and colour of the planet. These will all be passed in as arguments to the function.

The pygame.draw.circle() method will be used to draw the planet as a solid coloured circle. It will be given the surface to draw the circle on, get the colour of the circle with p.getColour(), draw the circle at the position given to it, and draw the circle with a radius of the scaled size of the planet, distScale*p.getSize().

Telling pygame to draw a very large circle (i.e. millions of pixels in diameter) can cause a lot of lag. This would occur in my program if you were very zoomed in on a planet for whatever reason. To avoid this, a check will be done to see if the scaled radius of the planet is larger than the diagonal radius of the screen. This can be done by seeing if $\text{distScale} * \text{p.getSize()} < \text{vec.mag}(\text{centre})$ or not. If this happens, then a circle will not be drawn, and instead the surface will just be filled with the colour of the planet.

Here is pseudocode for all of this:

```

1  ▾  PROCEDURE drawPlanet(p, position, surface)
2      GLOBAL distScale
3  ▾  IF distScale*p.getSize() > vec.mag(centre) THEN // Avoids lag if the circle being drawn is
      very large
4      surface.fill(p.getColour())
5      ELSE
6      pygame.draw.circle(surface,p.getColour(),position,p.getSize()*distScale)
7      ENDIF
8  ENDPROCEDURE

```

displayPlanets

Drawing the planets is done with displayPlanets which runs a for loop iterating over the list planets and uses drawPlanet() to display each one. It only runs drawPlanet() if the planet would be visible to save on resources, no point drawing planets offscreen. It uses offscreen() to check this. It passes in the scaled and adjusted position of the planet to drawPlanet() so that the circle is drawn at the correct position. Here is pseudocode showing this:

```

1  ▾  PROCEDURE displayPlanets(p, planets, surface)
2      FOR p IN planets
3  ▾  IF offscreen(p.getScaledPos()[:2]+adjustment) = TRUE THEN // Don't draw planet if it
      wouldn't be visible
4      CONTINUE
5      ELSE
6      drawPlanet(p, p.getScaledPos()[:2]+adjustment, surface)
7      ENDIF
8      ENDFOR
9  ENDPROCEDURE

```

displayLines

`displayLines()` works in a similar way with a for loop iterating over planets, but then it also needs to draw a few hundred lines for each planet. It does this by using `getRecords()` to get a list of the planet's previous positions in chronological order, with the oldest record being the first item in the list. This list will be 300 long in the standard solar system, applying `scaledPos()` individually to every vector in this list would be very computationally expensive, and a bad idea. Luckily since all `scaledPos()` does is multiply by a scalar, the `distConst`, we can use numpy arrays to do this operation all in one go, as you can multiply matrices by scalars in numpy. This is a massive performance increase and is the only reason the program runs at a reasonable framerate. At the same time adjustment is added to all of the positions. The list created is called `orbitalPath`.

A variable, `index`, is set to -1. This is used to keep track of what line we're on, as we want to draw older lines with a more faded colour. This will give a visual indicator of where the planet was at what time. If the line was all one solid colour it would not be clear where the planet was recently and where the planet was a while ago.

A for loop is then used to iterate over `orbitalPath`, drawing a line between each consecutive pair of positions. `index` will be incremented by one each loop and an "`indexRatio`" is calculated, this is `index/len(orbitalPath)`. This is used to determine the colour of the line.

There are two situations where we don't want to draw a line:

1. The line is offscreen so there is no point in drawing it. This is checked with `offscreen()`
2. The line belongs to a satellite who's host isn't focused on at the moment and the satellite itself isn't focused on. If the host is not focused on it's very unlikely the satellite would be visible as moons are so much smaller than planets, so in this case there's no point drawing it. To have access to this information `displayLines()` will be passed `focus` and `comFocus` as arguments, we will check first if the current planet is a satellite by seeing if `p.getHost() == None` or not and then check if the planet is focused on and if the planet's host is focused on.

If neither of these criteria are met then the line is drawn. This is done using `pygame.draw.aaline()` as it produces a smoother result than using `pygame.draw.line()`.

`pygame.draw.aaline()` needs a surface to draw the line on, the colour of the line and the start and end points for the line. The surface will be given to the function as an argument. The colour will be `indexRatio*p.getColour()`. This makes it so that older lines are drawn darker, with the darkest ones being drawn as black and completely fading into the background. The start and end points will simply be the current position and the next position in `orbitalPath`. Here is pseudocode showing all of this:

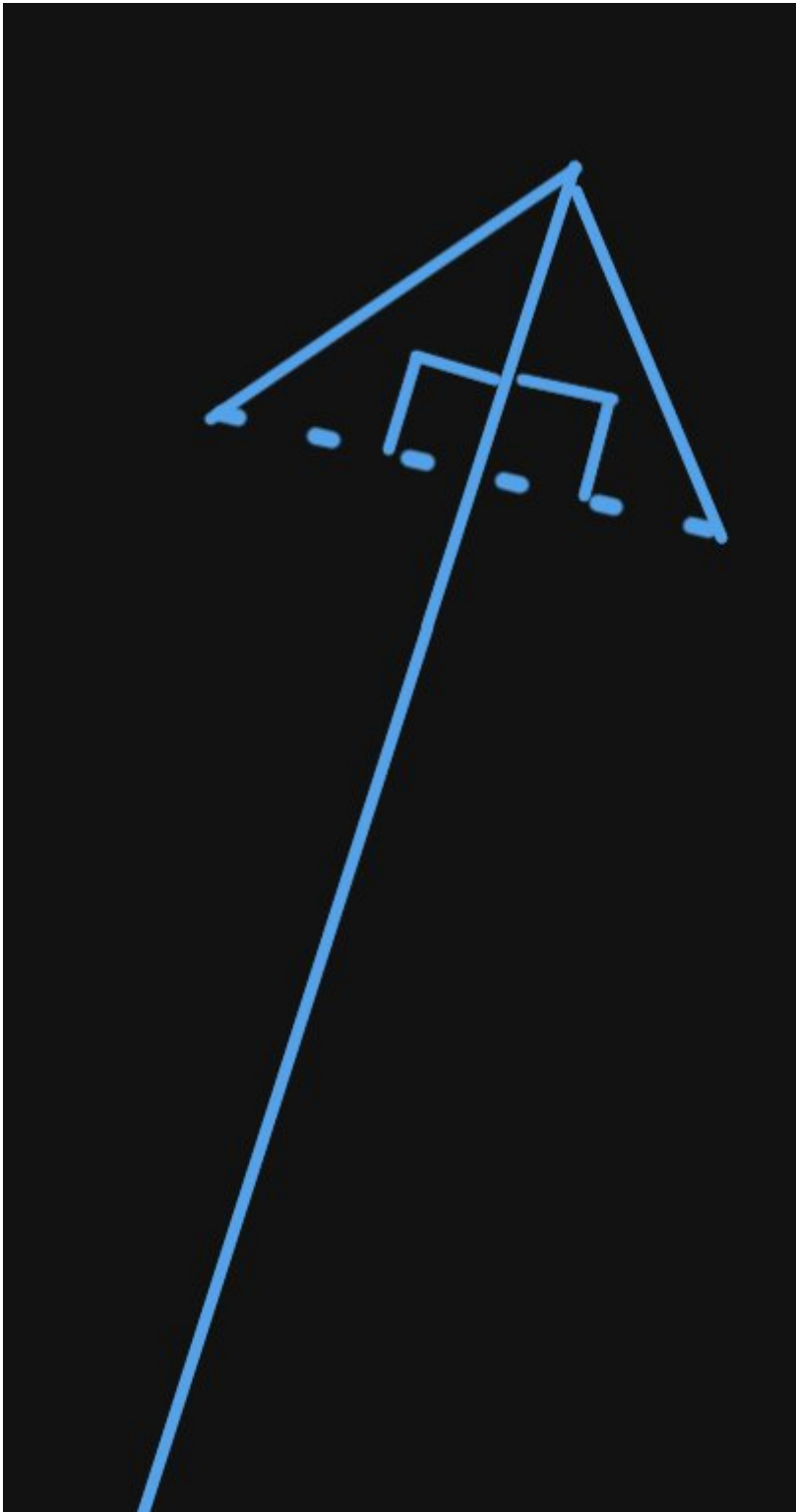
```

1 ✓ PROCEDURE displayLines(planets, adjustment, focus, comFocus, surface)
2     FOR p IN planets
3         index ← -1 // This will be used to create the fade effect for lines
4         orbitalPath = [i[:2]+adjustment for i in scaledPos(np.array(p.getRecords()))] //
scaledPos() is applied to the whole array of records at once which makes the program a lot faster.
The positions are then made 2d so they can be displayed and adjustment is added to all of them so
that they can be displayed in the right position.
5 ✓         FOR i ← 0 TO LEN(orbitalPath)
6             line ← [orbitalPath[i], orbitalPath[i+1]] // Create a pair of points that a line can
be drawn between
7             index ← index+1 // Increment the index
8 ✓             IF offscreen(line[0]) = TRUE AND offscreen(line[1]) THEN // Don't draw offscreen lines
9                 CONTINUE
10            ENDIF
11
12 ✓            IF p.getHost() ≠ NONE AND (p.getHost() ≠ planets[focus] AND p = planets[focus] OR
comFocus = TRUE) THEN // Check if planet is a satellite and if so if its host or the planet
itself is currently the focus. If not it is unlikely to be visible and the lines should not be
drawn
13                CONTINUE
14            ENDIF
15
16            indexRatio ← index/len(orbitalPath) // Calculate the indexRatio to create the fade
effect
17            pygame.draw.aaline(surface, ([int(indexRatio*colour)] for colour in p.getColour()),
line[0], line[1], TRUE) // Draw the line and scale the colour by indexRatio to make it fade to
black for older lines. The 1 in the last argument just tells pygame to blend the lines to make
them look nicer.
18        ENDFOR
19    ENDFOR
20 ENDPROCEDURE

```

drawArrow

drawArrow should be able to take in a surface to draw an arrow, the colour to draw the arrow, and the start and end points of that arrow. To do this, we need to think about how an arrow is drawn. An arrow is made up of 3 lines, like this:



The key thing to notice is that the two diagonal lines, the ones that make up the arrow head, lie on the perpendicular to the main line of the arrow. This means to draw an arrow we need to first draw a main line, and then pick two points that are on the perpendicular to that line near the tip. We then connect the tip to those two points with lines, and we have an arrow.

The way this is done in code is by making a vector representing the main line, this is `endPos - startPos` where `endPos` and `startPos` are the ends and beginning of the line respectively. Let's

call this mainVector, so $\text{mainVector} = \text{endPos} - \text{startPos}$. We then use `pygame.draw.aaline()` to draw a line between those two points.

Next, we calculate the normal to mainVector. This is done using the `vec.normal()` method from my vectors file. So $\text{normal} = \text{vec.normal}(\text{mainVector})$. To pick our two points, we start at our startPos, go most of the way along mainVector, and then go a tiny bit along the normal in one direction or the other, like this:

$\text{point1} = \text{startPos} + 0.9 * \text{mainVector} + 0.05 * \text{length} * \text{normal}$

$\text{point2} = \text{startPos} + 0.9 * \text{mainVector} - 0.05 * \text{length} * \text{normal}$

This will generate two points lying on the perpendicular to mainVector. Now all we have to do is draw two lines between these points and the endPos of the line.

Here is pseudocode showing how this all works:

```
1  PROCEDURE drawArrow(surface, colour, startPos, endPos)
2      startPos, endPos ← startPos[:2], endPos[:2] // [:2] is because the arrows are displayed in 2d
3      mainVector ← endPos - startPos
4      length ← vec.mag(mainVector)
5      pygame.draw.aaline(surface, colour, startPos, endPos) // Draw the main line
6      normal ← vec.normal(mainVector) // Used to generate the two points either side of the main
   line
7      point1 ← startPos + 0.9*mainVector + 0.05*length*normal
8      point2 ← startPos + 0.9*mainVector - 0.05*length*normal
9  ENDPROCEDURE
```

displayArrows

`displayArrows()` needs to draw arrows that come out of the currently selected planet based on the forces acting on that planet. The function should have a check to see if `comFocus` is true as if it is that means no planet is selected and it shouldn't display any arrows. A copy of the forces on the planet are stored in its `arrows` attribute and can be accessed with `p.getArrows()`. The currently selected planet can be found using `planets[focus]`.

The function will iterate over every arrow in `p.getArrows()` and draw them each.

`drawArrow()` takes 4 inputs, the surface, the colour, the startPos and the endPos. The surface will simply be passed into `displayArrows()` as an argument. The colour is just the first item stored in an arrow. The startPos should be the scaled and adjusted position of the planet.

The endPos is the complicated bit. It will be the adjusted and scaled position of the planet + some vector that represents the force. That vector should be in the direction of the force (referred to as `arrow[1]` from here on out) acting on the planet to accurately show the direction of it, and its length should be related to the size of `arrow[1]` as well as `distScale` so that arrows get smaller as you zoom out. However just making it directly proportional to `arrow[1]` would cause issues, as the forces are far, far too big to visible, and also the forces acting on an object like the sun are way bigger than forces acting on an object like the earth.

One idea would be to simply divide by the mass of the planet, this means the arrow would represent acceleration instead of force. The problem with this is that the arrows on the sun

would end up being the same length as the arrows on the earth, which would look silly as the sun is way bigger compared to the arrows so less of them would be visible, and also because it wouldn't convey the idea that the sun does have more force on it than the earth does.

What I've settled on to achieve a visually appealing and physically representative arrow is to instead divide by the log of the mass, this stops the arrows from getting too big but still means that larger masses have larger arrows. So the vector will be proportional to $\text{distScale} * \text{arrow}[1] / \log(p.\text{getMass}())$. The proportionality constant is arbitrary, I have chosen $2 * 10^{-11}$ as it leads to reasonably sized arrows.

So the `endPos = p.getScaledPos()[2]+adjustment+2*10-12*distScale*arrow[1][2]/log(p.getMass())`

Note that the `[2]` makes everything 2d.

Here is pseudocode recapping everything said here:

```

1  ✓  PROCEDURE displayArrows(planets, adjustment, focus, comFocus, surface)
2      GLOBAL distScale
3  ✓  IF comFocus = FALSE THEN // No point trying to draw arrows if no planet is selected
4      p ← planets[focus] // Set p to be the currently selected planet
5      FOR arrow IN p.getArrows()
6          drawArrow(surface, arrow[0], p.getScaledPos()[2]+adjustment,
p.getScaledPos()+adjustment+[2]+2*10-11*distScale*arrow[1][2]/(maths.log(p.getMass())))
7      ENDFOR
8  ENDIF
9  ENDPROCEDURE

```

Comparison mode

This is the last part of the display. The purpose of the comparison mode is to give users a good way to understand the size difference between planets and other objects. The way this will be achieved is by displaying the two objects being compared next to each other, with the same `distScale`. This means their relative sizes will be the same as they are in real life. The screen will be split vertically down the middle into two rectangles. The first planet will be displayed at the centre of the left rectangle, and the other at the center of the right rectangle. These rectangles will be represented by pygame surfaces. This is done so that the planet cannot “escape” its side of the screen, if both planets were drawn onto one surface then they could overlap and this would look silly.

There is an array, `planetsToCompare`, which holds the two planets that need to be compared. How this array is manipulated will be covered in the section on inputs. For the sake of brevity I will refer to these planets as `p1` and `p2`, in code they will be referenced as `planetsToCompare[0]` and `planetsToCompare[1]`.

First the comparison surfaces are filled with black to create the background, the same way the screen is filled with black when displaying the solar system. Next the planets are drawn at `centre[x]/2, centre[y]` to each surface, which puts them at the centre of each rectangle. `p1` is drawn to `comparisonSurface1` and `p2` is drawn to `comparisonSurface2`. This is done using

`drawPlanet()`, with the position being passed in as `(centre[x]/2, centre[y])`. `drawPlanet()` will automatically do the scaling for us and draw the planets at the correct scale.

Next we need to use `pygame` to blit the surfaces to the screen. The first surface is at (0,0) and the second surface is drawn at `(centre[x]+3, 0)`, leaving room for a 6 pixel wide barrier to distinguish the two surfaces.

A 6 pixel wide blue barrier is drawn using `pygame.draw.line()`, its `startPos` is `(centre[x],0)` and `endPos` is `(centre[x],centre[y])`, which means it goes straight down the middle of the screen. This makes the distinction between the two sides clear.

Finally, information on both planets is displayed using `displayPlanetInfo()`, which will be explained below in the interface section. This works the same way as displaying the planet information normally, it's just that info is now shown for two planets instead of one.

Here is pseudocode showing how all this works:

```

1  pygame.draw.rect(comparisonSurface1, 'black', (0, 0, centre[x]*2, centre[y]*2))
2  pygame.draw.rect(comparisonSurface2, 'black', (0, 0, centre[x]*2, centre[y]*2))
3  drawPlanet(planets[planetsToCompare[0]], np.array([centre[x]/2, centre[y]]), comparisonSurface1)
4  drawPlanet(planets[planetsToCompare[1]], np.array([centre[x]/2, centre[y]]), comparisonSurface2)
5  screen.blit(comparisonSurface1, (0, 0))
6  screen.blit(comparisonSurface2, (centre[x]+3, 0))
7  pygame.draw.line(screen, "blue", [centre[x],0], [centre[x],centre[y]*2], 6)
8  // Display info on both selected planets
9  displayPlanetInfo(planets[planetsToCompare[0]],planets, 20)
10 displayPlanetInfo(planets[planetsToCompare[1]], planets, centre[x]*2-280)

```

The 20 and `centre[x]*2-280` being used as arguments to `displayPlanetInfo()` tell it where to display the info. 20 means 20 pixels from the left edge of the screen and `centre[x]*2-280` means 280 pixels from the right edge of the screen (so very close to the right edge.) This will make more sense when `displayPlanetInfo()` is explained in the interface section below.

Interface

I will split the interface into two parts: The menu, which shows the user information about the planets as requested by the client and also tells them some of the keybinds, and the input system, which lets the user actually interact with the program.

Menu

A mockup for the menu was already shown in the analysis. There are two sides to it, on the left will be information about energies and G, and on the right will be information about the planet selected. Each section displaying information will be on a transparent black background, `menuSurface1` and `menuSurface2` for displaying energies and displaying planet info respectively. The opacity of the surface will be lowered to make the interface look more natural, i.e. it won't completely obscure things behind it. Text will then be rendered in front of the surface using `pygame's font.render()` method. This will meet the client's needs as it shows the info they've requested and also lets users edit planet masses as requested. Here is what the menu might look like:

```

Press 'A' to edit  $G$  & planet mass
GPE:  $-3.81e+35$  J
KE:  $1.89e+35$  J
Total energy:  $-1.92e+35$  J
Total momentum:  $5.59e+27$  Ns
 $G$ :  $6.67e-11$  Nm2/kg2

```

```

Object: Earth
Mass:  $5.97e+24$  kg
Radius:  $6.37e+3$  km
Distance from sun:  $1.5e+8$  km

```

I will make two functions for the menu, `displayAttributeInfo()` and `displayPlanetInfo()`. Putting them into functions makes the code neater and allows them to be reused, `displayPlanetInfo()` is used differently in the comparison mode from how it's used in the normal solar system display mode. The horizontal offset of the text will given as an argument to the functions so that the place where the text is displayed can be chosen.

`displayAttributeInfo()` will need to display different text based on whether the user is currently changing attributes or not, as explained in the analysis. If they are changing attributes then the function should display different text explaining the keybinds. The function will be given `changingAttributes`, a boolean telling whether or not the user is changing attributes, as an argument, so that it can use an if statement to determine what to do. Here is what the menu would look like when editing attributes:

```

Press 'A' to stop editing attributes
Press 'W' to increase value of  $G$ 
Press 'S' to decrease value of  $G$ 
Press 'E' to increase planet's mass
Press 'D' to decrease planet's mass
 $G$ :  $6.67e-11$  Nm2/kg2

```

```

Object: Earth
Mass:  $5.97e+24$  kg
Radius:  $6.37e+3$  km
Distance from sun:  $1.5e+8$  km

```

The functions themselves are quite simple, each line of text is rendered and then blitted onto the screen, that's it.

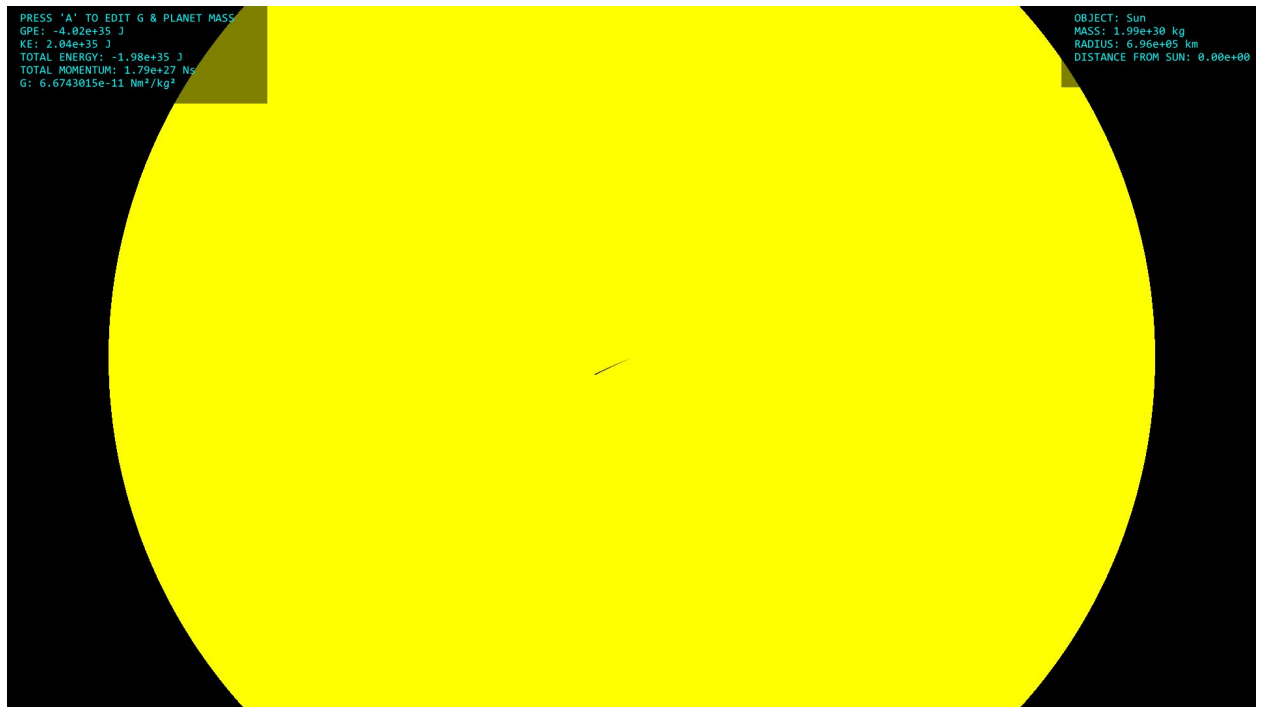
Here is pseudocode showing how `displayAttributeInfo()` works, `displayPlanetInfo()` will work in a very similar way, just with different text and without the if statement:

```

1  ✓  PROCEDURE displayAttributeInfo(changingAttributes, properties, horizontalPos, surface)
2      pygame.draw.rect(surface, 'black', (0, 0, 400, 150))
3      if changingAttributes: // If the user is editing attributes then display keybinds telling them
        what to do
4          text1 = "PRESS 'A' TO STOP EDITING ATTRIBUTES"
5          text2 = "PRESS 'W' TO INCREASE VALUE OF G"
6          text3 = "PRESS 'S' TO DECREASE VALUE OF G"
7          text4 = "PRESS 'E' TO INCREASE PLANET'S MASS"
8          text5 = "PRESS 'D' TO DECREASE PLANET'S MASS"
9          text6 = "G: " + f'{G} Nm2/kg2'
10     else:
11         text1 = "PRESS 'A' TO EDIT G & PLANET MASS"
12         text2 = "GPE: " + f'{properties[0]:.2e} J'
13         text3 = "KE: " + f'{properties[1]:.2e} J'
14         text4 = "TOTAL ENERGY: " + f'{properties[0]+properties[1]:.2e} J'
15         text5 = "TOTAL MOMENTUM: " + f'{vec.mag(properties[2]):.2e} Ns'
16         text6 = "G: " + f'{G} Nm2/kg2'
17     textSurface1 = font.render(text1, True, (0, 255, 255)) // Rendering all the text
18     textSurface2 = font.render(text2, True, (0, 255, 255))
19     textSurface3 = font.render(text3, True, (0, 255, 255))
20     textSurface4 = font.render(text4, True, (0, 255, 255))
21     textSurface5 = font.render(text5, True, (0, 255, 255))
22     textSurface6 = font.render(text6, True, (0, 255, 255))
23     screen.blit(surface, (horizontalPos-20, 0)) // Blitting the transparent surface and the text
        to the screen so that it's visible
24     screen.blit(textSurface1, (horizontalPos, 10))
25     screen.blit(textSurface2, (horizontalPos, 30))
26     screen.blit(textSurface3, (horizontalPos, 50))
27     screen.blit(textSurface4, (horizontalPos, 70))
28     screen.blit(textSurface5, (horizontalPos, 90))
29     screen.blit(textSurface6, (horizontalPos, 110))

```

And here is the result, showing the translucent property of the menu surface:



As you can see the sun is still visible behind the menu surface, but the translucent black background makes the text easier to read.

Inputs

Checking for inputs is done with `pygame.event.get()` and a bunch of if statements for every relevant input. I will just cover what each input does and how it works.

left / right arrows

If comparison is true then we're in comparison mode, so the left arrow key increments `planetsToCompare[0]` by 1 modulo `len(planets)`, and the right arrow key increments `planetsToCompare[1]` by 1 modulo `len(planets)`, having the effect of changing which planets are displayed. The reason we do it mod `len(planets)` is so that once the end of the list is reached it wraps round back to the start instead of causing an index out of range error.

If comparison is false then the left arrow decrements focus by 1 modulo `len(planets)` and the right arrow increments focus by 1 modulo `len(planets)`. This leads to the camera being centred on a different planet, as explained in the display section.

up / down arrows

The up arrow key increments `timeScale` by `0.1*YEAR` and the down arrow key decrements `timeScale` by `0.1*YEAR`. An if statement is used to check if decrementing the `timeScale` would make it negative, in which case it is set to be 0. If the `timeScale` is 0 and the user presses the up arrow key then it is set to be `0.05*YEAR` as that's the default value for `timeScale`.

space

Very simple, just toggles the value of `comFocus`.

F

Also very simple, just toggles the value of `arrows`, which is whether or not `displayArrows()` should be run.

A

Toggles the value of changingAttributes.

W, S, E and D

W and S increment and decrement the value of G, and E and D increment and decrement the mass of the currently selected planet. Because the masses of the planets take such a huge range of values across many orders of magnitude, and likewise for the values G needs to be able to take to have any visible effect on the system, a more involved approach is needed than just increasing and decreasing them by a fixed value.

This is the formula I'll be using to change the values:

$$x = x \pm 0.5 * 10^{\text{maths.floor}((\text{maths.log}(x, 10)))}$$

Where `maths.floor(a)` returns the greatest integer $\leq a$.

This has the effect of making the increments x changes in based on the order of magnitude of x . Let's say $x = 5100$, which is $5.1 * 10^3$. Then $\log(x, 10) = 3.7075702\dots$, so $\text{floor}(\log(x, 10)) = 3$. This means x would change in increments of $0.5 * 10^3$, which is 500, so x would go 5100, 5600, 6100, etc up until 9600, where it changes to 10,100. Now its increments would change because $\text{floor}(\log(x, 10)) = 4$, so now it goes up in increments of 5000. This works going the other way too.

This lets users quickly increase the mass of planets to be a different order of magnitude and provides a much smoother experience when editing values. It also has the side effect of preventing masses and G from becoming negative as the increments get smaller and smaller so you can never get past 0. This is good because negative G or mass would cause a lot of issues as it is something that does not ever happen in the real world.

Mousewheel up and down

Scrolling down will zoom out (decrease `zoomScale`) and scrolling up will zoom in (increase `zoomScale`). Just like with G and mass, the distance scale needs to be able to take a very wide range of orders of magnitudes, from being able to zoom in on and show the Earth-Moon system to being small enough to show the voyager satellites and the Sun in the same view. A different approach will be used to the one used with G and mass though, due to the way zooming is handled in code to make it look smooth and natural

Essentially, there are two scales, `zoomScale` and `distScale`. `distScale` is what's actually used when displaying planets. `zoomScale` is a target that `distScale` asymptotically approaches over time via interpolation, and is the value that gets changed by the user when scrolling. How this interpolation is done will be explained in more detail in the next section covering miscellaneous functions, including `zoom()`. This is done because it means that when zooming in or out it initially changes very quickly (but smoothly) and then slows down over time, which looks natural to humans. `zoomScale` will be changed based on percentage, zooming in increases it by 40% with each scroll, and zooming out decreases it by 40% with each scroll. This lets it change quickly and in a way that doesn't look jarring to people. So, scrolling up increases `zoomScale` by 40% and scrolling down decreases `zoomScale` by 40%

There is one other thing to take into account though. If someone has zoomed out so quickly that `distScale` is taking a while to catch up to `zoomScale` and then tries to zoom in, `zoomScale` will change but it will still be smaller than `distScale`, so the program will still be zooming out. So, if the user scrolls up to zoom in and `zoomScale` is smaller than `distScale`, then before increasing

zoomScale by 40% linear interpolation is done to bring zoomScale much closer to distScale so that the program actually starts zooming in. This works the same way for zooming out if zoomScale is larger than distScale. Pseudocode showing how this works is below:

```

1  IF event.type == pygame.MOUSEWHEEL THEN // Check if the user has scrolled
2      IF event.y == 1 THEN // If the user has scrolled up
3          IF zoomScale < distScale THEN
4              zoomScale ← 0.3*zoomScale + 0.7*distScale // Linear interpolation to bring zoomScale
              much closer to distScale if it's less than it
5          ENDIF
6          zoomScale ← 1.4*zoomScale // Increasing zoomScale by 40%
7      ENDIF
8      IF event.y == -1 THEN // If the user has scrolled down
9          IF zoomScale > distScale THEN
10             zoomScale ← 0.3*zoomScale + 0.7*distScale
11         ENDIF
12         zoomScale ← 0.6*zoomScale // Decreasing zoomScale by 40%
13     ENDIF
14 ENDIF

```

Miscellaneous functions

This section is just to cover the few functions that haven't fit neatly into the other sections. These are `com()`, which finds the centre of mass of planets, `sumPhysicalProperties()` which adds up the energy and momentum of planets, and `zoom()` which handles the interpolation for the `distScale`.

com

`com` stands for centre of mass, it's a real acronym used by physicists. We need the centre of mass so that the user can focus on it, particularly useful when the mass of planets has been increased by a large amount. To find the centre of mass is quite simple, this is the formula for it:

$$\frac{\sum m_i x_i}{\sum m_i}$$

Where m_i is the mass of the i th planet and x_i is its position vector. This can be done very simply with a for loop in code, just use `p.getMass()` to get the mass of the planet and `p.getPos()` to get the position of the planet. Here is pseudocode showing how it works:

```

1  FUNCTION com(planets)
2      positionMassSum ← np.array([0.0, 0.0, 0.0]) // Use numpy arrays for vector addition
3      massSum ← 0
4      FOR p IN planets
5          positionMassSum ← positionMassSum + p.getMass()*p.getPos() // Sum m*x
6          massSum ← massSum + p.getMass() // Sum m
7      ENDFOR
8      RETURN positionMassSum/massSum // Return the centre of mass, sum(m*x)/sum(m)
9  ENDFUNCTION

```

sumPhysicalProperties

The client has said that displaying energy and momentum to show conservation laws would be helpful, so this will need to be done. Luckily, planets already have their energies and momentum as attributes, so all this needs is a simple for loop to add them up, shown here:

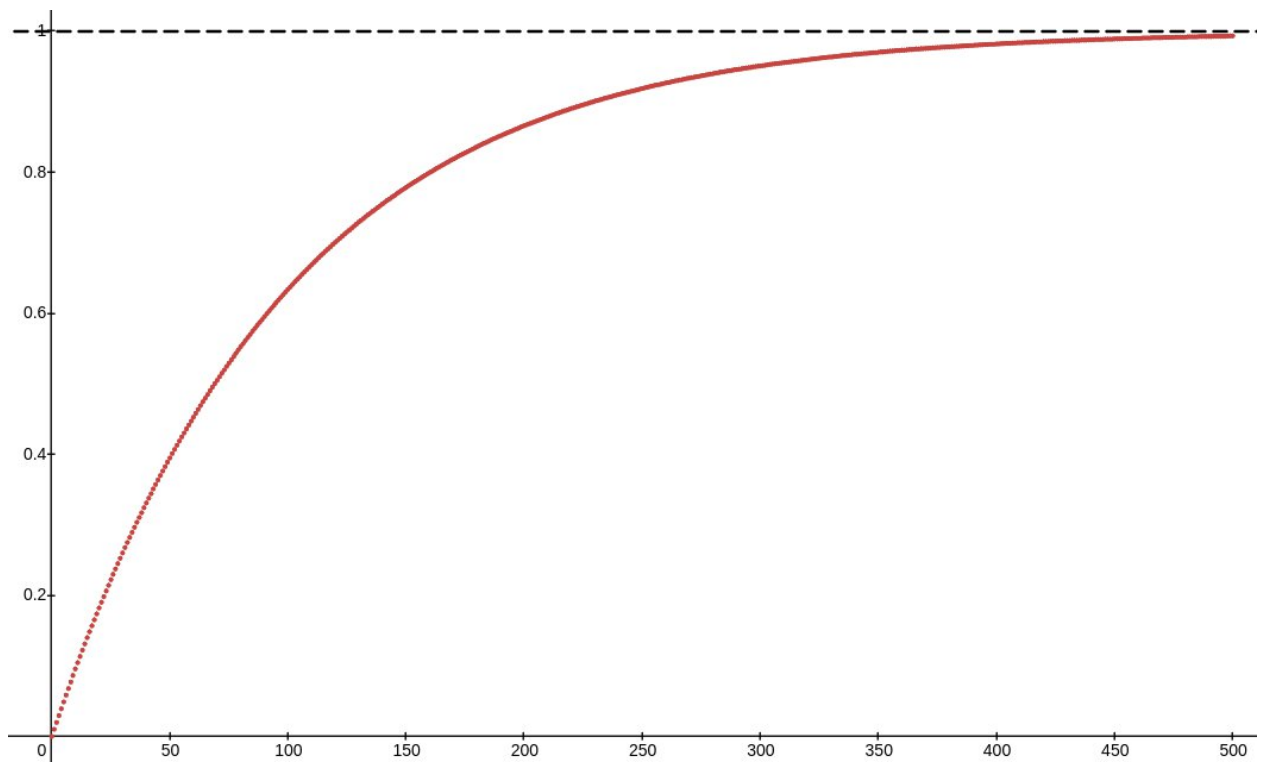
```

1  FUNCTION sumPhysicalProperties(planets)
2      gpe ← 0
3      ke ← 0
4      momentum ← 0
5      FOR p IN planets
6          gpe ← GPE + p.getGPE()
7          ke ← KE + p.getKE()
8          momentum ← momentum + p.getMomentum()
9      ENDFOR
10     RETURN [gpe, ke, momentum]
11 ENDFUNCTION

```

zoom

The problem with zooming is making it look natural. Clearly just changing the distScale instantly when you scroll would be extremely jarring, as the program would just snap to being more zoomed out or zoomed in. So, an extra variable is created, zoomScale, which is the intended new value for distScale. One approach would be to do linear interpolation between distScale and zoomScale so that you smoothly zoom out at a constant rate, but this would still have an abrupt stop once distScale reached zoomScale. Instead, a non-linear interpolation is used. This makes it so that distScale asymptotically approaches zoomScale, there is no abrupt stop. A graph of how this works is shown here, with distScale starting at 0 and then going up to 1:



The x-axis is time and y-axis is the value of distScale. As you can see distScale grows quickly at the start and then the growth slows down, and it approaches 1 but never quite reaches it. The dotted black line is the asymptote at $y=1$. This ends up creating a natural and smooth animation for zooming out that stops gradually instead of abruptly.

So, how is this interpolation done? Lets say distScale is at some value x_0 at tick $t = 0$ and we have a target zoomScale of Z . Then we can use this recursion relation to calculate the value of distScale at the next tick:

$$x_{t+1} = ax_t + (1 - a)Z$$

Where a is a constant that determines how fast distScale approaches Z . a must be between 0 and 1, if it is less than 0 then distScale still approaches Z but it oscillates around it, sometimes being negative sometimes positive, which looks ridiculous, and if a is greater than 1 then distScale diverges away from Z instead of approaching it. Through experimentation I have found that $a = 0.9$ is a suitable value for the program.

One thing to note is that if distScale grows very large it can cause lag due to pygame attempting to draw circles with very large radii, something it's not good at. There is no point for distScale to be particularly big as the users would not be able to see anything, the planet they'd selected would fill the whole screen, so distScale is capped at 1000.

Zoom() will be run every tick to carry out this interpolation process, and it makes distScale change smoothly over time. Here is pseudocode for it:

```

1 FUNCTION zoom(distScale, zoomScale)
2     IF distScale*0.9+zoomScale*0.1 > 1000 THEN // Capping distScale at 1000 to prevent lag
3         RETURN 1000
4     ENDIF
5     RETURN distScale*0.9 + zoomScale*0.1 // Perform non-linear interpolation to smoothly change
    the value of distScale
6 ENDFUNCTION

```

Technical Solution

horizonsParser

This contains all code inside the horizonsParser.py module

extractValue

All the code for the extractValue function is below:

```

1  ## Program title: Solar System Simulation by Archie Pennycook
2
3  import requests
4  import numpy as np
5  import datetime
6
7  ## Sometimes NASA puts a scale next to their units (e.g. they might say mass is measured as 10^23 kg)
8  ## If the scale is expected, it is found and the value is multiplied by it.
9  ## The conversionFactor is because the program works in SI, but NASA sometimes uses kilometres and other non-SI units
10 ## So it converts the unit back to SI
11 def extractValue(text, expectingScale=False, conversionFactor=1):
12
13     ## scaleAt records the index wher the scale is
14     ## So that the second for loop can skip to the position after the scale
15     scaleAt = 0
16     scale = 1
17     ## If there is a scale given (e.g. mass is in 10^24 kg) then it needs to be found
18     ## The scale will always include a '^' character, and this does not appear anywhere else in the ephemeris
19     ## So this is what is used to search for the scale
20     if expectingScale:
21         for i in range(50):
22             if text[i] == '^':
23                 for j in range(1,25):
24                     if not text[i+j].isdigit():
25                         scale = eval(text[i-2:i+j].replace('^','**'))
26                         break
27                 scaleAt = i+j
28                 break

```

```

29     ## Once a digit is found it loops until a non-digit character is found. This string is taken to be the value.
30     for i in range(scaleAt+1,40):
31         ## The text[i] == '-' is so negatives are included
32         if text[i].isdigit() or text[i] == '-':
33             for j in range(1,50):
34                 ## Preventing index out of range errors
35                 if i+j >= 50:
36                     break
37                 ## The E, + and - being allowed is for scientific notation, because velocity and position,
38                 ## unlike mass and radius, are presented in scientific notation.
39                 if not text[i+j].isdigit() and text[i+j] not in ['-','.', 'E', '+']:
40                     ## This loop removes error margins, not interested, just need a value
41                     for k in range(len(text[i:i+j])):
42                         if text[i+k:i+k+2] == '+-':
43                             j = k
44                             break
45                     value = float(text[i:i+j])
46                     break
47             break
48
49     return scale*value*conversionFactor
50

```

getEphemeris

All the code for getEphemeris is shown below:

```

51 def getEphemeris(target, useGM=False, ignoreMassSize=False, mass=0, size=0):
52     target = str(target)
53     startTime = datetime.datetime.now().strftime("%Y-%m-%d")
54     stopTime = (datetime.datetime.now() + datetime.timedelta(days=1)).strftime("%Y-%m-%d")
55     ## These request segments only exist to breakup the request so that it is visible in screenshots
56     ## As having it all in one line would lead to a very wide screenshot
57     ## Which would not be easy to read for the examiner
58     requestSegment1 = "https://ssd.jpl.nasa.gov/api/horizons.api?format=text&COMMAND="
59     requestSegment2 = "'&OBJ_DATA='YES'&MAKE_EPHEM='YES'&EPHEM_TYPE='VECTORS'&START_TIME="
60     requestSegment3 = "'&STOP_TIME="
61     requestSegment4 = "'&CENTER='500@0'&STEP_SIZE='1%20d'&QUANTITIES="
62     response = requests.get(requestSegment1+target+requestSegment2+startTime+requestSegment3+stopTime+requestSegment4)
63
64     position = np.array([0.0, 0.0, 0.0])
65     velocity = np.array([0.0, 0.0, 0.0])
66     ## The dictionaries are used to ensure we update the correct element of the vector
67     posDict = {
68         'X':0,
69         'Y':1,
70         'Z':2,
71     }
72     velDict = {
73         'VX':0,
74         'VY':1,
75         'VZ':2
76     }
77
78     for i in range(len(response.text)):
79         ## Sometimes it's convenient to manually set the mass and size, as a lot of the time they're not given by the ephemeris
80         ## so that's why ignoreMassSize is a thing
81         if not ignoreMassSize:
82             if not useGM:
83
84                 if response.text[i:i+6].lower() == 'mass x' or response.text[i:i+7].lower() in ['mass, 1', 'mass, x']:
85                     mass = extractValue(response.text[i:i+50],True) # Mass is in kg, so no multiplying by a factor of 10 is needed
86

```

```

87     ## For some bodies only G*mass, rather than the mass, is given, due to quirks in how we calculate the mass of big things
88     ## in this case I have to work out the mass myself from the GM value given.
89     ## NASA is not standard AT ALL about how they label the GM value in their ephemeris so I use a list with all the possibilities
90     ## This is needed for Jupiter's moons
91     else:
92         if response.text[i:i+6].lower() in ['gm, km', 'gm ', 'gm (km)':
93             mass = extractValue(response.text[i+20:i+50], False, 1/(6.6743015*10**-20))
94         if response.text[i:i+16].lower() in ['vol. mean radius', 'mean radius (km)':
95             size = extractValue(response.text[i:i+50], False, 1000)
96
97     if response.text[i:i+3] in posDict:
98         position[posDict[response.text[i:i+3]]] = extractValue(response.text[i:i+40], False, 1000)
99
100    if response.text[i:i+3] in velDict:
101        velocity[velDict[response.text[i:i+3]]] = extractValue(response.text[i:i+40], False, 1000)
102        ## NASA provides a list of coordinates of where the planet will be over the
103        ## next few days. This is not necessary for our purposes as we just need initial
104        ## conditions and then the rest will be simulated from there, so after
105        ## the final element of velocity is found, the loop breaks.
106        if velocity[2] != 0:
107            break
108
109    return [size, mass, position, velocity]
110

```

main

main is only used for testing, here is the code for it:

```

111  ## This is used for testing purposes, for test 01
112  def main():
113      ephemeris = getEphemeris(399)
114      print("RADIUS:", ephemeris[0], "m")
115      print("MASS:", ephemeris[1], "kg")
116      print("POSITION:", ephemeris[2])
117      print("VELOCITY:", ephemeris[3])
118
119  if __name__ == "__main__":
120      main()

```

vectors

Here is all the code for the vectors module:

```

1  ## Program title: Solar System Simulation by Archie Pennycook
2
3  import math as maths
4  import numpy as np
5  x = 0
6  y = 1
7

```

```

8  ## Finds the dot product of two vectors
9  def dot(vec1, vec2):
10     if len(vec1) != len(vec2):
11         print("Dimension error!")
12         return
13     return sum([vec1[i]*vec2[i] for i in range(len(vec1))])
14
15  ## Finds the magnitude of a vector
16  def mag(vec):
17     mag = sum([i**2 for i in vec])
18     return maths.sqrt(mag)
19
20  ## Returns a vector of length 1 in the same direction as the input
21  def unit(vec):
22     if mag(vec) == 0:
23         return vec
24     return vec*(1/mag(vec))
25
26  ## Returns a vector that is perpendicular to input line
27  ## This vector must have a length of 1 so it can be easily scaled
28  ## Done by taking the "negative reciprocal", swapping the values and multiplying one of them by -1
29  def normal(vec):
30     normal = np.array([-1*vec[y],vec[x]])
31     return unit(normal)
32
33  ## Main is just used for testing purposes, for test 02
34  def main():
35     print("DOT PRODUCT BETWEEN (-1,-2), (3,4):", dot(np.array([-1, -2]), np.array([3, 4])))
36     print('----')
37     print("MAGNITUDE OF (3,4):", mag(np.array([3, 4])))
38     print("UNIT OF (3,4):", unit(np.array([3,4])))
39     print("NORMAL TO (3, 4):", normal(np.array([3, 4])))
40     print("DOT BETWEEN NORMAL TO (3,4) AND (3,4) SHOULD BE ZERO:", dot(np.array([3, 4]), normal(np.array([3, 4]))))
41     print('----')
42     print("MAGNITUDE OF (-1, -2):", mag(np.array([-1,- 2])))
43     print("UNIT OF (-1, -2):", unit(np.array([-1,- 2])))
44     print("NORMAL TO (-1, -2):", normal(np.array([-1,- 2])))
45     print("DOT BETWEEN NORMAL TO (-1, -2) AND (-1, -2) SHOULD BE ZERO:", dot(np.array([-1, -2]), normal(np.array([-1,- 2]))))
46
47  if __name__ == "__main__":
48     main()

```

planets

This is the code for planets, the main part of the program which does all the actual simulation.

Setup

These are all of the constants defined at the start that are used throughout, as well as the modules imported:

```

1  ## Program title: Solar System Simulation by Archie Pennycook
2
3  import math as maths
4  import pygame
5  import numpy as np
6  import vectors as vec
7  import horizonsParser
8
9  ### These are all either constants or are variables that are only ever changed by main(),
10 ### and so it's okay to define them here and use them globally.
11 ## The distance constant is used to translate SI units (metres) into pixels.
12 ## 4 * 10**-9 means that the earth is about 600 pixels from the sun, for reference.
13 distScale = 4 * 10**-9
14 zoomScale = distScale
15 x = 0
16 y = 1
17 ## Gravitational constant, defines how strong gravity is. Real life G = 6.6743015*10**-11
18 G = 6.6743015*10**-11
19 ## How long to draw the lines representing the planets' orbits.
20 MAX_LINES = 3000
21

```

Classes

Here the three classes and all their methods, which are critical to the simulation are defined:

```

22 class celestialBody:
23     def __init__(self, name, size, mass, pos, vel, colour):
24         self.__host = None
25         self.__name = name
26         self.__size = size
27         self.__vel = vel
28         self.__mass = mass
29         self.__pos = pos
30         self.__colour = colour
31         self.__accel = 0
32         self.__resultant = 0
33         ## KE and GPE are standard acronyms for kinetic energy and gravitational potential energy
34         self.__ke = 0
35         self.__gpe = 0
36         self.__momentum = 0
37         self.__records = []
38         self.__arrows = []
39

```

```
40     ## All the getters and setters
41     def getHost(self):
42         return self.__host
43
44     def getName(self):
45         return self.__name
46
47     def getPos(self):
48         return self.__pos
49
50     def getScaledPos(self):
51         return scaledPos(self.getPos())
52
53     def getMass(self):
54         return self.__mass
55
56     def getVel(self):
57         return self.__vel
58
59     def getAccel(self):
60         return self.__accel
61
62     def getResultant(self):
63         return self.__resultant
64
```

```
65     def getColour(self):
66         |         return self.__colour
67
68     def getSize(self):
69         |         return self.__size
70
71     def getGPE(self):
72         |         return self.__gpe
73
74     def getKE(self):
75         |         return self.__ke
76
77
78     def getMomentum(self):
79         |         return self.__momentum
80
81     def getRecords(self):
82         |         return self.__records
83
84     def getArrows(self):
85         |         return self.__arrows
```

```

86 | def addRecord(self, record):
87 |     self.__records.append(record)
88 |     ## Gets rid of excess lines, prevents them from becoming too long and lagging the system
89 |     if len(self.__records) > LINE_LENGTH:
90 |         self.__records = self.__records[len(self.__records)-LINE_LENGTH:]
91 |
92 |
93 | def addArrow(self, arrow):
94 |     self.__arrows.append(arrow)
95 |     ## There should never be more than the amount of planets + 1 arrows at a time,
96 |     ## if there are, then some have been left over from previous ticks, and should be cleaned up
97 |     if len(self.__arrows) > len(planets):
98 |         self.__arrows = self.__arrows[len(self.__arrows)-len(planets):]
99 |
100 |     ## Adds a force to the resultant force on the planet
101 |     def addForce(self, force):
102 |         self.__resultant += force
103 |
104 |     def addGPE(self, energy):
105 |         self.__gpe += energy
106 |
107 |     def addKE(self, energy):
108 |         self.__ke += energy
109 |
110 |     def addMomentum(self, momentum):
111 |         self.__momentum += momentum
112 |
113 |     def addMass(self, mass):
114 |         self.__mass += mass
115 |
116 |     ## Sets velocity
117 |     ## Uses F = ma to find acceleration, add to vel
118 |     def secondLaw(self):
119 |         self.__accel = self.getResultant()/self.getMass()
120 |
121 |     ## Sets position
122 |     ## Updates velocity and then moves a planet by its velocity
123 |     ## This is verlet integration
124 |     def verlet(self, timeScale):
125 |         ## Complex scientific modelling
126 |         self.__vel += self.getAccel()*timeScale
127 |         self.__pos += self.getVel()*timeScale
128 |         ## Calculate KE and momentum while we're updating velocity so that we don't have to do it later
129 |         self.addKE(0.5*self.getMass()*vec.mag(self.getVel())**2)
130 |         self.addMomentum(self.getMass()*self.getVel())

```

```

131     ## End of getters and setters
132
133     ## Uses  $F = Gm/r^2$  to work out the force on a planet
134     ## Multiplies by unit(r) to make the force a vector.
135     def gravity(self, planet2):
136         ## Complex scientific modelling
137         r = planet2.getPos() - self.getPos()
138         F = G*(self.getMass()*planet2.getMass()/(vec.mag(r)**2))
139         ## Calculate GPE while working out gravitational force so we don't have to do it later
140         self.addGPE(-F*vec.mag(r))
141         planet2.addGPE(F*vec.mag(r))
142         return F*vec.unit(r)
143
144     ## The distinction between planet and satellite here is just whether or not
145     ## they have a specific host. This has no relation to actual planets in real life
146     ## The sun is a planet in my code.
147     class planet(celestialBody):
148         pass
149
150     ## Satellites (natural like the moon or manmade) show their orbits around their host planet,
151     ## rather than showing their actual path through space like other celestial bodies do.
152     ## This is more useful as it is not easy to see how the satellite orbits its planet otherwise
153     ## The "host" attribute is a planet object, aggregation is used to access the host's attributes
154     class satellite(celestialBody):
155         def __init__(self, name, size, mass, pos, vel, colour, host):
156             super().__init__(name, size, mass, pos, vel, colour)
157             self.__host = host
158             self.__resultant = 0
159             self.__records = []
160
161         def getHost(self):
162             return self.__host
163
164         ## Adds the host's position to the line so that it can be displayed, then returns that
165         def getRecords(self):
166             hostPos = self.getHost().getPos()
167             updatedRecords = [i+hostPos for i in self.__records]
168             return updatedRecords
169
170         ## hostLine is the line drawn for the host on the current tick.
171         ## Subtracting this from the line for our satellite "removes" the movement of the host.
172         ## This leaves only the movement of the satellite around the host.
173         def addRecord(self, record):
174             hostPos = self.getHost().getPos()
175             record = record - hostPos
176             self.__records.append(record)
177             if len(self.__records) > LINE_LENGTH:
178                 self.__records = self.__records[len(self.__records)-LINE_LENGTH:]
179

```

Functions

These are the majority of the functions used in my code, except for the two big ones used to setup and run the program (generateSolarSystem() and main().)

```

180  ## Switching from one zoom to another instantly is very jarring.
181  ## This uses interpolation to smoothly transition between zooms.
182  ## I could use linear interpolation but I actually think this non-linear thing looks nicer so I'm using that.
183  def zoom(distScale, zoomScale):
184      ## No point for the scale to go anywhere near 1000,
185      ## so this is in place to prevent any bugs that may occur from zooming in very far
186      if distScale*0.9+zoomScale*0.1 > 1000:
187          return 1000
188      return distScale*0.9 + zoomScale*0.1
189
190  ## Takes in a position vector and outputs that vector from the centre of mass scaled by the distance constant.
191  ## This way if a planet is 150 million km from the sun, it can be displayed as x amount of pixels from the sun.
192  def scaledPos(position):
193      global distScale
194      return distScale * position
195
196  ## Draws an arrow by drawing a line, picking two points either side of that line,
197  ## and drawing lines from the end of the first line to those two points
198  def drawArrow(surface, colour, startPos, endPos):
199      ## My solar system is in 3d space, but pygame can only handle 2d lines.
200      ## The [:2] is necessary to deal with that.
201      startPos, endPos = startPos[:2], endPos[:2]
202      mainVector = (endPos - startPos)
203      length = vec.mag(mainVector)
204      pygame.draw.aaline(surface, colour, startPos, endPos)
205      ## Generating the two points either side of the line
206      normal = vec.normal(mainVector)
207      point1 = startPos + 0.9*mainVector + 0.05*length*normal
208      point2 = startPos + 0.9*mainVector - 0.05*length*normal
209      pygame.draw.aaline(surface, colour, point1, endPos)
210      pygame.draw.aaline(surface, colour, point2, endPos)
211
212  def displayArrows(planets, adjustment, focus, comFocus, surface):
213      global distScale
214      ## If no planet is being focused on then there's no arrows to draw
215      if not comFocus:
216          p = planets[focus]
217          for arrow in p.getArrows():
218              drawArrow(surface, arrow[0], p.getScaledPos()[:2]+adjustment, p.getScaledPos()[:2]+2*10**-11*distScale*arrow[1][:2]/(maths.log(p.getMass()))+adjustment)
219
220  def displayLines(planets, adjustment, focus, comFocus, surface):
221      for p in planets:
222          index = -1
223          orbitalPath = [i[:2]+adjustment for i in scaledPos(np.array(p.getRecords()))]
224          # lines = [[orbitalPath[i], orbitalPath[i+1]] for i in range(len(orbitalPath)-1)]
225          for i in range(len(orbitalPath)-1):
226              line = [orbitalPath[i], orbitalPath[i+1]]
227              index += 1
228              ## Don't draw lines offscreen to avoid lag
229              if offscreen(line[0]) and offscreen(line[1]):
230                  continue
231              ## A satellite's lines should not be drawn if its host or itself is not selected
232              ## It is unlikely to be visible and will cause lag if drawn
233              if p.getHost() != None and (p.getHost() != planets[focus] and p != planets[focus] or comFocus):
234                  continue
235              ## Index ratio is used to reduce opacity and thickness of the older lines
236              ## An index counter is used as python cannot find the index of np arrays with multiple elements
237              indexRatio = index/len(orbitalPath)
238              pygame.draw.aaline(surface, ([int(indexRatio*colour) for colour in p.getColour()]), line[0], line[1], True)
239

```

```

240 def drawPlanet(p, position, surface):
241     global distScale
242     scaledSize = p.getSize()*distScale
243     ## If the planet would fill the whole screen, there's no point trying to draw
244     ## more of the circle than necessary, just fill the screen.
245     ## This avoids severe lag when zooming in very closely.
246     if scaledSize > vec.mag(centre):
247         surface.fill(p.getColour())
248     else:
249         pygame.draw.circle(surface,p.getColour(),position,scaledSize)
250
251 ## NASA's data on the solar system is all given in 3 dimensional coordinates
252 ## So a simulation of 3d space is used for this program. It also leads to a more accurate model.
253 ## However, displaying 3d graphics is computationally intensive and doesn't contribute to understanding
254 ## of the solar system, so a top down 2d view is displayed. This is why only the first two coordinates of the planet
255 ## are used for drawing it.
256 def displayPlanets(planets, adjustment, surface):
257     for p in planets:
258         if offscreen(p.getScaledPos()[:2]+adjustment):
259             continue
260         drawPlanet(p, p.getScaledPos()[:2]+adjustment, surface)
261
262 def displayAttributeInfo(font, changingAttributes, properties, horizontalPos, surface):
263     ## This is absolutely horrible code but it was the best way I could figure out of doing it
264
265     ## The reason I have multiple textSurfaces is because pygame does not seem to support the "\n" character,
266     ## so this is the only way I could get new lines.
267     pygame.draw.rect(surface, 'black', (0, 0, 400, 150))
268     if changingAttributes:
269         text1 = "PRESS 'A' TO STOP EDITING ATTRIBUTES"
270         text2 = "PRESS 'W' TO INCREASE VALUE OF G"
271         text3 = "PRESS 'S' TO DECREASE VALUE OF G"
272         text4 = "PRESS 'E' TO INCREASE PLANET'S MASS"
273         text5 = "PRESS 'D' TO DECREASE PLANET'S MASS"
274         text6 = "G: " + f'{G} Nm2/kg2'
275     else:
276         text1 = "PRESS 'A' TO EDIT G & PLANET MASS"
277         text2 = "GPE: " + f'{properties[0]:.2e} J'
278         text3 = "KE: " + f'{properties[1]:.2e} J'
279         text4 = "TOTAL ENERGY: " + f'{properties[0]+properties[1]:.2e} J'
280         text5 = "TOTAL MOMENTUM: " + f'{vec.mag(properties[2]):.2e} Ns'
281         text6 = "G: " + f'{G} Nm2/kg2'
282     textSurface1 = font.render(text1, True, (0, 255, 255))
283     textSurface2 = font.render(text2, True, (0, 255, 255))
284     textSurface3 = font.render(text3, True, (0, 255, 255))
285     textSurface4 = font.render(text4, True, (0, 255, 255))
286     textSurface5 = font.render(text5, True, (0, 255, 255))
287     textSurface6 = font.render(text6, True, (0, 255, 255))
288     screen.blit(surface, (horizontalPos-20, 0))
289     screen.blit(textSurface1, (horizontalPos, 10))
290     screen.blit(textSurface2, (horizontalPos, 30))
291     screen.blit(textSurface3, (horizontalPos, 50))
292     screen.blit(textSurface4, (horizontalPos, 70))
293     screen.blit(textSurface5, (horizontalPos, 90))
294     screen.blit(textSurface6, (horizontalPos, 110))
295

```

```

296  ## This displays information on the desired planet.
297  def displayPlanetInfo(font, targetPlanet, planets, horizontalPos, surface):
298      pygame.draw.rect(surface, 'black', (0, 0, 350, 125))
299      text5 = "OBJECT: " + f'{targetPlanet.getName()}'
300      text6 = "MASS: " + f'{targetPlanet.getMass():.2e} kg'
301      text7 = "RADIUS: " + f'{targetPlanet.getSize()/1000:.2e} km'
302      text8 = "DISTANCE FROM SUN: " + f'{vec.mag(targetPlanet.getPos()-planets[0].getPos())/1000:.2e} km'
303      textSurface5 = font.render(text5, True, (0, 255, 255))
304      textSurface6 = font.render(text6, True, (0, 255, 255))
305      textSurface7 = font.render(text7, True, (0, 255, 255))
306      textSurface8 = font.render(text8, True, (0, 255, 255))
307      screen.blit(surface, (horizontalPos-20, 0))
308      screen.blit(textSurface5, (horizontalPos, 10))
309      screen.blit(textSurface6, (horizontalPos, 30))
310      screen.blit(textSurface7, (horizontalPos, 50))
311      screen.blit(textSurface8, (horizontalPos, 70))
312
313  def displayStartScreen(font):
314      text1 = "Press the left and right arrow keys to switch between planets"
315      text2 = "Press the up and down arrow keys to speed up and slow down time"
316      text3 = "Press the space key to switch the focus to the centre of mass of the solar system"
317      text4 = "Press 'F' to show the forces acting on the selected planet"
318      text5 = "Press 'C' to enter comparison mode to compare sizes of planets"
319      text6 = "Press 'A' to edit the strength of gravity (G) and the mass of planets"
320      text7 = "Press Enter to start the program!"
321      startScreenSurface1 = font.render(text1, True, (0, 255, 255))
322      startScreenSurface2 = font.render(text2, True, (0, 255, 255))
323      startScreenSurface3 = font.render(text3, True, (0, 255, 255))
324      startScreenSurface4 = font.render(text4, True, (0, 255, 255))
325      startScreenSurface5 = font.render(text5, True, (0, 255, 255))
326      startScreenSurface6 = font.render(text6, True, (0, 255, 255))
327      startScreenSurface7 = font.render(text7, True, (255, 0, 0))
328
329      ## Display the start messages at equally spaced intervals down the screen.
330      screen.blit(startScreenSurface1, (500, 1/7 * 0.9*centre[y]*2))
331      screen.blit(startScreenSurface2, (500, 2/7 * 0.9*centre[y]*2))
332      screen.blit(startScreenSurface3, (500, 3/7 * 0.9*centre[y]*2))
333      screen.blit(startScreenSurface4, (500, 4/7 * 0.9*centre[y]*2))
334      screen.blit(startScreenSurface5, (500, 5/7 * 0.9*centre[y]*2))
335      screen.blit(startScreenSurface6, (500, 6/7 * 0.9*centre[y]*2))
336      screen.blit(startScreenSurface7, (500, 7/7 * 0.9*centre[y]*2))
337
338      pygame.display.flip()
339
340  ## Finds the centre of mass of the system
341  def com(planets):
342      positionMassSum = np.array([0.0,0.0,0.0])
343      massSum = 0
344      for p in planets:
345          positionMassSum += p.getMass()*p.getPos()
346          massSum += p.getMass()
347      return positionMassSum/massSum
348

```

```

349  ## Uses vectors to find the adjustment vector from either the centre of the screen to the centre of mass,
350  ## or the centre of the screen to the currently selected planet
351  def focusAdjustment(planets, focus, comFocus):
352      if comFocus:
353          currentFocus = com(planets)
354      else:
355          currentFocus = planets[focus].getPos()
356      focusDisplacement = centre - scaledPos(currentFocus)[:2]
357      return focusDisplacement
358

```

simulateTick is where pretty much all the physics happens and is the most important part of the program

```

359  def simulateTick(planets, focus, timeScale):
360      for p1 in planets:
361          ## Reset energy of planet so it can be recalculated
362          p1.addGPE(-p1.getGPE())
363          p1.addKE(-p1.getKE())
364          p1.addMomentum(-p1.getMomentum())
365
366          ## Only need to look at the planets in the list after p1 as we make use of Newton's third law
367          ## To add force to p2 when finding the force on p1
368          ## Saves a lot of iterations
369          ## This is complex scientific modelling
370          for p2 in planets[planets.index(p1)+1:]:
371              plgravity = p1.gravity(p2)
372              p1.addForce(plgravity)
373              ## Can take away here due to Newton's third law, each force has equal and opposite reaction force
374              ## This lets us skip the current p1 when finding the force on p2 later, saving time
375              p2.addForce(-plgravity)
376              ## Draws force arrows showing the forces acting on the planet
377              if planets.index(p1) == focus:
378                  p1.addArrow(["white", np.copy(plgravity)])
379              if planets.index(p2) == focus:
380                  p2.addArrow(["white", -1*np.copy(plgravity)])
381          ## Draw the resultant arrow in the colour of the planet
382          if planets.index(p1) == focus:
383              p1.addArrow([p1.getColour(), np.copy(p1.getResultant())])
384
385          p1.secondLaw()
386          ## Uses verlet integration to update velocity and acceleration of planet
387          p1.verlet(timeScale)
388          ## Add a record of the planet's position so we can draw orbital lines.
389          p1.addRecord(np.copy(p1.getPos()))
390          ## Reset the resultant to 0 so it can be calculated again next tick
391          p1.addForce(-p1.getResultant())
392
393  ## Takes in vector, returns False if within the screen, True otherwise
394  def offscreen(vector):
395      if vector[x] < 0 or vector[x] > centre[x]*2 or vector[y] < 0 or vector[y] > centre[y]*2:
396          return True
397      return False
398
399  ## This is so that the program can demonstrate that momentum and energy are conserved
400  def sumPhysicalProperties(planets):
401      gpe = 0
402      ke = 0
403      momentum = 0
404      ## Add up all the attributes for each planet
405      for p in planets:
406          gpe += p.getGPE()
407          ke += p.getKE()
408          momentum += p.getMomentum()
409      return [gpe, ke, momentum]
410

```

generateSolarSystem

This is the function that uses `getEphemeris()` to generate all the planets.

```

411 def generateSolarSystem():
412     ## This list contains the name, NASA ID and colour of all the planets
413     planets = []
414
415     planetNames = ["Sun", "Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn", "Uranus", "Neptune", "Pluto"]
416     planetColours = [(255, 255, 0), (65, 68, 74), (139, 115, 85), (0, 0, 255), (255, 99, 47), (250, 164, 87), (195, 146, 79), (98, 174, 230), (67, 109, 252), (204, 168, 132)]
417     ## I do not want to add all 100+ moons of Jupiter, Saturn, Uranus and Neptune, hence only the important ones, our moon and the Galilean moons, are added
418     moonNames = ["Moon", "Io", "Europa", "Ganymede", "Callisto"]
419     moonColours = [(111, 109, 114), (253, 245, 144), (68, 169, 241), (92, 88, 76), (70, 103, 97)]
420     ## The use of getEphemeris here is a complex API call
421     ephemeris = horizonsParser.getEphemeris(10) ## Adding the sun
422     planets.append(planet(planetNames[0], ephemeris[0], ephemeris[1], ephemeris[2], ephemeris[3], planetColours[0]))
423
424     for i in range(1, 10):
425         ephemeris = horizonsParser.getEphemeris(i*100+99)
426         ## This is adding the moon, it makes sense for it to be just after earth in the planet order
427         if i == 4:
428             moonEphemeris = horizonsParser.getEphemeris(301)
429             planets.append(satellite(moonNames[0], moonEphemeris[0], moonEphemeris[1], moonEphemeris[2], moonEphemeris[3], moonColours[0], planets[3]))
430             ## For some reason, Jupiter's mass is given in grams by NASA
431             ## Despite all other masses being given in kilograms
432             ## I do not know why
433             ## Dividing ephemeris[1] by 1000 resolves that problem
434         if i == 5:
435             ephemeris[1] = ephemeris[1]/1000
436             ## Here we add Jupiter's 4 largest moons; Europa, Io, Ganymede and Callisto (this is done at i=6 so that they are AFTER jupiter in the list)
437             ## Their IDs follow the pattern 501, 502, 503 etc so 500+j is used to generate them
438         if i == 6:
439             for j in range(1,5):
440                 moonEphemeris = horizonsParser.getEphemeris(500+j, True)
441                 planets.append(satellite(moonNames[j], moonEphemeris[0], moonEphemeris[1], moonEphemeris[2], moonEphemeris[3], moonColours[j], planets[6]))
442
443     planets.append(planet(planetNames[i], ephemeris[0], ephemeris[1], ephemeris[2], ephemeris[3], planetColours[i]))
444
445     # Adding a few prominent satellites (voyager 1&2)
446     ephemeris=horizonsParser.getEphemeris(-31, False, True, 722, 13) # Voyager 1, need to manually input mass and size as NASA does not provide it
447     planets.append(planet("Voyager 1", ephemeris[0], ephemeris[1], ephemeris[2], ephemeris[3], moonColours[0])) ## moonColours[0] is grey, spacecraft are grey, close enough
448     ephemeris=horizonsParser.getEphemeris(-32, False, True, 722, 13) # Voyager 2
449     planets.append(planet("Voyager 2", ephemeris[0], ephemeris[1], ephemeris[2], ephemeris[3], moonColours[0]))
450     return planets

```

Setup 2

This is the last bit of setup that is needed before calling the main function. The generated solar system is stored, Pygame is started and the screen is created.

```

451 planets = generateSolarSystem()
452
453 ## LINE_LENGTH is the amount of lines each planet can have so that the total amount of lines never exceeds the maximum lines
454 ## The -3 here is because of the moons, voyager 1 and 2, which are not usually visible and so will not cause extra lag from their orbital paths
455 LINE_LENGTH = int(MAX_LINES / (len(planets)-7))
456 pygame.init()
457 ## Screen is never edited and is needed by lots of functions so makes sense to have it as a global variable
458 screen = pygame.display.set_mode((0, 0), pygame.FULLSCREEN)
459 ## Centre is never edited and is needed by lots of functions so makes sense to have it as a global variable
460 centre = np.array(pygame.display.get_surface().get_size())/2
461

```

main

This is the main function, where everything actually happens.

```

462 def main():
463     global distScale
464     global zoomScale
465     global centre
466     global G
467     global screen
468     framerate = 120
469     ## Time-scale, how much vel and position should change per tick
470     ## Lower value = slower but more accurate simulation
471     year = 31536000/framerate
472     timeScale = 0.05*year
473
474     ## Centre of screen
475     clock = pygame.time.Clock()
476     started = False
477     running = True
478     ## List of arrows to be drawn
479     arrowsToDraw = []
480     ## What planet the camera should center on
481     focus = 0
482     comFocus = False
483     ## Whether or not to draw arrows
484     arrows = False
485     ## Whether or not we're in comparison mode
486     comparison = False
487     ## Whether to let the user change attributes
488     changingAttributes = False
489     planetsToCompare = [0, 6]
490     ## Creating the surfaces for the planets being compared to be drawn on
491     comparisonSurface1 = pygame.Surface((centre[x]-3, centre[y]*2))
492     comparisonSurface2 = pygame.Surface((centre[x]-3, centre[y]*2))
493     ## These are the two fonts that will be used in the menus
494     font = pygame.font.SysFont('codenewroman', 18)
495     bigFont = pygame.font.SysFont('codenewroman', 22)
496     ## Menu surface 1 is for displaying the energies and momentum etc
497     menuSurface1 = pygame.Surface((400, 200), pygame.SRCALPHA)
498     ## Menu surface 2 is for displaying planet info
499     menuSurface2 = pygame.Surface((400, 200), pygame.SRCALPHA)
500     ## Make them slightly transparent so that it looks nicer
501     menuSurface1.set_alpha(128)
502     menuSurface2.set_alpha(128)

```

```

503     ## Display a start screen that also explains all the keybinds
504     while not started:
505         screen.fill((0,0,0))
506         for event in pygame.event.get():
507             if event.type == pygame.QUIT:
508                 running = False
509
510             if event.type == pygame.KEYDOWN:
511                 ## If the user presses enter, start the program
512                 if event.key == pygame.K_RETURN:
513                     started = True
514
515         displayStartScreen(bigFont)
516
517     while running:
518         screen.fill((0,0,0)) ## Make the background black
519         for event in pygame.event.get(): ## Handle inputs
520             if event.type == pygame.QUIT:
521                 running = False
522
523             if event.type == pygame.KEYDOWN:
524                 if event.key == pygame.K_RIGHT:
525                     if comparison:
526                         planetsToCompare[1] = (planetsToCompare[1]+1) % len(planets)
527                     else:
528                         focus = (focus+1)%len(planets)
529                 elif event.key == pygame.K_LEFT:
530                     if comparison:
531                         planetsToCompare[0] = (planetsToCompare[0]+1) % len(planets)
532                     else:
533                         focus = (focus-1)%len(planets)
534
535                 elif event.key == pygame.K_UP:
536                     if timeScale == 0:
537                         timeScale = 0.05*year
538                     else:
539                         timeScale += 0.1*year
540                 elif event.key == pygame.K_DOWN:
541                     timeScale -= 0.1*year
542                     ## Don't want negative time
543                     if timeScale < 0:
544                         timeScale = 0
545                 elif event.key == pygame.K_SPACE:
546                     ## Toggles whether or not the screen is centered on the centre of mass
547                     comFocus = not comFocus
548                 elif event.key == pygame.K_f:
549                     arrows = not arrows
550                 elif event.key == pygame.K_c:
551                     comparison = not comparison
552                 elif event.key == pygame.K_a:
553                     changingAttributes = not changingAttributes

```

```

553     ## G is very small, but users might want very large values of G.
554     ## To achieve this, I make the increments G go up in scale with the order of magnitude of G.
555     ## The same thing is done with masses of planets, as a very different increment is needed
556     ## for something the size of a satellite compared to something the size of our sun
557     ## This also has the bonus of making it impossible to make G or a mass 0 or negative, which prevents division by zero
558     ## and other peculiar bugs
559     elif event.key == pygame.K_w:
560         if changingAttributes:
561             G += 0.5*10**maths.floor((maths.log(G, 10)))
562     elif event.key == pygame.K_s:
563         if changingAttributes:
564             G -= 0.5*10**maths.floor((maths.log(G, 10)))
565     elif event.key == pygame.K_e:
566         if changingAttributes:
567             oldMass = planets[focus].getMass()
568             if not comFocus:
569                 planets[focus].addMass(0.5*10**maths.floor(maths.log(planets[focus].getMass()+1,10)))
570     elif event.key == pygame.K_d:
571         if changingAttributes:
572             if not comFocus:
573                 planets[focus].addMass(-0.5*10**maths.floor(maths.log(planets[focus].getMass()+1,10)))
574
575     if event.type == pygame.MOUSEWHEEL:
576         if event.y == 1:
577             if zoomScale < distScale: ## Linear interpolation to bring zoomScale much closer to distScale if it's less than it
578                 zoomScale = zoomScale*0.3 + distScale*0.7
579             zoomScale += 0.4*zoomScale
580         if event.y == -1:
581             if zoomScale > distScale: ## Linear interpolation to bring zoomScale much closer to distScale if it's greater than it
582                 zoomScale = zoomScale*0.3 + distScale*0.7
583             zoomScale -= 0.4*zoomScale
584
585     ## Works out gravitational force between all planets and moves them accordingly each tick
586     simulateTick(planets, focus, timeScale)
587     distScale = zoom(distScale, zoomScale)
588
589     if comparison: ## if comparison is true
590         ## These rects get rid of old drawings, similar to doing screen.fill((0,0,0)) to refresh the display
591         pygame.draw.rect(comparisonSurface1, 'black', (0, 0, centre[x]*2, centre[y]*2))
592         pygame.draw.rect(comparisonSurface2, 'black', (0, 0, centre[x]*2, centre[y]*2))
593         drawPlanet(planets[planetsToCompare[0]], np.array([centre[x]/2, centre[y]]), comparisonSurface1)
594         drawPlanet(planets[planetsToCompare[1]], np.array([centre[x]/2, centre[y]]), comparisonSurface2)
595         screen.blit(comparisonSurface1, (0, 0))
596         screen.blit(comparisonSurface2, (centre[x]+3, 0))
597         pygame.draw.line(screen, "blue", [centre[x],0], [centre[x],centre[y]*2], 6)
598         ## Display info on both selected planets
599         displayPlanetInfo(font, planets[planetsToCompare[0]], planets, 20, menuSurface2)
600         displayPlanetInfo(font, planets[planetsToCompare[1]], planets, centre[x]*2-320, menuSurface2)
601
602     else: ## Display the solar system if not in comparison mode
603         ## focusAdjustment makes it so that the screen follows whichever planet the user wants to look at
604         ## Alternatively, follows the centre of mass, useful for binary star systems
605         adjustment = focusAdjustment(planets, focus, comFocus)
606         displayPlanets(planets, adjustment, screen)
607         displayLines(planets, adjustment, focus, comFocus, screen)
608         if arrows:
609             displayArrows(planets, adjustment, focus, comFocus, screen) # Draw arrows if arrows is true
610
611         ## Properties includes the key physical attributes of the system, potential energy, kinetic energy, momenteu
612         properties = sumPhysicalProperties(planets)
613         ## Displaying the menu
614         displayAttributeInfo(font, changingAttributes, properties, 20, menuSurface1)
615         ## Display info on current planet as requested by client
616         if not comFocus:
617             displayPlanetInfo(font, planets[focus], planets, centre[x]*2-320, menuSurface2)
618
619     pygame.display.flip() # Update the display
620     clock.tick(framerate) # Move onto the next tick
621
622 if __name__ == "__main__":
623     main() # Run the program!

```

Testing

The evidence for the tests will primarily be in the form of a video which is accessible here:

<https://youtu.be/DvnON3rZwGo> and also here:

https://drive.google.com/file/d/1aJfv_tlzvrYxwAwLpiNIksReTSqx1Ghb/view?usp=sharing

However the first two tests for objectives 1aii and 5 will be evidenced with screenshots, as there is nothing animated that needs to be shown. The screenshots can be found below the testing table

Objectives 1aii and 5 were implemented with separate files and then those files were imported as modules into the main program, the functions in these files should be tested independently of the main program. The main() function for these files will be used to test them and demonstrate they work. Here is the code for test 0

```

97  ## Testing to see if the parser works before implementing it in main program
98  def main():
99      ephemeris = getEphemeris(399)
100      print("RADIUS:", ephemeris[0], "m")
101      print("MASS:", ephemeris[1], "kg")
102      print("POSITION:", ephemeris[2])
103      print("VELOCITY:", ephemeris[3])
104
105  if __name__ == "__main__":
106      main()

```

1:

And here is the code for test 02:

```

30  ## Main is just used for testing purposes
31  def main():
32      print("DOT PRODUCT BETWEEN (-1,-2), (3,4):", dot(np.array([1, 2]), np.array([3, 4])))
33      print('---')
34      print("MAGNITUDE OF (3,4):", mag(np.array([3, 4])))
35      print("UNIT OF (3,4):", unit(np.array([3,4])))
36      print("NORMAL TO (3, 4):", normal(np.array([3, 4])))
37      print("DOT BETWEEN NORMAL TO (3,4) AND (3,4) SHOULD BE ZERO:", dot(np.array([3, 4]), normal(np.array([3, 4]))))
38      print('---')
39      print("MAGNITUDE OF (-1, -2):", mag(np.array([-1,- 2])))
40      print("UNIT OF (-1, -2):", unit(np.array([-1,- 2])))
41      print("NORMAL TO (-1, -2):", normal(np.array([-1,- 2])))
42      print("DOT BETWEEN NORMAL TO (-1, -2) AND (-1, -2) SHOULD BE ZERO:", dot(np.array([-1, -2]), normal(np.array([-1,- 2]))))
43
44  if __name__ == "__main__":
45      main()

```

The tests have been chosen in an order that allows me to do most of them in a single run of the program. I've tried to also make the order sensible, i.e. The first test for the main program is with the start screen, as that's the first thing you'll see when you open the program. Some tests will inevitably destroy the solar system, and some tests require the solar system to have not been running for very long (i.e. The ones with NASA, I should ideally start my simulation and their simulation at the same time to make it a fair comparison.) I will do the tests (apart from the first two which are just screenshots) in four sections, tests 3-9, then I restart to do the comparisons with NASA's simulation (this requires changing the size of the program's window to have it side by side with NASA so must be done as a separate test), then I do tests 11-20 and then after test 20 the solar system will have been destroyed so I will restart again to do tests 21-24. I will

record 4 separate videos for these tests and then edit them all together to create a smoother viewing experience for the examiner.

I have done my best to explain what inputs I will be doing when carrying out each test. Note that I will scroll up and down to zoom in and out to make the tests easier to see, I have not added the scrolling to every input section as it's unnecessary and would clutter it a lot.

Test number	Objective tested	Description	Input data / event	Expected results	Actual results	Timestamp in video	Date
01	5 – Use JPL's Horizons system to automatically import data on planets via API requests, 5a – parse the data and extract relevant values	I will run the horizonsParser file by itself which will get the ephemeris for Earth and print the extracted values for mass, radius, position and velocity. These values can then be checked against an ephemeris generated using NASA's website, and with a search engine.	N/A	The values given by my parser should match those given by NASA and google.	As expected	N/A	2024-12-10
02	1aii – Necessary vector operations should be implemented in a separate file	I will run the vectors file by itself. This will run a test that tests all 4 functions (dot product, magnitude, unit and normal) on the test vectors (-1,-2) and (3,4) and print the results of these operations on the test vectors. These are chosen because 3,4 is a pythagorean triple so the unit and magnitude are nice values instead of being irrational and (-1, -2) tests if it handles negative	N/A	The results printed by my program should match the mathematically correct answers.	As expected	N/A	2024-12-10

		elements correctly.					
03	2a - a start screen that explains keybinds	The start screen should be displayed	N/A	A start screen displaying keybinds and telling the user to press "Enter" to start the program will appear.	As expected	00:12	2024-12-14 (my birthday!)
04	3 – display the solar system, 5b – Use data from NASA to load the real solar system via dynamic object creation	The user should be able to start the program with a button press and this should display the solar system.	"Enter" key	Upon pressing enter the program should hide the start screen and display the solar system. If the solar system is being displayed that means the data from NASA was successfully used to load the solar system as I have not hardcoded anything in that the data from NASA doesn't get imported.	As expected	00:22	2024-12-14
05	3b – Orbital lines should be shown to demonstrate the path of the planets	The path of planets should be traced out by a line matching their colour.	N/A	Ensure that planets trace out a coloured path.	As expected	00:27	2024-12-14
06	1c – energy and momentum should be conserved, 2ci – the menu should display kinetic, potential, total energy and momentum, 2b – The program should explain Newtonian physics by demonstrating key concepts like energy conservation	Energy and momentum should be calculated and displayed, total energy and momentum should not change. Conservation is the single most important part of astrodynamics, if energy and momentum are conserved in my program this is a very, very strong indication that the physics is correct.	N/A	There should be numbers in the top left of the screen showing energies and the momentum, and the total energy and momentum should stay constant and never change.	As expected	00:31	2024-12-14
07	3c – The user should be able to focus the camera on any planet of their choosing or the	Should be able to switch focus by pressing the arrow keys and focus on the c.o.m. by	Right arrow key x23, left arrow key x3	Pressing the right arrow key should cycle through the planets and eventually loop back to the sun when	As expected	00:57-04:36	2024-12-14

	center of mass	pressing space.		pressed enough times. Pressing the left arrow key should cycle in the other direction.			
08	2cii – information about planet from a reliable source should be shown, 2ciii – the information should be stored as attributes of a class (if 2cii is successful that means 2ciii works as 2cii is displayed using getter methods of the class)	Information about a planet should be displayed in the top right, can be checked against real values from NASA's site	N/A	The mass, radius and distance from the sun of a planet should be displayed in the top right. These values should match the values shown by a search engine. I will do this while cycling through planets in test 07, showing the NASA fact files for the first 4 planets (mercury, venus, earth, mars) compared to what is displayed by my program. I only do the first 4 planets as showing it for all the planets would take ages and be very tedious	As expected	01:05-04:04	2024-12-14
09	3d – The user should be able to zoom in and out	It should be possible to zoom in and out of the solar system to see more of it or to get a more detailed view of one specific part of it	Scroll up, scroll down	Earth should be selected following the previous test. Scrolling up should zoom in and make the Earth-Moon system visible, scrolling down should zoom out and make the rest of the solar system visible.	As expected	04:38-04:55	2024-12-24
10	3a – Accurately display the solar system with a scale model, 5b – Use data from NASA to load the real solar system with dynamic object creation	Compare my solar system side by side with NASA's Eyes on the Solar System model, to ensure that the model is accurate. If they look similar this shows my model is accurate.	N/A	My program should match what is shown my NASA in terms of scale (note that NASA uses different colours than my model does, e.g. Mercury is purple in their model but grey in mine, this has no effect on the accuracy of the model). If it is accurate then this is an indication that the data from NASA I used was accurate and the solar system	As expected	04:56-06:40	2024-12-24

				was loaded correctly, and if they both look similar after running the simulations for a while this is a strong indication my physics is implemented correctly.			
11	1a – Model the effects of gravity using Newton's laws of motion, 1ai – Laws should be implemented as methods in the celestialBodies class, 1b – Calculate the motion of objects using verlet integration, 1bi – Verlet integration should be implemented as methods in the celestialBodies class	The paths of planets should look sensible and match physical intuition, i.e. They should be roughly circular / elliptical around the sun, and moons should orbit their hosts.	Right arrow key a lot to cycle through planets, scrolling to zoom in and out	Planets should orbit the sun in ellipses, moons should orbit their host planets. If this works then this is a strong indication my program is accurate. I will check multiple planets as well as moons (I will zoom in to display the moons)	As expected	06:42-07:37	2024-12-24
12	4 – Include the ability to compare sizes of planets	The comparison mode should show planets side by side at their correct relative sizes	C, left arrow, right arrow x5, left arrow x5	Upon pressing C the sun and Jupiter should be in comparison. Pressing the left and right arrows should cycle through the planets to compare different bodies.	As expected	07:40-08:33	2024-12-24
13	4a – The menu should display info on both planets being compared	The comparison mode should display the information of the left planet in the left corner and the information of the right planet in the right corner	N/A	Upon pressing C, information about the sun (its mass, size, and distance from itself) should be in the top left corner, and information about Jupiter should be in the right corner. This should change to show information about the other planets as I change which planets are being compared.	As expected	07:40-08:33	2024-12-24

14	2bi – Force diagrams should be included to explain the motion of the planets using Newtonian physics	When pressing “F” to turn on arrows, arrows showing the forces on planets should pop up on the currently selected planet	F, Right arrow x6	Arrows should appear on one planet and point towards the other planets, for most planets the resultant arrow (the one that’s the colour of the planet) should point towards the Sun. I will turn on arrows then press the right arrow key a few times to show the arrows working on different planets.	As expected	08:35-09:30	2024-12-24
15	2d – Users should be able to speed up and slow down the simulation	When pressing the up and down arrow keys the simulation should speed up and slow down	Up arrow x4, down arrow x5, down arrow key x3, up arrow key	When pressing the up arrow key the simulation should speed up, when pressing the down arrow key the simulation should slow down. After pressing the down arrow key 5 times the simulation should be paused. Trying to press it again should do nothing, the simulation should not enter negative time as that is nonsense. Pressing up arrow should then resume the simulation.	As expected	09:35-10:21	2024-12-24
16	2civ – Users should be able to press A to edit attributes	Pressing A should enter a mode that lets users edit attributes, and the menu should change to explain the keybinds to do that	A	Pressing A will change the menu into something that explains the keybinds needed to change attributes	As expected	10:21	2024-12-24
17	2civ – Users should be able to press A to edit attributes, 2civ2 – Users should be able to edit the mass of planets	Decreasing the mass of a planet should have a visible effect.	A, Dx10	I will decrease the Earth’s mass, this should be reflected in the display that shows its mass, and it should cause the moon’s orbit to become wider until Earth loses hold of	As expected	10:31-11:00	2024-12-24

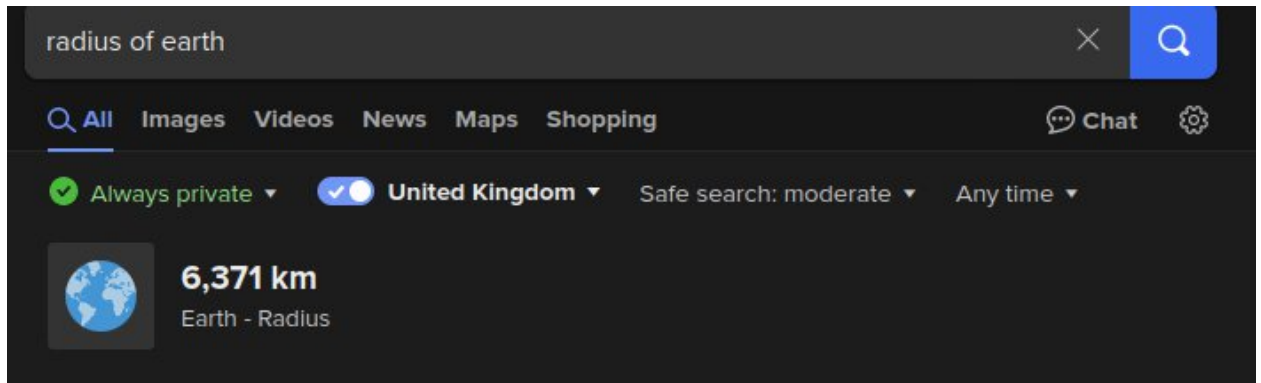
				the moon and the moon is flung out of its orbit.			
18	2civ – Users should be able to press A to edit attributes, 2civ2a – The mass of planets should never be negative	It shouldn't be possible to make the mass negative	Dx40	This is just to demonstrate that the logarithmic scale works, every time the mass of Mars decreases by an order of magnitude the amount it decreases every time D is pressed should change too, so that it can never become negative.	As expected	11:00-11:24	2024-12-24
19	2civ – Users should be able to press A to edit attributes, 2civ2 – Users should be able to edit the mass of planets	Increasing the mass of a planet should have a visible effect	Too many E presses to count	I will select Saturn and increase its mass until its mass is around that of the sun. This should cause the inner solar system to drift towards Saturn, and should significantly disturb the orbit of the other planets.	As expected	11:30-12:47	2024-12-24
20	3c – The user should be able to focus the camera on any planet of their choosing, or on the center of mass	With the mass of Saturn having been significantly increased, the center of mass of the solar system should now be roughly half way inbetween the Sun and Saturn, so pressing space should put the camera inbetween them.	Space x2	Pressing space once should toggle the focus to be on the center of mass, and it should be inbetween Saturn and the Sun. Pressing space again should change the focus to be back on the sun.	As expected	12:16	2024-12-24
21	2civ – Users should be able to press A to edit attributes, 2civ1 – Users should be able to edit the value of G	Making G bigger should have a visible effect	A, W x10	Pressing A followed by pressing W should increase the value of G. Pressing W 10 times should make gravity substantially stronger, which would cause planets to enter more elliptical orbits or fall into the sun and get flung out of the solar system.	As expected	13:00-13:15	2024-12-24

22	2civ – Users should be able to press A to edit attributes, 2civ1 – Users should be able to edit the value of G	Making G smaller should have a visible effect	S x10	Pressing A followed by pressing S should decrease the value of G. This should make gravity substantially weaker which should make planets exit their orbits, as gravity will not be strong enough to hold them in their orbit anymore.	As expected	13:15-13:25	2024-12-24
23	2civ – Users should be able to press A to edit attributes, 2civ1a – G should never become negative	It shouldn't be possible to make G negative	S x40	This is just to demonstrate that the logarithmic scale works, every time G decreases by an order of magnitude the amount it decreases every time S is pressed should change too.	As expected	13:30-13:42	2024-12-24
24	6 – Be sufficiently optimised and not suffer frame drops	This will be evident by just looking at the program for the duration of the testing, if there are no frame drops while doing all the other tests then this is a good indication that my program performs well. I will be running the program on my 15 year old laptop, so if it runs well on that it should run well on any modern computer.	N/A	There shouldn't be any frame drops while testing the program and the motion of everything should look smooth.	As expected	The whole duration of the video, it runs well for all of it	2024-12-24

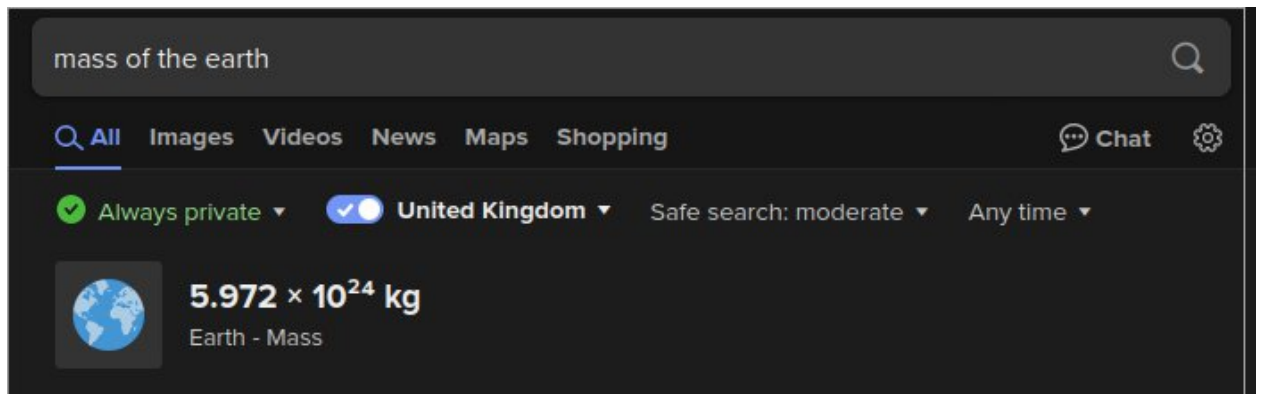
Here are the results of test 01:

```
[Artix Linux: Planets]$ python horizonsParser.py
RADIUS: 6371010.0 m
MASS: 5.97219e+24 kg
POSITION: [2.95660058e+10 1.43421441e+11 1.89141844e+07]
VELOCITY: [-2.96167447e+04 6.02595529e+03 -1.50696999e+00]
```

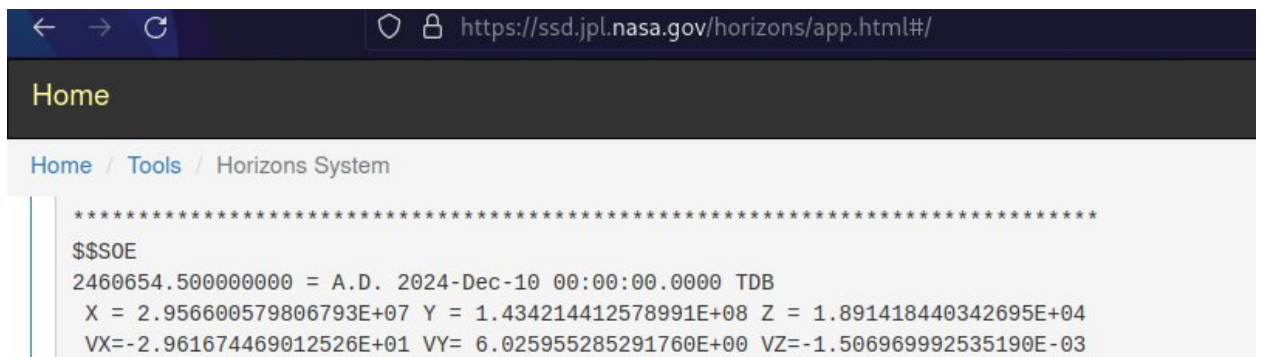
The radius of the earth is 6371km which is 6371000 so the first result is correct.



The mass of the earth is 5.792×10^{24} so that's correct



And the positions and velocities match the NASA ephemeris results:



Note that instead of listing them as X =, Y = etc I have put them into vectors. Top row in this screenshot is the position, bottom row is the velocity.

Here are the results of test 02:

```

[Artix Linux: Planets]$ python vectors.py
DOT PRODUCT BETWEEN (-1,-2), (3,4): -11
---
MAGNITUDE OF (3,4): 5.0
UNIT OF (3,4): [0.6 0.8]
NORMAL TO (3, 4): [-0.8 0.6]
DOT BETWEEN NORMAL TO (3,4) AND (3,4) SHOULD BE ZERO: 0.0
---
MAGNITUDE OF (-1, -2): 2.23606797749979
UNIT OF (-1, -2): [-0.4472136 -0.89442719]
NORMAL TO (-1, -2): [ 0.89442719 -0.4472136 ]
DOT BETWEEN NORMAL TO (-1, -2) AND (-1, -2) SHOULD BE ZERO: 0.0

```

These are the mathematically correct results. The dot product between $(-1, -2)$ and $(3,4)$ is $-1*3 + -2*4 = -3-8 = -11$, which is correct. The magnitude of $(3,4)$ is $\sqrt{3^2+4^2}=\sqrt{25}=5$, and so on. Every result here is correct so the test is successful.

Evaluation

Overall evaluation

I think my project meets the objectives of being a good educational tool for physics. When showing my program to friends they have frequently said that it's "really cool" and that they found it very fun to watch the planets move about and see the solar system get destroyed when you changed the mass of planets. This is good as engaging children is an important part of education, so it's good that other students seem to find the program interesting.

However, the main focus was on the educational aspect. Not the cool factor. I have received detailed feedback from the physics department at my school, which were the initial client that I interviewed for my analysis. This is what was said (blue responses are the client):

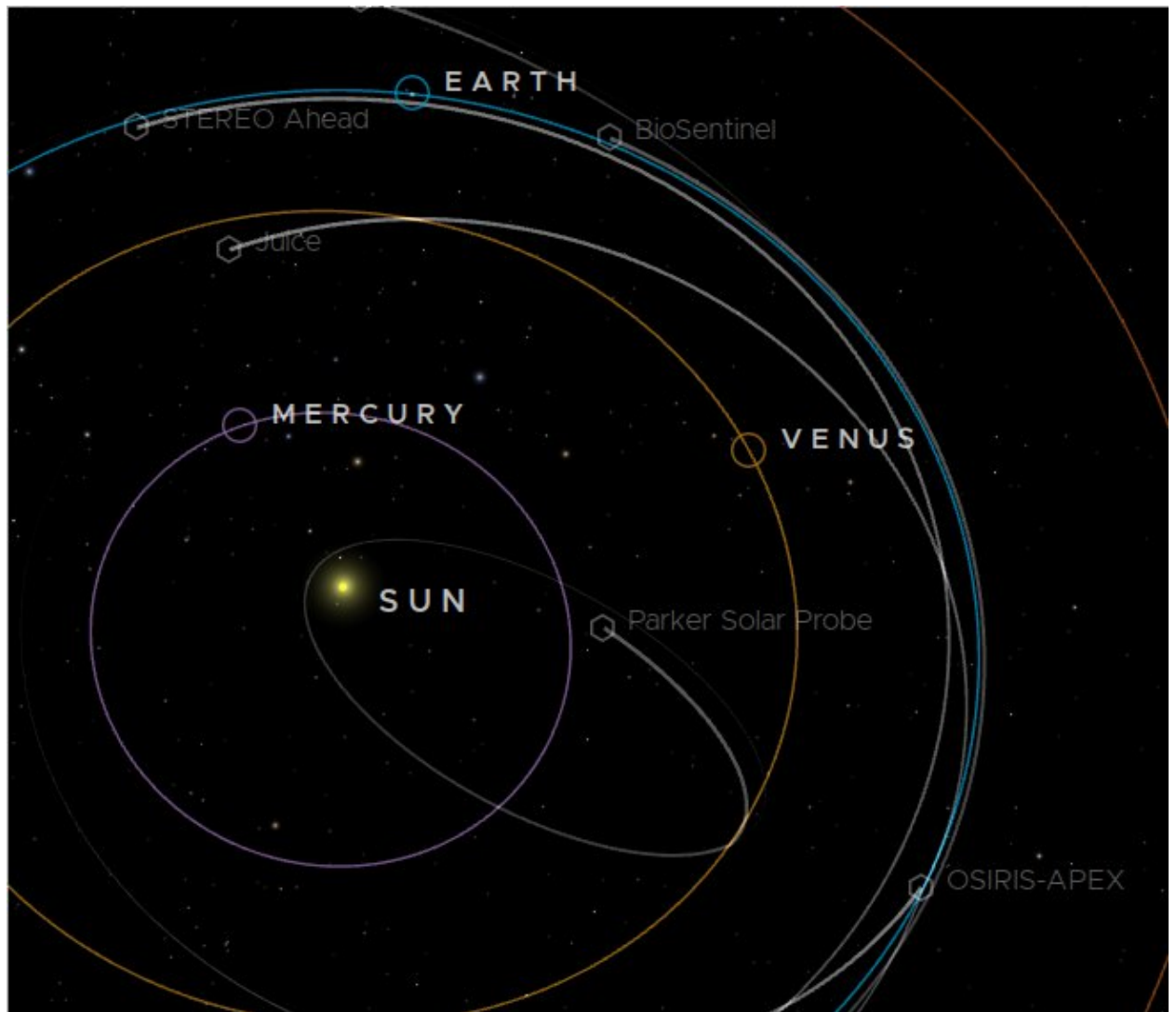
1. Do you think the program does an adequate job of being a useful educational tool for the solar system? Is there anything you think it did particularly well or that it could improve on? It provides a reliable source of solar system data so that students can research information about the planets easily. The animation clearly demonstrates the difference in orbital speeds which is something we need to explain at GCSE
2. Do you think the program did a good job of connecting the solar system to other topics in physics e.g. energy and newtonian mechanics? Using the data to perform calculations provides a good link to Newtonian mechanics. I do like that it shows the GPE and KE too. This is more suited to the A level course. At GCSE there is a requirement to understand centripetal force and the affect that has on the speed of an object in orbit, also the basic link between mass and weight.
3. Do you think the interactivity available in the program was useful for teaching purposes? Is there any extra ways of interacting with the solar system that you think would be helpful? As a teaching tool for younger students, having an interesting fact file for each planet would be a nice addition. I like that you can investigate changing parameters and how that affects the solar system – the mechanics here is more suited to A level teaching
4. Do you think the program ran well? Were there any drops in performance or bits of lag that you think would harm the user experience? I did think the program appeared to run well. It was easy to 'destroy' the solar system by outing in unsuitable parameters – I quite like that as it shows how improbable it is to get the exact solar system that we have – everything needs to be just right.
5. Was the information displayed by the program useful and is there anything else I should display? There was an excellent amount of detail available when you selected each planet. It would be good to have an information section where it outlines the calculations to get some of the changing data.
6. Are you happy with the physical accuracy of the program? Yes – I particularly like the fact you did not just use the average solar-planet distances.
7. Were you happy with the user experience? Is there anything you liked, and any parts that you didn't like that I could improve? It appeared easy to use and to move between places in the solar system. There are quite a lot of commands to remember so having them displayed more prominently or a info button to click on might be a good addition. The planets themselves are small and so pretty difficult to see – at the moment I'm not sure it could be projected for the whole class to see.
8. Can you see yourself using this program in a lesson in its current state, or would it need more improvements to be suitable for a lesson? I could see that we could use it in the teaching of the gravity topic to Y12, especially as they each have a laptop and so could display it on their own screens.

Overall the feedback is very positive. They say that the program does a good job at teaching the solar system and linking it to Newtonian mechanics. They are also happy with the physical accuracy of my program which indicates that the complex scientific models used in my program to model gravity work well, and was happy with the performance of the program. Note that in the response to question 2 they mention centripetal force, my program does show centripetal force through the force arrows you can turn on, but this is something I forgot to mention when first showing the program to the client. I have received feedback from them that the force diagram adequately shows the concept of centripetal force to students, and also that the size comparison included was very useful, so ignore the original answer to question 2.

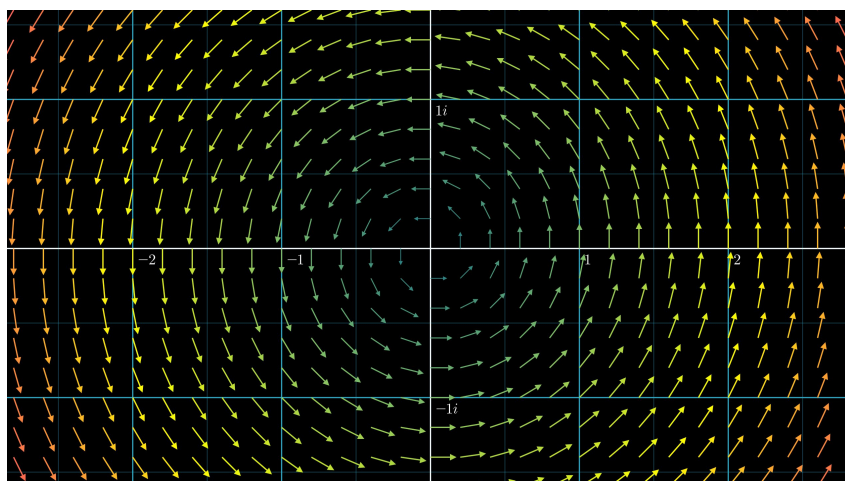
They are happy with the interactivity of the program, but think there is some room for improvements. They requested an interesting fact file for each planet, I could implement a button to bring up a fact file for the currently selected planet. I would write the fact files myself using suitable sources.

They say that the amount of detail displayed when selecting a planet was very good. However it was requested that some extra information be displayed – it should show the calculations used to get the changing data and there should be something like an info button to explain all the commands for manipulating the program. For the calculations, I would probably do something like show the intermediate steps, e.g. Rather than just displaying the kinetic energy I could display the total velocity of all the planets in the system, and show the kinetic energy for each planet, and then sum it up to show the total kinetic energy. For the info button this should bring up something that looks similar to the start screen where all the keybinds are explained. One other thing that isn't mentioned but may improve the ease of use of my program is by adding more mouse usage. At the moment everything is done through keybinds, it may be easier if some functionality used the mouse, e.g. A slider could be implemented to change the mass of planets.

It is also mentioned that it is rather difficult to see the planets, which I agree with. Something NASA does in their model of the solar system is draw circles around the planets and label them by name, shown below. Implementing something along these lines may help with the visibility problem.



One feature not mentioned by my client but that I think may be useful would also be implementing something to show the overall gravitational field in the solar system. Fields are a difficult topic in A level physics that people often struggle with, so a tool to help teach them would be quite helpful. For displaying them I could use something like the method used by 3blue1brown in his videos. Because making the strength directly proportional to size would lead to some very big and small arrows, he instead uses colour to indicate the strength of the vectors at different points, like this:



Using this kind of method to display the gravitational field across the solar system could be very useful for teaching fields to A level students.

In conclusion the project overall does very well at being a good educational tool, but some more information could be added to the display such as fact files and explanations of the calculations, and the user experience could be improved by adding buttons / displaying the keybinds more prominently. Something to display the field in the solar system would be useful too.

Optimisation

The program runs fairly well thanks to the vectorised code in `displayLines()` and the trick used in `simulateTick()` where we only consider planets after the current planet in the list of `celestialBodies`. The fact that it runs well is evident in the testing video, there are no frame drops and the program runs smoothly even though it's simulating the effects of gravity between 17 different bodies at the same time. The way the lines are drawn is still slightly inefficient though, they are redrawn a lot because I want them to fade, lines are re-drawn hundreds of times until they're cut from the record of positions for the planet. The fade effect is very detailed, it would probably not be noticeable if I didn't go through all 255 alpha values. This would mean I would only need to re-draw lines every few ticks when they changed colour, this would make the program more efficient and mean I could have more moons and other satellites in the system.

The one issue is the load times, which are rather long, as is seen at the start of the testing video. This is due to NASA taking a while to respond to requests, which may be because they're rate limiting me. One way to fix this would be to cache the responses from NASA each day, as they only change once a day. This would stop the program from taking a while to load.

Evaluation by objective

Objective	My evaluation	Feedback from client	Potential improvement
1 – Be physically accurate	Fully met.	Client said she was satisfied with the accuracy of the program.	N/A
2a – Have a start screen that explains the keybinds	Objective met.	Client requested a way to see the keybinds while using the program as well.	Add a button to show the keybinds while using the system.
2b – Explain how Newtonian physics connects to the solar system	Fully met.	Client said she thought that using the data to do calculations provided a good link to Newtonian mechanics	
2c – Have a menu that displays stats and lets the user edit certain values	Objective met.	Client said she thought there was an excellent amount of detail available, but some extra information on the planets and the	Add a fact file and a section showing the calculations used to get changing data as requested by client when providing feedback.

		calculations done would be good.	
2d – Let users speed up and slow down the simulation	Fully met.	N/A	N/A
3 – Accurately display the solar system with a scale model	Objective met, planets can be hard to see though.	Client said it was easy to use and move between places in the solar system (objective 3c) but that the planets were small and it wouldn't be suitable to project for the whole class to see.	Add circles / labels around planets as in NASA's Eyes on the Solar System
4 – Include the ability to compare sizes of two planets	Fully met.	Client said verbally that she was very satisfied with the comparison mode.	N/A
5 – Access NASA JPL's Horizons system to automatically import up to date data on planets via API requests	Objective met. Use of the database has led to long load times however.	Client liked that there was a reliable source of solar system data for the students.	Some form of caching to alleviate long load times.
6 – Be sufficiently optimised	Objective met. Room for improvement as explained in the section on optimisation	Client said she thought the program ran well	See the section optimisation for how performance could be further improved by changing the way lines are displayed.

References

Alternative programs mentioned in analysis (the NASA one, Eyes on the Solar System, is also used in testing to compare with my simulation):

- *Eyes on the Solar System* - NASA/JPL (no date). <https://eyes.nasa.gov/apps/solar-system/#/home>

- *My Solar System - PhET* (no date). https://phet.colorado.edu/sims/html/my-solar-system/latest/my-solar-system_en.html

The version of Newton's law of gravitation used in the analysis and design (and code) comes from this problem sheet:

- *Year 13 Physics Assignment 02 - COMPOS/University of Oxford* (no date). <https://compos.web.ox.ac.uk/sitefiles/physics13-02.pdf>

The database used throughout this project:

- *Horizons System - NASA/JPL* (no date). <https://ssd.jpl.nasa.gov/horizons/>

The fact files used in the testing to check the data displayed in test 08 can be found here:

- *NASA Space Science Data Coordinated Archive - NASA/JPL* (no date). <https://nssdc.gsfc.nasa.gov/planetary/factsheet/>

Resources I used to help implement Verlet integration:

- *Verlet integration - Wikipedia* (no date). https://en.wikipedia.org/wiki/Verlet_integration
- *Writing a physics engine from scratch – Pezzza's work/Youtube* (2022/04/15). https://www.youtube.com/watch?v=IS_qeBy3aQI

The pygame documentation was used extensively when building this project:

- *Pygame Front Page - pygame v2.6.0 documentation* (no date). <https://www.pygame.org/docs/>

The image used when talking about fields in the evaluation comes from this 3blue1brown lesson:

- *e^{ip}i in 3.14 minutes, using dynamics - 3blue1brown* (2019/07/07). <https://www.3blue1brown.com/lessons/eulers-formula-dynamically>